



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE CIENCIAS

Fundamentos Matemáticos para Programación Competitiva
Manual de Apoyo a la Docencia para ICPC

REPORTE DE ACTIVIDAD DOCENTE

QUE PARA OBTENER EL TÍTULO DE

Licenciado en Matemáticas

PRESENTA

Mariana Itzel Melo Tellez

TUTOR

Dr. Manuel Alcántara Juárez



Ciudad Universitaria, Ciudad de México, 2026.

Agradecimientos

Escribe aqui tus agradecimientos.

Índice general

1. Introducción	6
2. Introducción a la Programación Competitiva	7
2.1. ¿Por qué hacer programación competitiva?	9
2.1.1. Beneficios Académicos	9
2.1.2. Oportunidades laborales	10
2.2. Otros concursos y plataformas	10
2.2.1. Ejercicios	10
3. Jueces en Línea	11
3.1. ¿Qué hace un juez en línea?	11
3.1.1. De la propuesta al veredicto	11
3.2. Arquitectura general	12
3.3. ¿Cómo compara las respuestas?	13
3.4. Clarificaciones durante un concurso	13
3.5. Algunos jueces y plataformas en línea	14
3.6. Una observación importante	14
4. Entrada/Salida	15
4.1. Primer contacto con la entrada y la salida	15
4.2. Entrada estándar, salida estándar y funciones dadas	15
4.3. Patrones comunes de entrada	16
4.3.1. Unicaso	16
4.3.2. Multicaso específico	16
4.3.3. Multicaso controlado por valores	17
4.3.4. Multicaso controlado por EOF	17
4.3.5. Multicaso con salida numerada	18
4.3.6. Multicaso con datos en una línea	18

4.3.7. Opción de función	19
4.4. Una observación práctica	19
5. Álgebra	20
5.1. Recurrencias	22
5.1.1. Solucion DP	22
5.1.2. Solución Exponenciación de Matrices	22
5.2. Polinomios e interpolación de Lagrange	23
5.3. Transformada de Fourier rápida (FFT) y NTT	27
5.4. Ejercicios	30
5.5. Problemas	30
6. Teoría de números	31
6.1. Bases	31
6.2. Aritmética modular	33
6.3. Números primos y criba de Eratóstenes	34
6.4. Teorema de Bézout	36
6.5. Inverso multiplicativo modular	38
6.6. Congruencias lineales y Teorema Chino del Residuo	40
6.7. Funciones multiplicativas	42
6.8. Logaritmo discreto y Baby-step Giant-step	46
6.9. Ejercicios	47
6.10. Problemas	47
7. Combinatoria y conteo	49
7.1. Bases	49
7.2. Permutaciones	50
7.3. Combinaciones	51
7.4. Principio de Inclusión-Exclusión	53
7.5. Barras y estrellas	54
7.6. Lema de Burnside	55
7.6.1. Órbitas y puntos fijos	56
7.7. Números de Catalan	57
7.8. Números de Stirling de segundo tipo	58
7.9. Números de Bell	59
7.10. Ejercicios	59

7.11. Problemas	60
8. Geometría computacional	61
8.1. Bases	62
8.2. Intersecciones	64
8.3. Polígonos	64
8.4. Convex Hull	67
8.5. Algoritmos de barrido	68
8.6. Ejercicios	72
8.7. Problemas	72
9. Teoría de gráficas	73
9.1. Lema de los apretones de mano	73
9.2. Caminos y circuitos de Euler	74
9.2.1. Algoritmo de Hierholzer	74
9.3. Componentes fuertemente conexas	75
9.3.1. Algoritmo de Kosaraju	75
9.4. 2-SAT	77
9.5. Puentes y puntos de articulación	79
9.6. Árboles	81
9.7. Centroide	82
9.8. Ancestro común más bajo (LCA)	84
9.9. Flujo máximo	87
9.9.1. Ford-Fulkerson	88
9.9.2. Dinic	90
9.9.3. Teorema max-flow min-cut	93
9.9.4. Matching máximo	93
9.9.5. Teorema de König	96
9.9.6. Teorema de Hall	97

Capítulo 1

Introducción

Este manual es una guía de apoyo para fortalecer la preparación matemática en programación competitiva, con enfoque en el Concurso Internacional Universitario de Programación (ICPC). Está dirigido tanto a estudiantes de matemáticas que buscan iniciarse en esta área como a participantes que desean consolidar y ampliar sus herramientas teóricas.

Si bien existen libros de programación competitiva ampliamente reconocidos, gran parte de ellos se concentra en algoritmos, implementación y estructuras de datos. Este manual busca complementar ese panorama con un enfoque explícito en la formación matemática que requiere la competencia y con una propuesta adaptada al contexto académico de la Facultad de Ciencias.

Además de la exposición conceptual, el manual incluye un conjunto de problemas cuidadosamente seleccionados y también problemas de autoría propia, diseñados para reforzar de manera directa los conocimientos presentados en cada tema.

La organización del contenido responde a áreas temáticas, no a niveles de dificultad, con el fin de que cada lector avance de acuerdo con sus intereses y necesidades. En el Capítulo 2 se presenta una introducción general a la competencia ICPC, su dinámica y sus reglas principales; en el Capítulo 3 se abordan los fundamentos de entrada/salida y el uso de jueces en línea; y en el Capítulo 4 se desarrolla el bloque de teoría de números aplicado a resolución de problemas.

Al final del documento se incluyen las editoriales de los problemas presentados. Estas editoriales comienzan con *hints* para orientar el proceso de resolución y no muestran de inmediato la solución completa. Para obtener mejores resultados, se sugiere seguir este orden de trabajo: primero intentar resolver cada problema por cuenta propia, después apoyarse en los *hints* cuando sea necesario y, finalmente, consultar la editorial para comparar enfoques y comprender la solución propuesta.

Capítulo 2

Introducción a la Programación Competitiva

La programación competitiva suele describirse como un deporte mental: exige constancia, disciplina y entrenamiento estructurado. El objetivo no es solamente encontrar una solución correcta, sino hacerlo con precisión y en el menor tiempo posible.

Su filosofía se centra en la resolución eficiente de problemas clásicos de ciencias de la computación. En este contexto, los enunciados están diseñados para ser resolubles con técnicas y algoritmos conocidos; es decir, no se trata de problemas de investigación abierta, sino de retos que evalúan la capacidad de modelar, implementar y optimizar soluciones bajo presión de tiempo. Resolver un problema implica comprender el enunciado, elegir una estrategia algorítmica adecuada y traducirla a código en un lenguaje de programación. El componente competitivo aparece cuando, además de resolver correctamente, se debe hacerlo rápido y con la menor cantidad de intentos fallidos.

En comparación con hackatones u otros concursos de desarrollo, donde se privilegia la construcción de productos completos durante varios días, la ICPC y competencias similares se enfocan en problemas puntuales con criterios estrictos de correctitud, tiempo y memoria. El sitio oficial de la ICPC la describe como una competencia algorítmica universitaria. En la práctica, esto significa que cada solución enviada es evaluada automáticamente por un programa llamado juez en línea que verifica si el programa responde correctamente a todos los casos de prueba y respeta los límites establecidos. Esta dinámica es una diferencia clave frente a concursos con evaluación más abierta y en ocasiones algo subjetiva, como los hackatones.

La ICPC es actualmente la competencia universitaria más representativa en esta disciplina. Se compete en equipos de tres estudiantes de la misma institución y, según la región, el proceso se organiza en varias fases clasificatorias. En el caso de México, desde 2024 se introdujo una estructura de **cuatro etapas**; entre ellas, el Gran Premio de México, compuesto por tres fechas de competencia clasificatorias. En cada fecha se resuelve un conjunto de problemas, usualmente entre 9 y 13, en tiempo limitado y la clasificación considera tanto la cantidad de problemas resueltos como la penalización acumulada por tiempo e intentos incorrectos.

Para hacerlo más claro e ilustrativo, tomemos como ejemplo el problema clásico de Codeforces 231A.

A. Team

time limit per test: 2 seconds

memory limit per test: 256 megabytes

One day three best friends Petya, Vasya and Tonya decided to form a team and take part in programming contests. Participants are usually offered several problems during programming contests. Long before the start the friends decided that they will implement a problem if at least two of them are sure about the solution. Otherwise, the friends won't write the problem's solution. This contest offers n problems to the participants. For each problem we know which friend is sure about the solution. Help the friends find the number of problems for which they will write a solution.

Input

The first input line contains a single integer n ($1 \leq n \leq 1000$), the number of problems in the contest. Then n lines contain three integers each, each integer is either 0 or 1. If the first number equals 1, then Petya is sure about the problem's solution; otherwise he isn't sure. The second number shows Vasya's view, and the third number shows Tonya's view.

Output

Print a single integer: the number of problems the friends will implement on the contest.

Example

Input

```
3
1 1 0
1 1 1
1 0 0
```

Output

```
2
```

Como se observa en este ejemplo, usualmente todo problema de programación competitiva consta de las siguientes secciones:

- **Tiempo límite:** tiempo máximo de ejecución permitido.
- **Memoria límite:** memoria RAM máxima para la solución.
- **Descripción:** contexto y tarea específica a resolver.
- **Input (entrada):** formato exacto de los datos de entrada.
- **Output (salida):** formato exacto de la respuesta esperada.
- **Ejemplos:** casos de prueba para validar la interpretación.

Una pregunta valiosa para quien inicia es: *¿entiendo lo que me pide el problema?*; y después, *ya que entendí el problema, ¿puedo crear un código para resolverlo?*; finalmente, *¿cuánto tiempo me tomaría escribir un código correcto y sin errores?*. Esta autoevaluación ayuda a medir el progreso real en programación competitiva y, sobre todo, a construir confianza con cada intento. Incluso cuando la primera solución no sale a la perfección, cada ajuste mejora la lectura de problemas, la precisión del código y la velocidad de respuesta en concurso. Entre más problemas se resuelven, mayor "condición" se desarrolla para identificar patrones, tipos de enunciados, técnicas y trucos frecuentes; en consecuencia, cada vez resulta más natural acercarse a una solución correcta desde el primer intento.

La idea central para resolver el problema 231A es contar, en cada línea, cuántos valores son iguales a 1; si la suma es mayor o igual a 2, se incrementa el contador de problemas resueltos. Esta tarea permite practicar lectura de entrada, recorridos simples y condiciones lógicas. El código puede implementarse de forma directa:

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     int n;
6     cin >> n;
7     int implementados = 0;
8
9     for (int i = 0; i < n; i++) {
10         int a, b, c;
11         cin >> a >> b >> c;
12
13         // Si al menos dos están seguros, se implementa el problema.
14         if (a + b + c >= 2) {
15             implementados++;
16         }
17     }
18
19     cout << implementados << "\n";
20 }
```

En este ejercicio también queremos exponer un detalle importante: en la salida se imprime `"\n"`. Aunque parezca menor, el juez en línea suele comparar texto de forma estricta. Si el formato no coincide exactamente con lo esperado (por ejemplo, cuando el resultado debe ir en una línea y no se respeta), es posible recibir un veredicto **Wrong Answer** aun cuando la lógica sea correcta. Iniciar puede ser frustrante, pero después de superar esa curva de aprendizaje, este deporte se vuelve sumamente reconfortante.

2.1. ¿Por qué hacer programación competitiva?

2.1.1. Beneficios Académicos

La programación competitiva aporta beneficios académicos importantes para la formación en ciencias de la computación. En primer lugar, fortalece habilidades esenciales como el pensamiento algorítmico, el razonamiento lógico y la resolución eficiente de problemas. Al enfrentarse a ejercicios desafiantes, el estudiante aprende a descomponer tareas complejas en partes más pequeñas, detectar patrones y diseñar estrategias correctas y eficientes.

Además, funciona como un puente entre la teoría y la práctica. Conceptos vistos en clase –como estructuras de datos, análisis de complejidad, teoría de números o técnicas de optimización– dejan de ser abstractos cuando deben aplicarse para resolver problemas concretos bajo restricciones reales de tiempo y memoria.

Otro beneficio clave es que fomenta el aprendizaje continuo. Cada problema nuevo expone al estudiante a ideas distintas y lo motiva a investigar enfoques alternativos, comparar soluciones y refinar su forma de pensar. Con el tiempo, esto fortalece hábitos académicos valiosos: validar casos límite, justificar la correctitud de una solución y mejorar progresivamente la claridad del código.

2.1.2. Oportunidades laborales

La práctica constante en programación competitiva también tiene impacto directo en el perfil profesional. Muchas empresas tecnológicas evalúan a sus candidatos mediante entrevistas técnicas centradas en resolución algorítmica de problemas; por ello, entrenar este tipo de pensamiento representa una ventaja real en procesos de selección para pasantías y puestos de tiempo completo.

Durante estas evaluaciones no solo importa llegar a la respuesta correcta. También se observa cómo razona la persona candidata, cómo comunica sus ideas, cómo depura errores, cómo justifica la complejidad de su solución y cómo se adapta cuando cambian las condiciones del problema. Todas estas competencias se trabajan de forma natural en competencias de programación.

Finalmente, la experiencia en concursos también puede aportar visibilidad profesional: buenos resultados, constancia y participación activa en comunidades técnicas suelen reflejar iniciativa, disciplina y capacidad de trabajo intelectual bajo presión, cualidades altamente valoradas en la industria.

2.2. Otros concursos y plataformas

Además de la ICPC, existen competencias en las que puede participar cualquier persona, sin necesidad de ser estudiante. Algunos ejemplos son Google Hash Code, Google Kick Start y Mexico ICPC Masters.

También hay plataformas que organizan concursos de práctica de forma regular, como Codeforces y AtCoder. Estos espacios son ideales para entrenar de manera constante, comparar enfoques con otras personas y acostumbrarse a resolver problemas bajo presión de tiempo.

En resumen, existen muchos jueces en línea y miles de problemas disponibles para seguir practicando. Siempre habrá un nuevo reto esperándote.

2.2.1. Ejercicios

- Tabla para Pastel (omegaUp): Concurso FCIencias 2016.
- Watermelon (Codeforces 4A): Determinar si una sandía de peso w puede dividirse en dos partes pares positivas.
- Minimize (Codeforces 2009A): Para cada caso, hallar el mínimo de $(c - a) + (b - c)$ con $a \leq c \leq b$.
- I Wanna Be the Guy (Codeforces 469A): Decidir si dos jugadores, combinando los niveles que cada uno puede pasar, cubren todos los niveles del juego.

Capítulo 3

Jueces en Línea

Los jueces en línea son plataformas que reciben el código fuente de una solución, lo compilan, lo ejecutan con casos de prueba predefinidos y determinan automáticamente si el programa cumple con lo pedido en el problema. En programación competitiva, prácticamente toda la evaluación pasa por este tipo de sistemas.

3.1. ¿Qué hace un juez en línea?

Cuando una persona concursante envía una solución, el juez no revisa si “la idea parece correcta”, sino si el programa produce exactamente el comportamiento esperado. Para ello, utiliza un conjunto de casos de prueba preparados por quienes diseñaron el problema, junto con una salida correcta previamente validada.

Antes de que un problema llegue al concurso, normalmente pasa por una etapa de preparación. En muchas competencias existe una convocatoria o proceso interno para proponer problemas; cada propuesta debe venir acompañada de una solución conocida, correcta y justificable. En otras palabras, no se plantean problemas de investigación abierta ni preguntas cuya solución todavía sea incierta: el comité necesita saber de antemano que el problema se puede resolver y bajo qué complejidad.

Además del enunciado, quienes preparan el problema suelen entregar varios elementos técnicos: una o más soluciones de referencia, un conjunto de archivos de entrada con sus salidas esperadas, límites de tiempo y memoria, y en algunos casos un comparador especial. Estas piezas permiten construir la evaluación automática con suficiente confianza. Si el problema admite varias respuestas correctas, tolerancias numéricas o criterios más sutiles que una comparación literal, entonces el comparador también forma parte esencial del material del problema.

En muchos casos, los archivos de prueba no se escriben uno por uno de manera artesanal, sino que se generan a partir de generadores y después se validan con soluciones de referencia. El objetivo es cubrir instancias pequeñas, casos borde y situaciones diseñadas para detectar errores frecuentes o algoritmos demasiado lentos. Así, el juez no solo revisa si una solución funciona en ejemplos sencillos, sino si se comporta correctamente en escenarios representativos y exigentes.

3.1.1. De la propuesta al veredicto

Una vez preparado el problema, el juez en línea sigue un proceso bien definido para evaluar cada envío. De manera general, recibe el programa de la persona concursante, lo compila si el lenguaje lo requiere,

lo ejecuta con los casos de prueba ocultos y compara la salida producida con la respuesta esperada o con el criterio establecido por el comparador. Si además se respetan las restricciones de tiempo y memoria, la solución se acepta. De forma detallada, este proceso puede descomponerse en varias etapas:

1. El concursante escribe su programa y lo envía a la plataforma.
2. El juez recibe el archivo y verifica que el envío sea válido para el lenguaje seleccionado.
3. Si el lenguaje lo requiere, el sistema compila el programa.
4. Si la compilación tiene éxito, el evaluador ejecuta la solución sobre los casos de prueba ocultos.
5. Para cada caso, el programa recibe una entrada determinada y produce una salida.
6. Esa salida se compara con la respuesta esperada, ya sea de forma estricta o mediante un comparador especial.
7. Al mismo tiempo, el juez controla el tiempo de ejecución, el uso de memoria y, en algunas plataformas, el volumen de salida.
8. Si todos los casos pasan correctamente y se respetan las restricciones, la solución recibe el veredicto **Accepted**.

Como resultado de este proceso, el juez devuelve un veredicto. Los más frecuentes son los siguientes:

- **Accepted (AC):** la solución produjo la respuesta correcta dentro de los límites establecidos.
- **Wrong Answer (WA):** la salida no coincide con la esperada o no satisface el criterio del checker.
- **Compilation Error (CE):** el programa no compiló correctamente.
- **Run Time Error (RTE):** ocurrió un error durante la ejecución, como división entre cero o acceso inválido a memoria.
- **Time Limit Exceeded (TLE):** la solución tardó más de lo permitido.
- **Memory Limit Exceeded (MLE):** la solución usó más memoria de la disponible.
- **Output Limit Exceeded (OLE):** el programa imprimió más salida de la permitida.
- **Presentation Error (PE):** en algunas plataformas, la respuesta es esencialmente correcta pero el formato no coincide; en otras, este caso se reporta simplemente como **WA**.

3.2. Arquitectura general

Aunque la implementación concreta puede variar entre plataformas, un juez en línea suele estar compuesto por varias partes que colaboran entre sí. En una versión un poco más realista, el envío pasa primero por un frontend, luego se registra como una solicitud de evaluación y entra en una cola. A partir de esa cola, uno o varios procesos trabajadores, o *workers*, toman envíos pendientes, los compilan y los ejecutan dentro de un entorno aislado. Después, la salida producida se envía a un comparador, que decide si la respuesta debe aceptarse o no.

Una forma simplificada de visualizarlo es la siguiente:

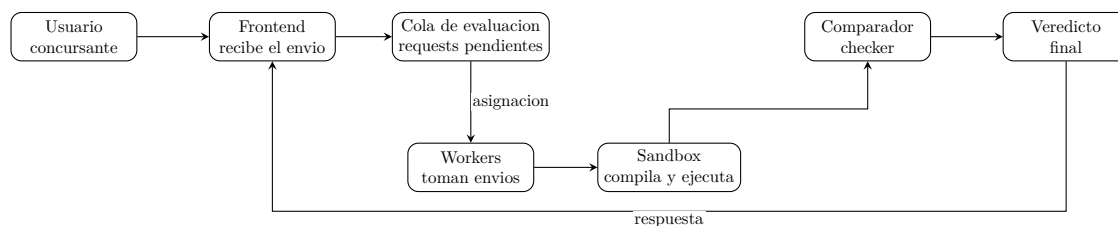


Figura 3.1: Esquema simplificado del flujo de evaluación en un juez en línea.

En este esquema, el **usuario o concursante** prepara y envía su solución; el **frontend** recibe ese envío y valida aspectos básicos antes de registrarlo; la **cola de evaluación** conserva los trabajos pendientes; los **workers** toman envíos de esa cola y coordinan su procesamiento; el **sandbox** compila y ejecuta el programa dentro de un entorno aislado y con recursos limitados; el **comparador o checker** contrasta la salida producida con la salida esperada, y en algunos casos puede ignorar diferencias de mayúsculas, normalizar texto o aplicar tolerancias numéricas; finalmente, el sistema devuelve un **veredicto** al usuario.

3.3. ¿Cómo compara las respuestas?

El punto más delicado del proceso suele ser la comparación entre la salida esperada y la salida producida por el programa. En muchos problemas, esta comparación es estricta: el juez revisa carácter por carácter, por lo que una diferencia mínima en espacios, saltos de línea o formato puede hacer que una respuesta correcta en idea sea rechazada.

Sin embargo, no todos los problemas usan exactamente el mismo criterio. Existen al menos tres escenarios comunes:

- **Comparación estricta:** la salida debe coincidir exactamente con el formato esperado.
- **Comparación con tolerancia:** se permiten pequeñas diferencias, por ejemplo en problemas con números reales donde se acepta cierto error absoluto o relativo.
- **Comparación por equivalencia:** el juez acepta múltiples salidas válidas, por ejemplo cuando los elementos pueden aparecer en cualquier orden o cuando se usa un checker especial que verifica propiedades de la respuesta y no solo texto literal.

Por esta razón, leer con cuidado la sección de salida del problema es tan importante como diseñar el algoritmo. Muchas soluciones fallan no por un error matemático o lógico, sino por no respetar exactamente lo que el problema solicita imprimir.

3.4. Clarificaciones durante un concurso

En competencias oficiales, el juez en línea no trabaja de manera aislada: suele existir un comité de personas organizadoras que supervisa el comportamiento de la plataforma y atiende dudas sobre los enunciados. Si una persona concursante detecta una posible ambigüedad o un error, puede enviar una aclaración para que el comité revise el caso.

Si se confirma un problema en el enunciado, en los datos o en el criterio de evaluación, el comité puede corregirlo y volver a juzgar las soluciones enviadas. En situaciones más delicadas, incluso puede

retirarse un problema del concurso. Esto muestra que, aunque el juez automatiza la evaluación, el diseño y mantenimiento del concurso sigue dependiendo de una supervisión humana cuidadosa.

3.5. Algunos jueces y plataformas en línea

Existen muchas plataformas en las que es posible practicar, competir o simplemente familiarizarse con el estilo de evaluación automática. Cada una tiene su propio enfoque, comunidad y tipo de problemas. A continuación se mencionan algunas de las más conocidas:

- **NetCode:** es una plataforma orientada a la práctica de programación y evaluación automática. Puede ser útil para entrenar con problemas y acostumbrarse al flujo básico de envío y veredicto. Se puede consultar en <https://netcode.dev>.
- **LeetCode:** es una plataforma muy popular para practicar problemas algorítmicos, especialmente entre quienes se preparan para entrevistas técnicas. Cuenta con una modalidad de paga que da acceso, entre otras cosas, a editoriales y materiales adicionales. Su dirección es <https://leetcode.com>.
- **Kattis:** es ampliamente utilizado tanto para práctica individual como en competencias universitarias. Su colección de problemas es extensa y su estilo de enunciados se parece mucho al de varios concursos formales. Puede consultarse en <https://open.kattis.com>.
- **omegaUp:** es una plataforma muy importante en el ámbito hispanohablante y particularmente en México. Permite practicar, organizar concursos y seguir rankings, por lo que ha sido una puerta de entrada para muchas personas que comienzan en programación competitiva. Su sitio es <https://omegaup.com>.
- **Codeforces:** es una de las plataformas más activas de la comunidad internacional de programación competitiva. Ofrece concursos frecuentes, problemas clasificados por dificultad, editoriales y una comunidad muy dinámica. Puede consultarse en <https://codeforces.com>.

Explorar varias plataformas también ayuda a desarrollar flexibilidad, ya que no todas presentan los problemas del mismo modo ni usan exactamente el mismo estilo de juez. Con el tiempo, familiarizarse con estas diferencias hace más natural adaptarse a concursos y entornos diversos.

3.6. Una observación importante

Para quienes comienzan, uno de los mayores puntos de frustración es descubrir que un programa puede fallar aun cuando la idea general sea correcta. Un simple espacio extra, una línea faltante o un formato diferente al solicitado puede ser suficiente para recibir un veredicto negativo.

Con la práctica, esta parte deja de parecer arbitraria y empieza a formar parte natural del proceso de resolución. Aprender a leer con precisión, imprimir exactamente lo pedido y revisar cuidadosamente la salida es también parte del entrenamiento en programación competitiva.

Capítulo 4

Entrada/Salida

4.1. Primer contacto con la entrada y la salida

Lo primero que una persona suele aprender al hacer programación competitiva es manejar correctamente la entrada y la salida de un problema. Recordemos que los jueces en línea ejecutan nuestro programa y le proporcionan texto que corresponde a la entrada; ese texto puede venir en varios formatos, y por ello es importante estar atento a la manera en que se presentan los datos.

La mayoría de los problemas trabajan con **entrada estándar** y **salida estándar**. Esto significa que el programa recibe los datos por medio de un flujo de entrada, por ejemplo `cin` en C++, y escribe su respuesta mediante un flujo de salida, asociado con `cout`. En un concurso no se acostumbra abrir archivos manualmente; el juez se encarga de proporcionar la entrada al programa y de capturar su salida para revisarla.

Este estilo aparece con mucha frecuencia en concursos formales, donde justamente parte del trabajo consiste en leer correctamente la entrada y producir la salida con el formato exacto que pide el enunciado. En ese sentido, no basta con tener una buena idea: también hace falta traducirla a un programa que interprete bien los datos desde el primer momento.

4.2. Entrada estándar, salida estándar y funciones dadas

En muchos concursos y jueces clásicos, leer la entrada es parte explícita del problema. El programa debe reconocer cuántos datos vienen, en qué orden aparecen y cómo termina cada caso. Esto forma parte del entrenamiento, porque obliga a prestar atención no solo al algoritmo sino también al formato.

Sin embargo, existen plataformas en las que este paso se simplifica. En algunos jueces en línea se proporciona una función ya definida y lo único que se espera es completar su implementación. En esos casos, el propio juez se encarga de interpretar la entrada, llamar a la función con los parámetros adecuados y revisar el resultado devuelto. Esto hace que la curva de aprendizaje inicial sea un poco más amable, pues ya no es necesario lidiar desde el principio con toda la parte de lectura y escritura.

Ninguno de estos estilos es “mejor” que el otro: simplemente responden a objetivos distintos. Los concursos formales suelen exigir dominio de entrada y salida completas, mientras que otras plataformas buscan facilitar el enfoque directo sobre la lógica del problema.

4.3. Patrones comunes de entrada

Una de las habilidades más útiles al comenzar es reconocer rápidamente qué tipo de entrada se está usando. A continuación se presentan algunos patrones frecuentes, junto con ejemplos pequeños en C++.

4.3.1. Unicaso

En este patrón, hay un único caso de prueba cada vez que el juez ejecuta el programa. Se leen los datos una sola vez, se procesa la información y se imprime la respuesta.

Por ejemplo, supongamos que la entrada se compone de un número, seguido de un espacio, seguido de otro número y finalmente un salto de línea.

Entrada ejemplo

50 100

Salida ejemplo

150

En C++, la instrucción `cin >> a >> b;` ignora automáticamente los espacios y saltos de línea entre enteros, por lo que no hace falta procesarlos de forma manual.

```
1 int main() {  
2     int a, b;  
3     cin >> a >> b;  
4     cout << a + b << "\n";  
5 }
```

La misma idea puede verse con `scanf`, que también ignora espacios en blanco entre números cuando se usa un formato como `"%d%d"`.

```
1 int main() {  
2     int a, b;  
3     scanf("%d %d", &a, &b);  
4     printf("%d\n", a + b);  
5 }
```

4.3.2. Multicaso específico

Aquí el número de casos de prueba aparece en la primera línea de la entrada. Después de leer ese número, se repite el mismo proceso para cada caso.

Por ejemplo, si se tienen tres casos y en cada uno se pide sumar dos números, la entrada podría ser

Entrada ejemplo

3
1 2
10 20
50 100

Salida ejemplo

3
30
150

En este caso, primero se lee cuántos casos hay y luego se repite el mismo bloque de lectura y escritura ese número de veces.

```
1 int main() {  
2     int t;  
3     cin >> t;  
4  
5     while (t--) {
```



```

6         int a, b;
7         cin >> a >> b;
8         cout << a + b << "\n";
9     }
10 }

```

4.3.3. Multicaso controlado por valores

En algunos problemas no se especifica cuántos casos hay, sino que se da una condición de parada. Un ejemplo clásico es seguir leyendo pares a , b hasta encontrar $a = 0$ y $b = 0$.

Por ejemplo, la entrada podría ser

Entrada ejemplo

```

4 5
10 20
7 8
0 0

```

Salida ejemplo

```

9
30
15

```

El último par 0 0 no se procesa: solo sirve para indicar que ya no hay más casos.

```

1 int main() {
2     while (true) {
3         int a, b;
4         cin >> a >> b;
5
6         if (a == 0 && b == 0) break;
7
8         cout << a + b << "\n";
9     }
10 }

```

4.3.4. Multicaso controlado por EOF

Otro patrón clásico consiste en leer hasta encontrar el final del archivo, es decir, hasta EOF. Esto es muy común en jueces tradicionales y en problemas donde no se indica de antemano cuántos casos existen.

Por ejemplo, si la entrada es

Entrada ejemplo

```

8 2
100 50
7 13

```

Salida ejemplo

```

10
150
20

```

El programa sigue leyendo mientras todavía existan datos válidos en la entrada.

```

1 int main() {
2     int a, b;
3     while (cin >> a >> b) {
4         cout << a + b << "\n";
5     }
6 }

```

4.3.5. Multicaso con salida numerada

En este esquema se especifica el número de casos, pero además la salida debe incluir un formato como “Caso #i: respuesta”. Es importante respetar exactamente la forma pedida por el enunciado.

Por ejemplo, la entrada podría ser

Entrada ejemplo

```
3
2 3
10 1
7 8
```

Salida ejemplo

```
Caso #1: 5
Caso #2: 11
Caso #3: 15
```

Aquí no basta con imprimir el resultado: también hay que numerar cada caso según el formato exigido.

```
1 int main() {
2     int t;
3     cin >> t;
4
5     for (int caso = 1; caso <= t; caso++) {
6         int a, b;
7         cin >> a >> b;
8         cout << "Caso #" << caso << ": " << (a + b) << "\n";
9     }
10 }
```

4.3.6. Multicaso con datos en una línea

En algunos problemas, cada caso de prueba corresponde a una línea completa que contiene una cantidad arbitraria de enteros. En ese caso, además de leer, hace falta procesar la línea completa como una unidad.

Por ejemplo, si la primera línea indica cuántos casos habrá y luego cada caso es una línea con una cantidad arbitraria de enteros, la entrada podría ser

Entrada ejemplo

```
3
1 2 3
10 20
7 8 9 10
```

Salida ejemplo

```
6
30
34
```

En este patrón ya no basta con hacer `cin >> x` de forma fija, porque cada línea puede contener una cantidad distinta de números.

```
1 int main() {
2     int t;
3     cin >> t;
4     cin.ignore();
5
6     while (t--) {
7         string linea;
8         getline(cin, linea);
9
10        stringstream ss(linea);
11        int x, suma = 0;
12        while (ss >> x) {
13            suma += x;
14        }
15    }
16 }
```

```

14     }
15
16     cout << suma << "\n";
17 }
18 }

```

4.3.7. Opción de función

Finalmente, hay jueces en los que no se escribe un programa completo, sino únicamente una función ya indicada. El propio sistema se encarga de leer la entrada, construir los parámetros y llamar a esa función.

En una plataforma de este tipo, podría pensarse en un caso donde internamente el juez llama a una función con los valores 50 y 100, y espera como resultado 150. Desde la perspectiva del usuario, esa interacción muchas veces no aparece como entrada y salida estándar visibles, porque el propio sistema se encarga de esa parte.

Entrada ejemplo

50 100

(simulada por el juez)

Salida ejemplo

150

Para ilustrarlo de manera sencilla, se puede simular el comportamiento con el siguiente programa:

```

1 class Solution {
2 public:
3     int suma(int a, int b) {
4         return a + b;
5     }
6 };

```

Este formato aparece con frecuencia en plataformas orientadas a entrevistas o práctica guiada. Aunque simplifica la interacción con la entrada y la salida, no sustituye la necesidad de entender los patrones clásicos, ya que estos siguen siendo fundamentales en programación competitiva.

4.4. Una observación práctica

Aprender entrada y salida no es un tema menor ni puramente mecánico. De hecho, es una de las primeras habilidades que distinguen a quien empieza a sentirse cómodo frente a un juez en línea. Leer con precisión, identificar el patrón de los datos y respetar exactamente el formato pedido ahorra muchos errores y permite concentrarse en lo verdaderamente importante: resolver el problema.

Capítulo 5

Álgebra

El álgebra es, en esencia, el estudio de patrones numéricos, en la programación competitiva es muy común que ideas de soluciones se basen en propiedades algebraicas que veremos en esta sección.

Supongamos que queremos sumar los primeros n números naturales, claramente si N es un número pequeño como 4, rápidamente sumaríamos los números del 1 al 4 y diríamos que la respuesta es 10, sin embargo el sumar número por número no resulta práctico cuando n es un número muy grande como podría ser un millón, ¿habrá alguna forma de saber rápidamente la respuesta para cualquier n ? La respuesta es sí, llamémosle S a la suma de todos los números de 1 a n , supongamos que ponemos en una fila todos los números a sumar, es decir

$$1 + 2 + 3 + 4 + 5 + \cdots + n$$

ahora, junto a esta pongamos otra fila de los números a sumar al revés, notemos que sumarlos en un orden diferente no afecta el resultado

$$\begin{array}{ccccccc} S & = & 1 & + & 2 & + & \cdots + n \\ S & = & n & + & (n-1) & + & \cdots + 1 \end{array}$$

notemos que si sumamos cada columna tenemos que cada una suma $n+1$ y nótese que tenemos n columnas entonces si sumamos las dos listas que es $2S$ es igual que sumar las n columnas, es decir $2S = n(n+1)$ de donde tenemos que

$$\sum_{i=1}^n i = 1 + 2 + \cdots + n = \frac{n(n+1)}{2}$$

Esta suma es un ejemplo de **progresión aritmética**, más generalmente una progresión aritmética es una sucesión de números $\{a_1, a_2, a_3, \dots\}$ tal que cualquier término se puede obtener del anterior al sumarle una cantidad fija d . Usando la idea de arriba podemos deducir que para cualquier progresión aritmética

$$a_1 + \cdots + a_n = \frac{n(a_1 + a_n)}{2}$$

Ejercicio: Si en lugar de $1 + 2 + 3 + \cdots + n$ tuviéramos $1^2 + 2^2 + 3^2 + \cdots + n^2$ cuál sería el resultado?

Ahora supongamos que queremos saber cuánto es la siguiente suma

$$\sum_{i=0}^n r^i = 1 + r + r^2 + \cdots + r^n$$

llamemosle a la suma $S = 1 + r + r^2 + \dots + r^n$, nótese que si multiplicamos ambos lados por r tenemos que

$$rS = r + r^2 + \dots + r^{n+1}$$

ahora de ambos lados podemos restar S , es decir

$$rS - S = r + r^2 + \dots + r^n - S = r + r^2 + \dots + r^{n+1} - (1 + r + r^2 + \dots + r^n)$$

notemos que del lado derecho la mayoría de los elementos se van a cancelar, de forma que finalmente nos queda, nótese que esto tiene sentido si $r \neq 1$ de lo contrario estaríamos dividiendo entre 0.

$$S = \frac{r^{n+1} - 1}{r - 1}$$

Para $|r| < 1$ y $n \rightarrow \infty$, la serie converge:

$$\sum_{i=0}^{\infty} r^i = \frac{1}{1 - r}$$

Las sumas no solo sirven para obtener la respuesta del problema, también nos ayudan a evaluar complejidades, las identidades más comunes que se usan en programación competitiva son:

$$\begin{aligned} \sum_{i=1}^n i &= \frac{n(n+1)}{2} \\ \sum_{i=1}^n i^2 &= \frac{n(n+1)(2n+1)}{6} \\ \sum_{i=1}^n i^3 &= \left(\frac{n(n+1)}{2} \right)^2 \\ \sum_{i=0}^{n-1} r^i &= \frac{r^n - 1}{r - 1} \quad (r \neq 1) \\ \sum_{i=1}^n \frac{1}{i} &= \ln n + \gamma \quad (\text{armónica}) \end{aligned}$$

Cuadro 5.1: Principales identidades algebraicas

$\gamma \approx 0,5772$ es la constante de Euler-Mascheroni.

$\sum_{i=1}^n \lfloor n/i \rfloor \approx n \ln n$ porque $\sum_{i=1}^n n/i = n(\ln(n) + \gamma)$. Esto explica por qué iterar sobre todos los divisores de todos los números hasta n cuesta $O(n \log n)$.

Una técnica muy poderosa es cambiar el orden de una doble sumatoria. La igualdad:

$$\sum_{i=1}^n \sum_{j=i}^n f(i, j) = \sum_{j=1}^n \sum_{i=1}^j f(i, j)$$

parece trivial pero frecuentemente transforma una suma difícil en una fácil.

Ejemplo Calcular $\sum_{i=1}^n \sum_{j=i}^n \frac{1}{j}$. Cambiando el orden:

$$\sum_{j=1}^n \frac{1}{j} \sum_{i=1}^j 1 = \sum_{j=1}^n \frac{j}{j} = \sum_{j=1}^n 1 = n$$

Lo que parecía difícil se vuelve más sencillo al cambiar el orden.

5.1. Recurrencias

La sucesión $0, 1, 1, 2, 3, 5, 8, \dots$ es muy famosa y es conocida como la sucesión de fibonacci, la forma en la que se nos enseña a definirla es haciendo el primer término $F_0 = 0$, el segundo $F_1 = 1$ y desde ahí calcular los siguientes con la recurrencia

$$F_n = F_{n-1} + F_{n-2}$$

Más generalmente una recurrencia lineal se ve de la siguiente forma

$$R_x = \sum_{i=1}^n C_i R_{x_i}$$

esto significa que R_x depende de los anteriores n términos, es decir es de orden n y cada término puede tener una constante C_i multiplicandole. Por ejemplo, la recurrencia de Fibonacci es de orden 2 y la constante en cada uno de los términos es 1.

Ahora nos surge una pregunta natural, dada una recurrencia lineal de orden n , ¿cómo encontramos el término k ésimo?

Existen varias formas de solucionar este problema, y cada uno nos arroja una complejidad diferente, no es que exista una mejor o peor forma de calcular la respuesta, porque todo dependerá de qué tan grandes con k y n .

5.1.1. Solucion DP

Se le llama programación dinámica (DP por sus siglas en inglés) cuando memorizamos datos que nos ayuden a evitar hacer dobles calculos, por ejemplo en fibonacci podemos almacenar los términos que vayamos obteniendo y así cuando lleguemos al término n ya tendremos calculado $n - 1$ y $n - 2$ y no tendremos que volver a calcularlo. En el caso de una recurrencia lineal general la forma de implementarlo con DP es bastante simple, es como hacer fuerza bruta pero memorizando lo que ya calculamos, es decir

```
1 //necesitamos los primeros n terminos porque cada termino depende de los n anteriores
2 for(int i = n; i <= k; i ++){
3     dp[i] = 0;
4     //para cada uno de los terminos anteriores j sumamos el termino multiplicado por
    su constante
5     for(int j = 1; j <= n; j++){
6         dp[i] += c[j] * dp[i-j];
7     }
8 }
```

este procedimiento tiene una complejidad $O(nk)$, es recomendable por simplicidad usarse en casos donde $nk < 1e7$ para evitar TLE.

5.1.2. Solución Exponenciación de Matrices

De la solución con programación dinámica notemos que para cada paso sólo nos importa tener la información de los últimos n términos y todos los demás, podemos olvidarlos. Si guardamos los n ultimos elementos en una matriz columna, podemos aplicarle una transformación lineal para obtener los siguientes elementos, es decir si originalmente teníamos los elementos $\{R_1, R_2, \dots, R_n\}$, después

de la transformación lineal tendremos los elementos $\{R_2, R_3, \dots, R_{n+1}\}$, la transformación lineal es relativamente sencilla y consiste en multiplicar por una matriz cuadrada como sigue

$$\begin{bmatrix} C_1 & C_2 & \cdots & C_{n-1} & C_n \\ 1 & 0 & \cdots & 0 & 0 \\ 0 & 1 & \cdots & 0 & 0 \\ \vdots & \vdots & \cdots & \vdots & \vdots \\ 0 & 0 & \cdots & 1 & 0 \end{bmatrix} \begin{bmatrix} R_n \\ R_{n-1} \\ R_{n-2} \\ \vdots \\ R_1 \end{bmatrix} = \begin{bmatrix} R_{n+1} \\ R_n \\ R_{n-1} \\ \vdots \\ R_2 \end{bmatrix}$$

podríamos entonces pensar que ya que cada multiplicación cambia los índices en uno, tendríamos que hacer $k - n$ multiplicaciones para obtener el término k , podríamos hacerlo, pero hacer esto nos implicaría una complejidad $O(n^3(k - n))$, en realidad podemos multiplicar la matriz cuadrada por si misma $k - n$ veces usando exponenciación binaria¹ de esta forma obtenemos el k ésimo término en tiempo $O(n^3 \log(k - n))$.

Existe otra forma de resolver de recurrencias lineales, pero necesitamos herramientas más elaboradas por lo que lo abordaremos en la sección de transformada rápida de Fourier.

5.2. Polinomios e interpolación de Lagrange

Un polinomio de grado n se representa como un vector de $n + 1$ coeficientes:

$$p(x) = a_0 + a_1x + a_2x^2 + \cdots + a_nx^n$$

Lo primero que nos interesa será poder evaluarlo en algún punto de forma eficiente, para esto usaremos el **método de Horner**, que simplemente consiste en reescribir el polinomio como:

$$p(x) = a_0 + x(a_1 + x(a_2 + \cdots + x(a_{n-1} + x \cdot (0 + a_n)) \cdots))$$

Es decir, se empezará a calcular desde el coeficiente de mayor grado y se va acumulando de adentro hacia afuera, la implementación la haremos con módulo ya que es la más usada, sin embargo si no se necesita basta con quitar "mod" del código.

```
1 ll horner(vector<ll> a, ll x, ll mod) {
2     ll res = 0;
3     for (int i = a.size()-1; i >= 0; i--)
4         res = (res * x + a[i]) % mod;
5     return res;
6 }
```

Lo que queremos hacer en esta sección es dados n puntos cualesquiera, encontrar un polinomio que pasa por todos ellos, a esto se le llama interpolación de Lagrange.

Es importante mencionar que dados $n + 1$ puntos distintos $(x_0, y_0), (x_1, y_1), \dots, (x_n, y_n)$, existe un **único** polinomio de grado a lo más n que pasa por todos ellos.

El razonamiento detrás de por qué esto es cierto es sencillo, supongamos que no se cumple, es decir, que hay dos polinomios diferentes p y q de grado $\leq n$ que pasan por todos los puntos dados. Entonces tendremos que $p - q$ es un polinomio de grado $\leq n$ que tiene $n + 1$ raíces (los x_i), pero sabemos que un polinomio no nulo de grado $\leq n$ tiene a lo más n raíces, de donde tenemos que tiene que ser

¹Ver sección de exponenciación binaria

nulo, es decir $p - q = 0$, por lo que $p = q$.

Esto nos dice que $n + 1$ puntos determinan completamente un polinomio de grado n . La pregunta es: ¿cómo construirlo?

La idea de Lagrange es construir el polinomio interpolante como una suma de piezas, donde cada pieza se encarga de un solo punto.

Para cada i , definimos el **polinomio base de Lagrange** $\ell_i(x)$ como el único polinomio de grado n que vale 1 en x_i y 0 en todos los demás x_j :

$$\ell_i(x) = \prod_{j \neq i} \frac{x - x_j}{x_i - x_j}$$

Verifiquemos que cumple lo que queremos. Si evaluamos en $x = x_i$: cada factor del numerador es $x_i - x_j$ y cada factor del denominador es $x_i - x_j$, así que cada fracción vale 1 y el producto es 1. Si evaluamos en $x = x_k$ con $k \neq i$, el numerador tiene el factor $x_k - x_k = 0$, así que el producto entero es 0.

Con estas piezas, el polinomio interpolante es simplemente:

$$p(x) = \sum_{i=0}^n y_i \cdot \ell_i(x)$$

El polinomio pasa por todos los puntos pues en $x = x_i$, todos los $\ell_j(x_i) = 0$ excepto $\ell_i(x_i) = 1$, así que $p(x_i) = y_i$. Pasa exactamente por todos los puntos dados. Encontrar el polinomio nos lleva entonces tiempo $O(n^2)$.

Veamos un ejemplo concreto con los puntos $(0, 1)$, $(1, 2)$ y $(2, 5)$. Aquí $n = 2$, así que buscamos un polinomio de grado a lo más 2. Los polinomios base son

$$\ell_0(x) = \frac{(x-1)(x-2)}{(0-1)(0-2)} = \frac{(x-1)(x-2)}{2}, \quad \ell_1(x) = \frac{x(x-2)}{(1-0)(1-2)} = -x(x-2), \quad \ell_2(x) = \frac{x(x-1)}{(2-0)(2-1)} = \frac{x(x-1)}{2}.$$

El interpolante queda entonces

$$p(x) = 1 \cdot \ell_0(x) + 2 \cdot \ell_1(x) + 5 \cdot \ell_2(x) = \frac{(x-1)(x-2)}{2} - 2x(x-2) + \frac{5x(x-1)}{2} = x^2 + 1.$$

Comprobemos: $p(0) = 1$, $p(1) = 2$ y $p(2) = 5$, como queríamos.

En ocasiones podemos optimizar considerablemente este tiempo para casos en los que se nos pide calcular $f(x)$ dados $f(1), f(2), \dots, f(n)$, ya que en este caso $x_i = i$ de forma que podemos sustituir para obtener

$$p(x) = \sum_{i=0}^n f(i) \cdot \prod_{j \neq i} \frac{x-j}{i-j}$$

podríamos preguntarnos por qué esto es una optimización, nótemos que

$$\prod_{j \neq i} (x-j) = (x-1)(x-2) \cdots (x-(i-1))(x-(i+1)) \cdots (x-n)$$

entonces al menos para el numerador podemos precalcular los prefijos, es decir guardamos en un vector $x-1, (x-1)(x-2), (x-1)(x-2)(x-3), \dots, (x-1)(x-2) \cdots (x-n)$ y los sufijos, que es lo mismo pero del último elemento al primero, es decir $(x-n), (x-n)(x-(n-1))$ y así hasta $(x-n)(x-(n-1)) \cdots (x-1)$,

de forma que cuando queramos hacer la multiplicación de arriba podamos hacerlo en tiempo $O(1)$ multiplicando el prefijo $(x-1)\cdots(x-(i-1))$ con el prefijo $(x-n)\cdots(x-(i+1))$.

Nótese que en el caso de no tener módulo podemos simplemente precalcular $(x-1)(x-2)\cdots(x-n)$ y dividir entre $(x-i)$ cuando ocupemos calcularlo para un i .

Para el denominador también podemos hacer algo similar, pues

$$\prod_{j \neq i} i-j = (i-1)(i-2)\cdots(i-(i-1))(i-(i+1))\cdots(i-n)$$

notemos que dividiendo en dos como arriba ambas partes de la multiplicación se pueden expresar como factoriales, es decir

$$= (i-1)!(-1)^{n-i}(n-i)!$$

para esto entonces nos interesara poder precalcular los inversos de los factoriales² y así para cada i también lo podemos calcular en $O(1)$.

Como tenemos que calcular tanto numerador como denominador para cada i , calcular $f(x)$ nos cuesta $O(n)$, lo que es considerablemente mejor que el tiempo cuadrático de arriba.

Ejercicio. Para practicar lo visto, resolvamos el siguiente problema: dados n y k tales que $0 \leq k \leq 10^6$ y $1 \leq n \leq 10^9$, calcular

$$f(n) = \sum_{i=1}^n i^k$$

e imprimir la respuesta módulo $10^9 + 7$. Por ejemplo, si $n = 4$ y $k = 2$, se pide

$$f(4) = 1^2 + 2^2 + 3^2 + 4^2 = 1 + 4 + 9 + 16 = 30;$$

con módulo $10^9 + 7$ la salida sería 30.

Nuestro primer instinto probablemente sea calcular el resultado sumando uno a uno los términos de esta suma, sin embargo aún si pudiesemos calcular las potencias i^k de forma constante, tendríamos que sumar n términos, es decir en el peor caso 10^9 operaciones, lo que nos daría como veredicto TLE.

En realidad, podemos ver este problema desde otro enfoque, denotemos como $p_k(n) = \sum_{i=1}^n i^k$, lo primero que queremos saber es qué grado tiene el polinomio, para saber cuántos puntos necesitamos, esto lo haremos por inducción, nótese que si $k = 1$, por lo visto antes, $p_1(n) = \frac{n(n+1)}{2}$, es decir es un polinomio de grado $k+1 = 2$, supongamos entonces que $p_{k-1}(n)$ tiene grado k , ahora consideremos $p_k(n)$ y saquemos su derivada, es decir $p'_k(n) = kp_{k-1}(n)$ de donde como la derivada baja un grado, tenemos que $p_k(n)$ tiene grado $k+1$.

De lo anterior concluimos que necesitamos $k+2$ puntos por los que pase el polinomio para determinarlo, estos podemos encontrarlos fácilmente pues $f(0) = 0$ y de ahí $f(x) = f(x-1) + x^k$. Traes esto ocuparemos la interpolación vista arriba para calcular $f(n)$, de forma que la implementación nos quedaría

```
1 const ll mod = 1e9 + 7;
2 ll lagrangeInt(int n, int k) {
3     if (k == 0) return n;
```

²Ver la sección de inversos con modulo

```

4      ll res = 0;
5      if (n <= k + 1) {
6          for (int i = 1; i <= n; i++) {
7              res = (res + power(i, k)) % mod;
8          }
9          return res;
10     }
11     vector<ll> pre(k + 2), suf(k + 2), inv(k + 2);
12     inver[0] = 1;
13     pre[0] = n;
14     suf[k + 1] = n - (k + 1);
15     //precalcular prefijos
16     for (int i = 1; i <= k; i++){
17         pre[i] = pre[i - 1] * (n - i) % mod;
18     }
19     //precalcular sufijos
20     for (int i = k; i >= 1; i--){
21         suf[i] = suf[i + 1] * (n - i) % mod;
22     }
23     //precalcular inversos
24     for (int i = 1; i <= k + 1; i++){
25         inv[i] = inv[i - 1] * inv(i) % mod;
26     }
27
28     int yi = 0;
29     int num, denom;
30     for (int i = 0; i <= k + 1; i++) {
31         yi = (yi + power(i, k)) % mod;
32
33         //calcular numerador
34         if (i == 0){
35             num = suf[1];
36         }
37         else if (i == k + 1){
38             num = pre[k];
39         }
40         else{
41             num = pre[i - 1] * suf[i + 1] % mod;
42         }
43
44         //calcular denominador
45         denom = inv[i] * inv[k + 1 - i] % mod;
46
47
48         if ((i + k) % 2 == 1){
49             res += (yi * num % mod) * denom % mod;
50         }
51         else{
52             ans -= (yi * num % mod) * denom % mod;
53         }
54
55         //evitar que el resultado sea negativo
56         res = (res % mod + mod) % mod;
57     }
58     return res;
59 }
60
61 int main() {
62     int n, k; cin >> n >> k;
63     cout << lagrangeInt(n, k) << endl;
64 }

```

Ejercicio Optimizar la interpolación de lagrange como lo hicimos cuando los puntos son $1, 2, 3, \dots, n$, para cualquier progresión aritmética.

5.3. Transformada de Fourier rápida (FFT) y NTT

En la sección pasada ya trabajamos con polinomios, una duda que nos puede surgir al manipularlos es ¿cómo multiplicarlos eficientemente?, supongamos que tenemos un polinomio $A(x) = \sum_{i=0}^{n-1} a_i x^i$ y otro $B(x) = \sum_{i=0}^{n-1} b_i x^i$, el producto de ambos es $C(x) = A(x) \cdot B(x)$ y tiene coeficientes:

$$c_k = \sum_{i+j=k} a_i b_j$$

Esta operación se llama **convolución** y calcularla directamente cuesta $O(n^2)$, cuando $n \approx 10^5$, esto es 10^{10} operaciones, tiempo que nos daría TLE al intentar solucionar un problema de esta forma en la ICPC. El objetivo de *FFT* será poder hacer esta multiplicación en $O(n \log n)$.

De la sección de arriba sabemos que si tenemos un polinomio de grado $n-1$ este lo podemos determinar evaluando n puntos diferentes, basado en esto nos gustaría tener para ambos polinomios A y B sus valores en n puntos estratégicos para entonces multiplicar estos valores y obtener valores del polinomio C .

La elección de los n puntos es lo que hace eficientes el algoritmo. FFT elige las **raíces n -ésimas de la unidad** $\omega^k = e^{2\pi i k/n}$, que tienen una estructura recursiva que permite hacer la evaluación e interpolación en $O(n \log n)$ en lugar de $O(n^2)$.

Para la demostración completa del algoritmo y todos los detalles de implementación referimos al blog de Codeforces de [leanderKortmann](#), que tras leer bastantes recursos, parece ser la referencia más clara y amigable disponible para competencia.³

FFT aparece disfrazado en muchos problemas de competencia, especialmente cuando hay que contar pares con cierta suma. Reconocer que el problema se reduce a multiplicar polinomios es lo más importante en este tipo de problemas ya que usualmente basta con usar la misma plantilla de *FFT*.

FFT vs NTT: cuándo usar cada una

Hay dos variantes del algoritmo con distintos casos de uso:

	FFT	NTT
Aritmética	Números complejos (<code>long double</code>)	Números enteros mod p , p primo
Coefficientes grandes	Sí, con riesgo de error de redondeo	No, acotados por p
Respuesta exacta	Con <code>llroundl</code> si valores $\leq 10^{14}$	Siempre
Uso típico	Scores, correlaciones, valores grandes	Conteos módulo primo

Se recomienda usar *FFT* cuando los coeficientes pueden superar el módulo de *NTT* $\approx 10^9$; y *NTT* cuando la respuesta se pide módulo un primo p que se comporte bien; esto es, si $p-1$ tiene un factor de

³<https://codeforces.com/blog/entry/111371>

2 suficientemente grande, garantizando raíces primitivas de la unidad módulo p para tamaños potencia de 2. El más usado en competencia es $998244353 = 119 \cdot 2^{23} + 1$, que soporta transformadas de hasta $2^{23} \approx 8 \times 10^6$ elementos.

Las implementaciones de FFT y NTT que se usan en los problemas de este capítulo se dejan aquí para referencia:

FFT

```

1
2 #include <complex>
3 using ld = long double;
4 using cd = complex<ld>;
5 const ld PI = acosl(-1.0L);
6
7 void fft(vector<cd>& a, bool inv) {
8     int n = a.size();
9     for (int i = 1, j = 0; i < n; i++) {
10         int bit = n >> 1;
11         for (; j & bit; bit >>= 1) j ^= bit;
12         j ^= bit;
13         if (i < j) swap(a[i], a[j]);
14     }
15     for (int len = 2; len <= n; len <<= 1) {
16         ld ang = 2 * PI / len * (inv ? -1 : 1);
17         cd wlen(cosl(ang), sinl(ang));
18         for (int i = 0; i < n; i += len) {
19             cd w(1);
20             for (int j = 0; j < len / 2; j++) {
21                 cd u = a[i + j];
22                 cd v = a[i + j + len / 2] * w;
23                 a[i + j] = u + v;
24                 a[i + j + len / 2] = u - v;
25                 w *= wlen;
26             }
27         }
28     }
29     if (inv)
30         for (auto& x : a) x /= n;
31 }

```

NTT

```

1 const ll MOD = 998244353;
2 const ll PRIM = 3;
3
4 void ntt(vector<ll>& a, bool inv) {
5     int n = a.size();
6     for (int i = 1, j = 0; i < n; i++) {
7         int bit = n >> 1;
8         for (; j & bit; bit >>= 1) j ^= bit;
9         j ^= bit;
10        if (i < j) swap(a[i], a[j]);
11    }
12    for (int len = 2; len <= n; len <<= 1) {
13        ll w = inv
14            ? power(PRIM, MOD - 1 - (MOD - 1) / len, MOD)
15            : power(PRIM, (MOD - 1) / len, MOD);
16        for (int i = 0; i < n; i += len) {
17            ll wn = 1;

```

```

18     for (int j = 0; j < len / 2; j++) {
19         ll u = a[i + j];
20         ll v = a[i + j + len / 2] * wn % MOD;
21         a[i + j] = (u + v) % MOD;
22         a[i + j + len / 2] = (u - v + MOD) % MOD;
23         wn = wn * w % MOD;
24     }
25 }
26 }
27 if (inv) {
28     ll ni = power(n, MOD - 2, MOD);
29     for (auto& x : a) x = x * ni % MOD;
30 }
31 }

```

En la práctica, lo que se llama desde el problema es simplemente `multiply(a, b)`, que devuelve el vector de coeficientes del producto. Las funciones `ntt` y `fft` son internas.

```

1 vector<ll> multiply_ntt(vector<ll> a, vector<ll> b) {
2     int result_size = a.size() + b.size() - 1;
3     int n = 1;
4     while (n < result_size){
5         n <<= 1;
6     }
7     a.resize(n); b.resize(n);
8     ntt(a, false); ntt(b, false);
9     for (int i = 0; i < n; i++){
10         a[i] = a[i] * b[i] % MOD;
11     }
12     ntt(a, true);
13     a.resize(result_size);
14     return a;
15 }
16 vector<ll> multiply_fft(vector<ll> a, vector<ll> b) {
17     int result_size = a.size() + b.size() - 1;
18     int n = 1;
19     while (n < result_size) n <<= 1;
20     vector<cd> fa(n), fb(n);
21     for (int i = 0; i < (int)a.size(); i++){
22         fa[i] = a[i];
23     }
24     for (int i = 0; i < (int)b.size(); i++){
25         fb[i] = b[i];
26     }
27     fft(fa, false); fft(fb, false);
28     for (int i = 0; i < n; i++){
29         fa[i] *= fb[i];
30     }
31     fft(fa, true);
32     vector<ll> res(result_size);
33     for (int i = 0; i < result_size; i++){
34         res[i] = llroundl(fa[i].real());
35     }
36     return res;
37 }

```

5.4. Ejercicios

- Fibonacci Numbers (CSES 1722)
- Graph Paths I (CSES 1723)

5.5. Problemas

- Throwing Dice (CSES 1092)
- Graph Paths II (CSES 1724)
- Cowmpany Cowmpensation (Codeforces 995F)
- Nikita and Order Statistics (Codeforces 993E)
- Running Competition (Codeforces 1398G)
- Proyecto Hali Mary ★
- Rumbo al Mundial ★

Capítulo 6

Teoría de números

En programación competitiva, dominar o al menos tener buenas nociones de los conceptos principales de teoría de números no es un lujo, sino una necesidad. A lo largo de este texto y en la práctica dentro de la ICPC, el lector encontrará problemas donde aparecen de manera natural ideas como divisibilidad, números primos, residuos, máximo común divisor y aritmética modular. En términos generales, la teoría de números estudia las propiedades de los enteros, y por ello ofrece un lenguaje muy útil para describir, simplificar y resolver muchos problemas clásicos de concurso.

6.1. Bases

Definición (divisibilidad). Decimos que a **divide** a b (y escribimos $a \mid b$) si existe un entero k tal que $b = ka$.

Ejemplo. Se cumple $4 \mid 20$ porque $20 = 4 \cdot 5$ (aquí $k = 5$). En cambio, $6 \nmid 20$, pues no existe ningún entero k con $20 = 6k$.

Veamos propiedades de la divisibilidad que aparecen con frecuencia en competencia.

Propiedad 1. Si un mismo número divide a otros dos, entonces también divide a su suma y su diferencia:

$$a \mid b \text{ y } a \mid c \implies a \mid (b \pm c).$$

Ejemplo. Como $3 \mid 12$ y $3 \mid 21$, entonces $3 \mid (21 - 12) = 9$ y $3 \mid (21 + 12) = 33$.

Propiedad 2. Si un número divide a otro, entonces también divide cualquier múltiplo de ese segundo número:

$$a \mid b \implies a \mid bc.$$

Ejemplo. Como $4 \mid 12$, también $4 \mid (12 \cdot 5) = 60$.

Propiedad 3. Si un número divide a un segundo y ese segundo divide a un tercero, entonces el primero divide al tercero:

$$a \mid b \text{ y } b \mid c \implies a \mid c.$$

Ejemplo. Como $2 \mid 8$ y $8 \mid 40$, se sigue que $2 \mid 40$.

El **máximo común divisor** de dos números a y b , denotado $\gcd(a, b)$ (siglas en inglés), es el mayor entero que divide a ambos. El **mínimo común múltiplo** $\text{lcm}(a, b)$ es el menor entero positivo divisible por ambos.

Ejemplo (gcd y lcm). Si $a = 12$ y $b = 18$, entonces los divisores de 12 son

$$\{1, 2, 3, 4, 6, 12\}$$

y los divisores de 18 son

$$\{1, 2, 3, 6, 9, 18\}.$$

Los divisores comunes son $\{1, 2, 3, 6\}$, así que $\gcd(12, 18) = 6$.

De manera análoga, los primeros múltiplos positivos de 12 son

$$\{12, 24, 36, 48, \dots\}$$

y los de 18 son

$$\{18, 36, 54, \dots\}.$$

El primero que aparece en ambas listas es 36, por lo que $\text{lcm}(12, 18) = 36$.

Propiedad 4. Para enteros positivos a y b ,

$$\text{lcm}(a, b) = \frac{a \cdot b}{\gcd(a, b)}.$$

Ejemplo. Con $a = 12$ y $b = 18$,

$$\text{lcm}(12, 18) = \frac{12 \cdot 18}{\gcd(12, 18)} = \frac{216}{6} = 36.$$

Podría parecer que esto nos da una implementación directa de lcm, pero aquí aparece una diferencia importante entre la teoría y la práctica. Matemáticamente,

$$\text{lcm}(a, b) = \frac{a \cdot b}{\gcd(a, b)} = \left(\frac{a}{\gcd(a, b)} \right) \cdot b$$

son expresiones equivalentes. Sin embargo, en una implementación real el orden de las operaciones sí importa. Si se usa un `int` de 32 bits, normalmente se pueden representar valores entre -2^{31} y $2^{31} - 1$, es decir, aproximadamente hasta 10^9 en magnitud. Con `long long`, que usualmente ocupa 64 bits, el rango llega de -2^{63} a $2^{63} - 1$, aproximadamente hasta 10^{18} .

Ejemplo (overflow en lcm). Si $a = 10^{10}$ y $b = 10^9$, el producto $ab = 10^{19}$ ya no cabe en un `long long`, y si se hace esa multiplicación primero aparece un *overflow*.¹ Aunque la fórmula sea correcta en el papel, conviene implementar

$$\text{lcm}(a, b) = a / \gcd(a, b) * b$$

dividiendo antes de multiplicar para mantener los valores en el rango.

Hasta aquí relacionamos $\text{lcm}(a, b)$ con $\gcd(a, b)$. Para *calcular* $\gcd(a, b)$ se usa el algoritmo de Euclides.

Algoritmo de euclides Para enteros a y b con $b \neq 0$,

$$\gcd(a, b) = \gcd(b, a \bmod b).$$

Cualquier divisor común de a y b también divide a $a \bmod b = a - \lfloor a/b \rfloor \cdot b$, y viceversa.

¹En programación, *overflow* significa que el resultado de una operación queda fuera del rango que el tipo de dato puede representar. Cuando eso ocurre, el valor almacenado deja de ser confiable.

Ejemplo. Si $a = 30$ y $b = 18$, entonces

$$30 \bmod 18 = 12 = 30 - 1 \cdot 18.$$

Los divisores comunes de 30 y 18 son $\{1, 2, 3, 6\}$, y esos mismos números también dividen a 12. De hecho,

$$\gcd(30, 18) = \gcd(18, 12) = 6.$$

Se repite el paso hasta obtener residuo 0; el último resto no nulo es $\gcd(a, b)$.

Implementación. Una versión recursiva típica es la siguiente:

```
1 long long gcd(long long a, long long b) {
2     return b == 0 ? a : gcd(b, a % b);
3 }
4
5 long long lcm(long long a, long long b) {
6     return a / gcd(a, b) * b;
7 }
```

La complejidad es $O(\log \min(a, b))$. En C++17 puedes usar directamente `std::gcd`.

6.2. Aritmética modular

Motivación: ciclos y residuos

El ejercicio [3.1] al final del capítulo pide: si hoy es lunes, ¿qué día será dentro de 300 días?

Los días se repiten cada 7, así que basta el residuo de 300 entre 7. Como $300 = 7 \cdot 42 + 6$, después de 294 días volvemos a lunes y faltan 6 días más, que desembocan en *domingo*. Si en lugar de 300 fueran 314 días, $314 = 7 \cdot 44 + 6$ deja el mismo residuo 6, así que la respuesta es la misma: dos cantidades que coinciden en resto módulo 7 representan el mismo “desfase” dentro del ciclo semanal.

Congruencias

Fijemos un entero $n \geq 1$.

Definición. Diremos que x es **congruente** a y **módulo** n , y escribimos $x \equiv y \pmod{n}$, si x y y dejan el mismo residuo al dividirse entre n , o equivalentemente si $n \mid (x - y)$.

Ejemplo. $17 \equiv 5 \pmod{12}$ porque $12 \mid (17 - 5)$. Análogamente, $31 \equiv 3 \pmod{7}$ porque $7 \mid (31 - 3)$.

Supongamos $a \equiv b \pmod{n}$, $c \equiv d \pmod{n}$ y sea m un entero positivo. Entonces:

Propiedad 1 (suma).

$$a + c \equiv b + d \pmod{n}.$$

Ejemplo. Como $17 \equiv 5 \pmod{12}$ y $8 \equiv 20 \pmod{12}$, se sigue $17 + 8 \equiv 5 + 20 \pmod{12}$.

Propiedad 2 (producto).

$$ac \equiv bd \pmod{n}.$$

Ejemplo. Con las mismas congruencias, $17 \cdot 8 \equiv 5 \cdot 20 \pmod{12}$.

Propiedad 3 (potencias).

$$a^m \equiv b^m \pmod{n}.$$

Ejemplo. Como $10 \equiv 3 \pmod{7}$, entonces $10^2 \equiv 3^2 \pmod{7}$.

En sumas y productos, cada término puede sustituirse por otro congruente módulo n sin alterar la congruencia del resultado.

Las congruencias ayudan a evitar *overflow*: si trabajamos módulo m , conviene reducir cada resultado intermedio con $a \% m$ (ajustando signos si hace falta).

Exponenciación modular. El problema de calcular $a^b \pmod{m}$ aparece constantemente. Por ejemplo, para 2^7 una forma ingenua es

$$2 \cdot 2 \cdot 2 \cdot 2 \cdot 2 \cdot 2 \cdot 2,$$

lo cual requiere 6 multiplicaciones. En cambio, si reutilizamos resultados intermedios, podemos hacer

$$2^2 = 4, \quad 2^4 = (2^2)^2 = 16, \quad 2^7 = 2^4 \cdot 2^2 \cdot 2,$$

y con ello reducimos notablemente el número de operaciones. Esta es la idea detrás de la exponenciación rápida: en lugar de construir la potencia multiplicando uno por uno, aprovechamos que podemos ir partiendo el exponente y recomblando resultados ya calculados.

Propiedad (exponenciación rápida). Para $b \geq 1$ se puede escribir (en la práctica con división entera de b entre 2 en cada paso):

$$a^b = \begin{cases} (a^{b/2})^2 & \text{si } b \text{ es par,} \\ a \cdot a^{b-1} & \text{si } b \text{ es impar.} \end{cases}$$

De aquí un algoritmo $O(\log b)$; en cada paso se puede reducir módulo m para obtener $a^b \pmod{m}$.

```
1 long long power(long long a, long long b, long long mod) {
2     long long res = 1;
3     a %= mod;
4     while (b > 0) {
5         //si b es impar se multiplica una vez extra
6         if (b % 2) res = res * a % mod;
7         a = a * a % mod;
8         b = b/2;
9     }
10    return res;
11 }
```

6.3. Números primos y criba de Eratóstenes

Definición. Un entero $p > 1$ es **primo** si sus únicos divisores positivos son 1 y p .

Ejemplo. Los números 2, 3, 5, 7 y 11 son primos; 4 no lo es, pues $4 = 2 \cdot 2$.

Teorema Fundamental de la Aritmética. Todo entero $n > 1$ se puede escribir de forma única (salvo el orden) como producto de potencias de primos.

Ejemplo.

$$12 = 2^2 \cdot 3, \quad 60 = 2^2 \cdot 3 \cdot 5.$$

Propiedad (prueba de primalidad hasta $\sqrt{n}$). Si n es un número compuesto, entonces existen enteros a y b con $n = ab$ y $1 < a, b < n$. No pueden cumplirse a la vez $a > \sqrt{n}$ y $b > \sqrt{n}$, pues entonces $ab > n$. Por tanto, todo compuesto posee un divisor d con $1 < d \leq \lfloor \sqrt{n} \rfloor$. Basta probar divisibilidad con los enteros en ese rango.

Ejemplo. Como $91 = 7 \cdot 13$ y $7 \leq \sqrt{91}$, al buscar factores de 91 basta llegar hasta $\lfloor \sqrt{91} \rfloor = 9$; al hallar 7 se concluye que 91 no es primo. Si ningún entero entre 2 y $\lfloor \sqrt{n} \rfloor$ divide a n , entonces n es primo. Esto da un algoritmo $O(\sqrt{n})$.

```
1 bool prime(long long n) {
2     if (n < 2) return false;
3     if (n % 2 == 0) return n == 2;
4     for (long long i = 3; i * i <= n; i += 2) {
5         if (n % i == 0) return false;
6     }
7     return true;
8 }
```

Para listar todos los primos hasta n , el método estándar es la **criba de Eratóstenes**: se mantiene una tabla y se tachan los múltiplos de cada primo hallado; el siguiente entero no tachado es primo. Se comienza en 2; sus múltiplos 4, 6, 8, ... se marcan compuestos; el siguiente no tachado es 3, y así sucesivamente.

Ejemplo (criba hasta 12). Al inicio la tabla es:

2	3	4	5	6	7	8	9	10	11	12
---	---	---	---	---	---	---	---	----	----	----

Después de procesar el primo 2, se tachan sus múltiplos:

2	3	4	5	6	7	8	9	10	11	12
---	---	---	---	---	---	---	---	----	----	----

Luego el siguiente número no tachado es 3, por lo que ahora se eliminan sus múltiplos que todavía sigan activos, como 9. Este método funciona porque si un número no ha sido tachado por ningún primo más pequeño, entonces no puede tener divisores primos menores y por lo tanto debe ser primo.

Al terminar este proceso para los números hasta 12, el estado final queda así:

2	3	4	5	6	7	8	9	10	11	12
---	---	---	---	---	---	---	---	----	----	----

Los números que permanecen sin tachar son los primos: 2, 3, 5, 7, 11.

Observaciones de implementación. El bucle exterior puede limitarse a i con $i^2 \leq n$: si un número compuesto tuviera un factor $> \sqrt{n}$, el cofactor sería $x \leq \sqrt{n}$ y ya se habría tachado antes. Además, al marcar múltiplos del primo i basta empezar en $j = i^2$, porque los múltiplos $2i, 3i, \dots, (i-1)i$ ya fueron tachados por primos menores.

```
1 vector<bool> prime(n + 1, true);
2 prime[0] = prime[1] = false;
```

```

3 for (int i = 2; i * i <= n; i++) {
4     if (prime[i]) {
5         for (int j = i * i; j <= n; j += i) {
6             prime[j] = false;
7         }
8     }
9 }

```

Complejidad. La criba anterior corre en $O(n \log \log n)$, prácticamente lineal; suele ser viable hasta $n \approx 10^7$ con memoria razonable. Para n mayor o para tener el *menor factor primo* de cada entero (útil para factorizar rápido) existen variantes en tiempo $O(n)$; véase la referencia [1].

6.4. Teorema de Bézout

Teorema (Bézout). Para cualesquiera enteros a y b no ambos cero, existen enteros x e y tales que

$$ax + by = \gcd(a, b).$$

Ejemplo (coeficientes). Si $a = 12$ y $b = 18$, entonces $\gcd(12, 18) = 6$ y una elección es

$$12(-1) + 18(1) = 6,$$

es decir $x = -1$, $y = 1$. Otra elección válida es $12(2) + 18(-1) = 6$, o sea $x = 2$, $y = -1$. Los enteros x e y no son únicos.

Propiedad (familia de soluciones). Si (x_0, y_0) satisface $ax_0 + by_0 = \gcd(a, b)$ y $g = \gcd(a, b)$, entonces para todo $k \in \mathbb{Z}$,

$$\left(x_0 + k \frac{b}{g}, y_0 - k \frac{a}{g}\right)$$

también es solución de $ax + by = g$.

Propiedad (solvabilidad de $ax + by = c$). La ecuación

$$ax + by = c$$

tiene solución (x, y) en enteros si y sólo si $\gcd(a, b) \mid c$.

Ejemplo. La ecuación $12x + 18y = 6$ tiene solución porque $\gcd(12, 18) = 6$ divide 6. En cambio $12x + 18y = 5$ no tiene solución en enteros, pues $6 \nmid 5$.

Algoritmo de Euclides extendido: calcula $\gcd(a, b)$ y además encuentra x, y con

$$ax + by = \gcd(a, b)$$

La idea del algoritmo extendido es seguir los mismos pasos del Algoritmo de Euclides pero rastrear cómo se expresa cada residuo como combinación lineal de a y m . En cada paso de la división $r_{i-1} = q_i \cdot r_i + r_{i+1}$ se actualizan los coeficientes:

$$x_{i+1} = x_{i-1} - q_i \cdot x_i \quad y_{i+1} = y_{i-1} - q_i \cdot y_i$$

partiendo de $(x_0, y_0) = (1, 0)$ para a y $(x_1, y_1) = (0, 1)$ para b .

Ejemplo (Euclides extendido, $a = 30$, $b = 18$).

Paso 1. Divisiones de Euclides:

$$30 = 1 \cdot 18 + 12$$

$$18 = 1 \cdot 12 + 6$$

$$12 = 2 \cdot 6 + 0.$$

Como el último residuo no nulo es 6, concluimos que

$$\gcd(30, 18) = 6.$$

Paso 2. Escribimos ese 6 en función de los residuos anteriores:

$$6 = 18 - 1 \cdot 12.$$

Paso 3. Sustituimos ahora la expresión de 12 que apareció en el primer paso:

$$12 = 30 - 1 \cdot 18.$$

Al reemplazarla en la ecuación anterior, obtenemos

$$6 = 18 - (30 - 18).$$

Paso 4. Simplificamos:

$$6 = 30 \cdot (-1) + 18 \cdot (2).$$

Por lo tanto, una elección válida de coeficientes de Bézout es

$$x = -1, \quad y = 2.$$

```
1 // calcula gcd(a,b) y encuentra x,y tales que ax + by = gcd(a,b)
2 long long gcdExt(long long a, long long b, long long &x, long long &y) {
3     //Caso base pues gcd(a,0)=a
4     if (b == 0) {
5         x = 1; y = 0;
6         return a;
7     }
8     long long x1, y1;
9     long long g = gcdExt(b, a % b, x1, y1);
10    x = y1;
11    y = x1 - (a / b) * y1;
12    return g;
13 }
```

La complejidad es la misma que Euclides: $O(\log \min(a, b))$.

6.5. Inverso multiplicativo modular

En aritmética usual, el inverso de $a \neq 0$ es $\frac{1}{a}$, es decir el número que cumple $a \cdot \frac{1}{a} = 1$. En congruencias se busca lo mismo *módulo* m .

Definición. Dados enteros a y m con $m \geq 2$, un **inverso multiplicativo** de a módulo m es un entero a^{-1} tal que

$$a \cdot a^{-1} \equiv 1 \pmod{m}.$$

Ejemplo. El inverso de 3 módulo 7 es 5, pues $3 \cdot 5 = 15 \equiv 1 \pmod{7}$.

En problemas suele “dividirse” módulo m multiplicando por el inverso:

$$\frac{a}{b} \pmod{m} \equiv a \cdot b^{-1} \pmod{m},$$

siempre que b^{-1} exista módulo m .

Ejemplo (combinaciones módulo 7). Primero recordemos la definición usual:

$$\binom{5}{2} = \frac{5!}{2!3!} = \frac{120}{2 \cdot 6} = 10.$$

Ahora llevamos la misma idea a módulo 7: como no dividimos directamente en aritmética modular, reemplazamos cada división por un inverso:

$$\binom{5}{2} \equiv 5! \cdot (2!)^{-1} \cdot (3!)^{-1} \pmod{7}.$$

Aquí $5! \equiv 1$, $(2!)^{-1} \equiv 2^{-1} \equiv 4$ y $(3!)^{-1} \equiv 6^{-1} \equiv 6 \pmod{7}$, luego

$$\binom{5}{2} \equiv 1 \cdot 4 \cdot 6 = 24 \equiv 3 \pmod{7},$$

y coincide con $10 \equiv 3 \pmod{7}$.

Este enfoque aparece con frecuencia en la ICPC con módulos primos grandes (por ejemplo $10^9 + 7$).

Ejemplo (inverso inexistente). Con $m = 4$ y $a = 2$, si existiera b con $2b \equiv 1 \pmod{4}$ tendríamos un imposible, porque

$$2 \cdot 0 \equiv 0, \quad 2 \cdot 1 \equiv 2, \quad 2 \cdot 2 \equiv 0, \quad 2 \cdot 3 \equiv 2 \pmod{4},$$

y el residuo 1 nunca aparece al multiplicar por 2 módulo 4.

Propiedad (existencia del inverso). El inverso de a módulo m existe si y sólo si $\gcd(a, m) = 1$.

En efecto, las siguientes afirmaciones son equivalentes:

1. Existe un entero a^{-1} tal que $a \cdot a^{-1} \equiv 1 \pmod{m}$.
2. Existe un entero x tal que $ax \equiv 1 \pmod{m}$.
3. Existen enteros x e y tales que

$$ax - my = 1.$$

4. Se cumple que $\gcd(a, m) \mid 1$, y por tanto $\gcd(a, m) = 1$.

El paso de la segunda a la tercera afirmación es solo reescribir la congruencia como igualdad de enteros. El paso final es el teorema de Bézout aplicado a la ecuación $ax + (-m)y = 1$.

Si $\gcd(a, m) = 1$, el algoritmo de Euclides extendido produce x, y con $ax + my = 1$, es decir $ax \equiv 1 \pmod{m}$; ese x es un inverso de a módulo m .

```

1 long long inv(long long a, long long m) {
2     long long x, y;
3     gcdExt(a, m, x, y);
4     return (x % m + m) % m;
5 }

```

Nótese que al final en lugar de devolver x , devolvemos $(x \% m + m) \% m$, esto es porque el coeficiente x que devuelve `gcdExt` puede ser negativo. Por ejemplo, el inverso de 3 módulo 7 es 5, pero el algoritmo puede devolver $x = -2$, que es equivalente pues $-2 \equiv 5 \pmod{7}$. La expresión $(x \% m + m) \% m$ convierte cualquier x a un número congruente a x positivo en $[0, m)$.

Cuando $m = p$ es primo y $\gcd(a, p) = 1$, el pequeño teorema de Fermat implica que

$$a^{p-1} \equiv 1 \pmod{p}.$$

Multiplicando ambos lados por a^{-1} se obtiene $a^{p-2} \equiv a^{-1} \pmod{p}$. Así, el inverso modular se puede calcular también como una potencia, usando exponenciación modular en tiempo $O(\log p)$. Esta vía es útil cuando ya se tiene implementada la exponenciación rápida y no se quiere depender de `gcdExt` en ese momento.

Inversos para cada número módulo m

A veces necesitamos el inverso de cada $i \in \{1, \dots, m-1\}$ a la vez. Llamar `gcdExt` por cada i cuesta del orden de $O(m \log m)$.

Cuando el módulo es un primo p , existe un método lineal $O(p)$ que rellena `inv[i]`. Partimos de `inv[1] = 1`. Para $i \geq 2$, la división euclidiana

$$p = \left\lfloor \frac{p}{i} \right\rfloor \cdot i + (p \bmod i)$$

y manipulación módulo p llevan a la recurrencia usual (equivalente a la que implementa el código de abajo).

Propiedad (recurrencia de inversos módulo p primo). Para $2 \leq i < p$,

$$i^{-1} \equiv - \left\lfloor \frac{p}{i} \right\rfloor \cdot (p \bmod i)^{-1} \pmod{p},$$

donde $(p \bmod i)^{-1}$ ya está en `inv[p mód i]`. Esto funciona porque $p \bmod i$ siempre cae entre 1 e $i-1$ (cuando $2 \leq i < p$), así que ese índice corresponde a un valor ya calculado en iteraciones anteriores.

Ejemplo ($p = 7$). La tabla queda:

i	1	2	3	4	5	6
<code>inv[i]</code>	1	4	5	2	3	6

Comprobación rápida: $2 \cdot 4 = 8 \equiv 1 \pmod{7}$ y $3 \cdot 5 = 15 \equiv 1 \pmod{7}$. Aquí p debe ser primo y normalmente $m = p$.

```

1 vector<long long> inversos(int m, long long p) {
2     vector<long long> inv(m);
3     inv[1] = 1;
4     for (int i = 2; i < m; i++)
5         inv[i] = (p - (p / i) * inv[p % i] % p) % p;
6     return inv;
7 }

```

Pequeño teorema de Fermat

Diremos que dos enteros son **primos relativos** o **coprimos** si su máximo común divisor es 1.

Pequeño teorema de Fermat: Si p es primo y $\gcd(a, p) = 1$, entonces

$$a^{p-1} \equiv 1 \pmod{p}.$$

En particular, si multiplicamos ambos lados por a^{-1} (que existe módulo p porque $\gcd(a, p) = 1$), obtenemos

$$a^{p-2} \equiv a^{-1} \pmod{p},$$

que es la identidad que ya usamos arriba para calcular inversos módulo un primo.

Ejemplo. Con $p = 7$ y $a = 2$, se cumple $2^6 = 64 \equiv 1 \pmod{7}$.

Teorema de Euler

El pequeño teorema de Fermat aplica cuando el módulo es un primo p . El teorema de Euler generaliza esta idea a cualquier entero $m \geq 2$: si $\gcd(a, m) = 1$, entonces una potencia de a vuelve a dar 1 módulo m , y ese exponente es $\phi(m)$, donde $\phi(m)$ cuenta los residuos invertibles módulo m .

Teorema de Euler: Sea $m \geq 2$ un entero y sea $\phi(m)$ el número de enteros en $\{1, 2, \dots, m\}$ que son coprimos con m (la **función ϕ de Euler**). Si $\gcd(a, m) = 1$, entonces

$$a^{\phi(m)} \equiv 1 \pmod{m}.$$

En el caso particular $m = p$ primo, los enteros coprimos con p entre 1 y p son exactamente $1, 2, \dots, p-1$, luego $\phi(p) = p-1$. Sustituyendo en la fórmula de Euler obtenemos $a^{p-1} \equiv 1 \pmod{p}$ para todo a no divisible entre p : es decir, el pequeño teorema de Fermat es el caso $m = p$ del teorema de Euler, no un resultado independiente con otra lógica.

Ejemplo. Sea $m = 10$. Los enteros entre 1 y 10 que son coprimos con 10 son 1, 3, 7 y 9, por lo que $\phi(10) = 4$. Si tomamos $a = 3$, entonces $\gcd(3, 10) = 1$ y se cumple

$$3^4 = 81 \equiv 1 \pmod{10}.$$

6.6. Congruencias lineales y Teorema Chino del Residuo

Congruencia lineal

Ahora que sabemos calcular inversos modulares, podemos abordar las **congruencias lineales**: dado un módulo m y enteros a y b , buscamos enteros x que satisfagan

$$ax \equiv b \pmod{m}.$$

Al igual que con los inversos, estas ecuaciones *no siempre* tienen solución.

Propiedad (existencia de solución). Sea $g = \gcd(a, m)$. La congruencia $ax \equiv b \pmod{m}$ tiene al menos una solución si y sólo si $g \mid b$.

Si $g = 1$, entonces $\gcd(a, m) = 1$ y a tiene inverso módulo m . Multiplicando la congruencia por a^{-1} obtenemos una solución explícita:

$$x \equiv ba^{-1} \pmod{m}.$$

Si $g > 1$ pero aún $g \mid b$, escribimos $a = ga'$, $m = gm'$ y $b = gb'$ con enteros a' , m' y b' . Se cumple $\gcd(a', m') = 1$ (al dividir a y m por su máximo común divisor). Además, $m \mid (ax - b)$ si y sólo si $m' \mid (a'x - b')$, es decir

$$a'x \equiv b' \pmod{m'},$$

que es el caso coprimo y se resuelve multiplicando por el inverso de a' módulo m' .

Ejemplo. La congruencia $4x \equiv 3 \pmod{8}$ no tiene solución, porque $g = \gcd(4, 8) = 4$ y $4 \nmid 3$. En cambio, $10x \equiv 6 \pmod{14}$ sí tiene: aquí $g = \gcd(10, 14) = 2$ divide 6. Dividiendo entre 2 obtenemos $5x \equiv 3 \pmod{7}$. Como $5^{-1} \equiv 3 \pmod{7}$ (pues $5 \cdot 3 = 15 \equiv 1$), resulta $x \equiv 3 \cdot 3 \equiv 2 \pmod{7}$, y módulo 14 las soluciones son $x \equiv 2$ y $x \equiv 9$.

Teorema chino del residuo

Teorema (chino del residuo, dos congruencias). Sean $n_1, n_2 \geq 1$ y a_1, a_2 enteros. El sistema

$$\begin{aligned} x &\equiv a_1 \pmod{n_1}, \\ x &\equiv a_2 \pmod{n_2}, \end{aligned}$$

tiene solución si y sólo si $a_1 \equiv a_2 \pmod{\gcd(n_1, n_2)}$. Cuando hay solución, es única módulo $\text{lcm}(n_1, n_2)$.

La construcción que sigue obtiene una solución explícita y explica cómo fusionar las dos congruencias.

De la primera ecuación podemos escribir $x = a_1 + n_1b_1$ para algún entero b_1 . Sustituyendo en la segunda, $a_1 + n_1b_1 \equiv a_2 \pmod{n_2}$, es decir

$$n_1b_1 \equiv a_2 - a_1 \pmod{n_2}.$$

Sea $d = \gcd(n_1, n_2)$. La congruencia en b_1 tiene solución si y sólo si $d \mid (a_2 - a_1)$, pues n_1b_1 recorre, al variar b_1 , justamente los múltiplos de d módulo n_2 . Cuando se cumple la divisibilidad, conviene usar el algoritmo de Euclides extendido: hallamos enteros x' e y' tales que $n_1x' + n_2y' = d$, multiplicamos por $(a_2 - a_1)/d$ y obtenemos

$$n_1x' \frac{a_2 - a_1}{d} + n_2y' \frac{a_2 - a_1}{d} = a_2 - a_1.$$

En particular, tomando $b_1 = x'(a_2 - a_1)/d$ se satisface $n_1b_1 \equiv a_2 - a_1 \pmod{n_2}$ (el sumando con n_2 es múltiplo de n_2). Entonces una solución del sistema es

$$x = a_1 + n_1b_1 = a_1 + n_1x' \frac{a_2 - a_1}{d}.$$

Si x_1 y x_2 son dos soluciones, entonces $x_1 \equiv x_2 \pmod{n_1}$ y $x_1 \equiv x_2 \pmod{n_2}$, luego $x_1 - x_2$ es múltiplo de n_1 y de n_2 , y por tanto de $\text{lcm}(n_1, n_2)$. Es decir, las soluciones son únicas módulo $\text{lcm}(n_1, n_2)$ (y en particular hay infinitas soluciones en $\mathbb{Z}$ si las hay).

En implementaciones conviene reducir módulo $\text{lcm}(n_1, n_2)$ (o módulos intermedios) en cada paso para controlar el *overflow*.

El mismo procedimiento se extiende a k congruencias $x \equiv a_i \pmod{n_i}$: se fusionan dos a dos, reemplazando cada par por una sola congruencia módulo el mínimo común múltiplo de los módulos involucrados, hasta obtener una congruencia única. Una cota razonable de coste es $O(k \log \text{lcm}(n_1, \dots, n_k))$ si en cada paso se usa Euclides extendido sobre enteros acotados por dicho mcm.

Ejemplo. Busquemos x que satisfaga el sistema

$$\begin{aligned} x &\equiv 2 \pmod{3}, \\ x &\equiv 3 \pmod{5}. \end{aligned}$$

Escribimos $x = 2 + 3b$; entonces $2 + 3b \equiv 3 \pmod{5}$, es decir $3b \equiv 1 \pmod{5}$. Como $3^{-1} \equiv 2 \pmod{5}$, obtenemos $b \equiv 2 \pmod{5}$. Con $b = 2$ resulta $x = 2 + 3 \cdot 2 = 8$, y en efecto $8 \equiv 2 \pmod{3}$ y $8 \equiv 3 \pmod{5}$. Como $\text{gcd}(3, 5) = 1$, el módulo de unicidad es $\text{lcm}(3, 5) = 15$, así que todas las soluciones son $x \equiv 8 \pmod{15}$.

6.7. Funciones multiplicativas

Definición. Una función $f: \mathbb{Z}^+ \rightarrow \mathbb{Z}$ es **multiplicativa** si

$$f(ab) = f(a)f(b) \quad \text{cuando } \text{gcd}(a, b) = 1.$$

Si conocemos f en potencias de primos, podemos calcular $f(n)$ para cualquier n con factorización $n = p_1^{k_1} \dots p_r^{k_r}$: los factores $p_i^{k_i}$ son coprimos dos a dos, así que $f(n) = f(p_1^{k_1}) \dots f(p_r^{k_r})$.

Función ϕ de Euler

$\phi(n)$ cuenta cuántos enteros del conjunto $\{1, 2, \dots, n\}$ son coprimos con n (es decir, $\text{gcd}(k, n) = 1$). Equivale al orden del grupo multiplicativo $(\mathbb{Z}/n\mathbb{Z})^\times$ cuando $n \geq 2$.

Propiedad (multiplicatividad). Si $\text{gcd}(a, b) = 1$, entonces $\phi(ab) = \phi(a)\phi(b)$.

Ejemplo. Como $12 = 2^2 \cdot 3$ y $\text{gcd}(4, 3) = 1$, por multiplicatividad

$$\phi(12) = \phi(4)\phi(3).$$

Usando las fórmulas de potencias primas,

$$\phi(4) = 2^2 - 2^1 = 2, \quad \phi(3) = 3 - 1 = 2,$$

de modo que

$$\phi(12) = 2 \cdot 2 = 4.$$

En efecto, los coprimos con 12 entre 1 y 12 son 1, 5, 7 y 11.

Propiedad (primos y potencias). Si p es primo y $k \geq 1$, entonces

$$\phi(p) = p - 1, \quad \phi(p^k) = p^k - p^{k-1} = p^{k-1}(p - 1).$$

En efecto, en $\{1, \dots, p^k\}$ hay exactamente p^{k-1} múltiplos de p , que son precisamente los enteros que *no* son coprimos con p^k .

Propiedad (suma sobre divisores). Para todo entero $n \geq 1$,

$$\sum_{d|n} \phi(d) = n.$$

Ejemplo: Para $n = 12$, sus divisores positivos son

$$1, 2, 3, 4, 6, 12.$$

Entonces

$$\phi(1) + \phi(2) + \phi(3) + \phi(4) + \phi(6) + \phi(12) = 1 + 1 + 2 + 2 + 2 + 4 = 12,$$

Cálculo de $\phi(n)$ por factorización. El siguiente código recorre los primos que dividen a n y actualiza un acumulador según $\phi(p^k) = p^k - p^{k-1}$ implícitamente (resta una fracción por cada p). Tiene complejidad $O(\sqrt{n})$.

```
1 long long phi(long long n) {
2     long long res = n;
3     for (long long p = 2; p * p <= n; p++)
4         if (n % p == 0) {
5             while (n % p == 0) {
6                 n /= p;
7             }
8             res -= res / p;
9         }
10    if (n > 1) {
11        res -= res / n;
12    }
13    return res;
14 }
```

Criba de totientes en $[1, n]$. A veces necesitamos $\phi(i)$ para todo $1 \leq i \leq n$. Repetir el algoritmo anterior para cada i cuesta $O(n\sqrt{n})$. Con la misma idea que en la criba de Eratóstenes, para cada primo p recorremos sus múltiplos j y actualizamos $\text{phi}[j]$ restando la contribución del factor p . La complejidad es $O(n \log \log n)$.

```
1 vector<long long> cribaPhi(int n) {
2     vector<long long> phi(n + 1);
3     // inicializamos phi[i] = i
4     for (int i = 1; i <= n; i++) {
5         phi[i] = i;
6     }
7     for (int i = 2; i <= n; i++) {
8         // si phi[i] sigue siendo i, entonces i es primo
9         if (phi[i] == i) {
10             for (int j = i; j <= n; j += i) {
11                 phi[j] -= phi[j] / i;
12             }
13         }
14     }
15     return phi;
16 }
```

```

12     }
13     }
14 }
15     return phi;
16 }

```

Ejemplo (arreglo `phi` para $n = 10$). Tras la inicialización `phi[i] = i` para $1 \leq i \leq 10$, el arreglo queda:

i	1	2	3	4	5	6	7	8	9	10
<code>phi[i]</code>	1	2	3	4	5	6	7	8	9	10

Al terminar la criba:

i	1	2	3	4	5	6	7	8	9	10
<code>phi[i]</code>	1	1	2	2	4	2	6	4	6	4

que coincide con $\phi(1), \dots, \phi(10)$ (por ejemplo $\phi(10) = 4$ porque 1, 3, 7 y 9 son coprimos con 10).

Conexión con exponenciación modular. Más arriba en este capítulo vimos el teorema de Euler: si $\gcd(a, m) = 1$ y $m \geq 2$, entonces $a^{\phi(m)} \equiv 1 \pmod{m}$. Cuando m es primo, $\phi(m) = m - 1$ se reduce a el pequeño teorema de Fermat. Por eso, al calcular $a^n \pmod{m}$ con n grande, en la práctica aparece a menudo $\phi(m)$ al reducir exponentes.

Función de Möbius μ

La función de Möbius $\mu : \mathbb{Z}^+ \rightarrow \{-1, 0, 1\}$ se define por

$$\mu(n) = \begin{cases} 1 & \text{si } n = 1, \\ (-1)^k & \text{si } n \text{ es producto de } k \text{ primos distintos,} \\ 0 & \text{si } n \text{ es divisible por un cuadrado } > 1. \end{cases}$$

Es multiplicativa: si $\gcd(a, b) = 1$, entonces $\mu(ab) = \mu(a)\mu(b)$.

Ejemplo. $\mu(1) = 1$. Además, $\mu(30) = \mu(2 \cdot 3 \cdot 5) = (-1)^3 = -1$ porque es producto de 3 primos distintos. En cambio, $\mu(12) = 0$ ya que 12 es divisible por 2^2 .

Propiedad (inversión de Möbius). Si $g(n) = \sum_{d|n} f(d)$ para todo n , entonces

$$f(n) = \sum_{d|n} \mu(d) g(n/d).$$

Recíprocamente, esta fórmula determina f a partir de g .

Ejemplo (inversión en un caso sencillo). Si $f(n) = 1$ para todo n , entonces

$$g(n) = \sum_{d|n} f(d) = \sum_{d|n} 1 = \tau(n),$$

la cantidad de divisores de n . Aplicando inversión de Möbius,

$$f(n) = \sum_{d|n} \mu(d) g(n/d) = \sum_{d|n} \mu(d) \tau(n/d) = 1,$$

lo cual recupera exactamente la función original.

Propiedad (suma de μ sobre divisores). Para todo $n \geq 1$,

$$\sum_{d|n} \mu(d) = \begin{cases} 1 & \text{si } n = 1, \\ 0 & \text{si } n > 1. \end{cases}$$

Usaremos la notación de **Iverson**: $[P]$ vale 1 si la afirmación P es verdadera y 0 si no; por ejemplo $[\gcd(4, 8) = 4] = 1$. Entonces $\sum_{d|n} \mu(d) = [n = 1]$.

Ejemplo (pares coprimos). Contemos los pares (x, y) con $1 \leq x, y \leq n$ y $\gcd(x, y) = 1$. La respuesta es

$$f(n) = \sum_{j=1}^n \sum_{i=1}^n [\gcd(i, j) = 1].$$

Un barrido doble sería $O(n^2)$. En cambio, usando $\sum_{d|\gcd(i, j)} \mu(d) = [\gcd(i, j) = 1]$ y el hecho de que $d | \gcd(i, j)$ equivale a $d | i$ y $d | j$,

$$[\gcd(i, j) = 1] = \sum_{d=1}^n [d | i][d | j] \mu(d).$$

Sustituyendo e intercambiando el orden de las sumas,

$$f(n) = \sum_{d=1}^n \mu(d) \left(\sum_{i=1}^n [d | i] \right) \left(\sum_{j=1}^n [d | j] \right) = \sum_{d=1}^n \mu(d) \left\lfloor \frac{n}{d} \right\rfloor^2.$$

Con una criba para μ en $[1, n]$ (como la de abajo) y un ciclo sobre d , la complejidad es $O(n)$.

Criba de μ . Una implementación en tiempo lineal (variante de criba lineal) es la siguiente:

```

1 vector<int> cribaMu(int n) {
2     vector<int> mu(n + 1, 1);
3     vector<bool> compuesto(n + 1, false);
4     vector<int> primos;
5     for (int i = 2; i <= n; i++) {
6         //revisar si es primo
7         if (!compuesto[i]) {
8             primos.push_back(i); mu[i] = -1;
9         }
10        for (int p : primos) {
11            if (i * p > n){
12                break;
13            }
14            compuesto[i * p] = true;
15            //revisar si tiene cuadrados
16            if (i % p == 0) {
17                mu[i * p] = 0;
18                break;
19            }
20            //mu(ip)=mu(i)mu(p)=-mu(i)
21            mu[i * p] = -mu[i];
22        }
23    }
24    return mu;
25 }
```

Ejemplo (arreglo μ para $n = 10$). Tras la criba lineal:

i	1	2	3	4	5	6	7	8	9	10
$\mu[i]$	1	-1	-1	0	-1	1	-1	0	0	1

Los ceros corresponden a enteros divisibles por un cuadrado (4, 8, 9, etc.).

6.8. Logaritmo discreto y Baby-step Giant-step

Problema (logaritmo discreto). Fijados un módulo $m \geq 2$ y enteros a y b con $\gcd(a, m) = 1$, se busca un entero x (si existe) tal que

$$a^x \equiv b \pmod{m}.$$

A tal x se le llama **logaritmo discreto** de b en base a módulo m . Es el análogo modular de resolver $a^x = b$ en los reales; aquí x solo queda determinado salvo múltiplos del orden de a en el grupo de unidades módulo m .

Ejemplo (existencia y no existencia). Módulo 7, las potencias de 3 son $3^0 \equiv 1$, $3^1 \equiv 3$, $3^2 \equiv 2$, $3^3 \equiv 6$, $\dots$; por tanto $x = 2$ resuelve $3^x \equiv 2 \pmod{7}$. En cambio $2^x \equiv 3 \pmod{7}$ no tiene solución, porque $2^x \pmod{7}$ solo toma los valores 1, 2 y 4.

Fuerza bruta. Recorrer $x = 0, 1, \dots, m-1$ tiene coste $O(m)$.

Baby-step Giant-step (BSGS). Supongamos $\gcd(a, m) = 1$. Sea $n = \lceil \sqrt{m} \rceil$. Todo x en un rango razonable puede escribirse como $x = in - j$ con $0 \leq i, j \leq n$. Entonces

$$a^{in-j} \equiv b \pmod{m} \iff a^{in} \equiv ba^j \pmod{m}.$$

Los *baby steps* precálculan $b \cdot a^j \pmod{m}$ para cada j y los guardan en una tabla (por ejemplo un `unordered_map`). Los *giant steps* recorren i y comparan $a^{in} \pmod{m}$ con esa tabla; si hay coincidencia, se recupera $x = in - j$. La complejidad es $O(\sqrt{m})$ en tiempo y memoria (usando la función `power` de la sección de exponenciación modular para calcular a^{in}).

Ejemplo (BSGS paso a paso). Resolvamos

$$3^x \equiv 13 \pmod{17}.$$

Aquí $m = 17$, así que tomamos $n = \lceil \sqrt{17} \rceil = 5$. Buscamos una representación $x = in - j$.

Baby steps. Calculamos $13 \cdot 3^j \pmod{17}$ para $j = 0, \dots, 5$:

j	0	1	2	3	4	5
$13 \cdot 3^j \pmod{17}$	13	5	15	11	16	14

Guardamos esos valores en una tabla $\text{valor} \rightarrow j$.

Giant steps. Calculamos $3^{5i} \pmod{17}$ para $i = 1, 2, \dots$:

$$3^5 \equiv 5, \quad 3^{10} \equiv 8, \quad 3^{15} \equiv 6 \pmod{17}.$$

En $i = 1$ aparece una coincidencia inmediata: $3^5 \equiv 5$, y en la tabla de baby steps ese valor corresponde a $j = 1$. Entonces

$$x = in - j = 1 \cdot 5 - 1 = 4.$$

Verificación:

$$3^4 = 81 \equiv 13 \pmod{17}.$$

Así, la solución es $x \equiv 4 \pmod{16}$ (misma clase al considerar el orden de 3 módulo 17).

```

1 long long bsgs(long long a, long long b, long long m) {
2     a %= m; b %= m;
3     if (b == 1 || m == 1){
4         return 0;
5     }
6
7     //calculamos solo una vez el techo de la raiz
8     long long n = ceil(sqrt((double)m));
9     unordered_map<long long, long long> baby;
10
11    //baby steps guardar b*a^j para j = 0..n
12    long long aj = b;
13    for (long long j = 0; j <= n; j++) {
14        baby[aj] = j;
15        aj = aj * a % m;
16    }
17
18    //Usamos la exponenciacion binaria que vimos antes
19    long long an = power(a, n, m);
20    long long cur = an;
21    for (long long i = 1; i <= n; i++) {
22        if (baby.count(cur)) {
23            //recuperar el exponente
24            long long x = i * n - baby[cur];
25            if (x > 0){
26                return x;
27            }
28        }
29        cur = cur * an % m;
30    }
31    return -1;
32 }

```

Para módulo no primo o cuando $\gcd(a, m) > 1$, hay variantes que reducen al caso coprime; véase la referencia [2].

6.9. Ejercicios

- Strange Function (Codeforces 1542C)
- Sum of Divisors (CSES 1082)
- Monkey Tradition (Light OJ)

6.10. Problemas

- Exponentiation II (CSES 1712)
- Hossam and Trainees (CSES 1771C)

- Fantastic Beasts (Latam Regional Contest 2028)
- Digits Sum (Codeforces 1553A)
- Two Divisors (Codeforces 1916B)
- Bermuda Triangle (Codeforces 2097C)
- GCD Festival (COMPFEST 13)

Capítulo 7

Combinatoria y conteo

Desde inicios de la civilización el contar ha sido de utilidad, y aunque para cosas simples como contar el cambio de una compra basta con contar moneda por moneda, cuando queremos contar cosas más grandes como el número de placas posibles en la ciudad de México o la cantidad de diferentes manos de poker que puedes tener, contar una por una no es un método eficiente ni confiable pues lo más probable es que cometamos un error, es aquí donde entra la necesidad de otras técnicas que estudiaremos a continuación.

7.1. Bases

Principio de multiplicación

Supongamos que vamos a comprar un helado en una tienda donde puedes escoger cono de sabor vainilla o chocolate, helado de fresa, mango o chicle y de toppings chispas de chocolate o cereal, ¿cuántas maneras diferentes hay de elegir un helado diferente?

Pensemos que así como podemos elegir la combinación vainilla, fresa, chispitas podemos elegir la combinación chocolate, fresa, chispitas o vainilla, mango, cereal, hay bastantes formas de combinar los sabores y notemos que el hecho de que elijamos un sabor de cono, no limita para nada nuestra elección de helado o de topping. Es así como llegamos a la conclusión que por cada sabor de cono podemos elegir cualquiera de los 3 helados y cualquiera de los 2 toppings, y cada combinación será diferente, de donde tendremos $2 \times 3 \times 2 = 12$ formas de elegir un helado.

Esto se llama **principio de multiplicación** y nos dice que si una tarea se divide en k etapas **independientes**, es decir la elección de una etapa no limita las opciones de las demás, con $n_1, n_2, \dots, n_k$ formas de realizar cada una, el número total de formas de completar la tarea es $n_1 \cdot n_2 \cdots n_k$.

Principio de adición

Ahora supongamos que en la heladería nos dicen que además de los helados, ofrecen aguas frescas de 5 sabores, si quisiéramos solamente comprar un solo artículo de la heladería, ¿de cuántas formas podríamos hacerlo? Pues bien, ya vimos que tenemos 12 helados diferentes, a esto le sumaremos las 5 opciones de agua pues al comprar un sólo artículo tenemos que elegir una de las dos opciones y así obtenemos que hay 17 formas diferentes de realizar la tarea.

Esto se llama **principio de adición** y nos dice que si una tarea puede completarse de A o de B formas, donde A y B son mutuamente excluyentes, el total es $|A| + |B|$.

Principio del complemento

Imaginemos que queremos comprar un helado que no sea de chicle, ¿de cuántas formas podemos hacerlo?, calcularlo directamente sería ver todas las opciones que no tienen chicle, a veces esta tarea es más complicada que calcular lo opuesto, es decir, vamos a contar cuántos helados diferentes tienen chicle, notemos que si el sabor de helado está fijo, tenemos 2 opciones de cono y 2 de toppings, es decir 4 opciones. Ahora podemos simplemente restar a los 12 que teníamos, la cantidad que sí tienen chicle, dejándonos con 8. Esta idea se llama **principio del complemento** y nos conviene usarla cuando los casos no deseados tienen una estructura más simple que los deseados.

Principio del palomar

En la heladería además hay una promoción para motivar la convivencia, si llegan dos personas con el mismo cumpleaños juntas, se les regalará a ambas personas un helado, ¿cuántas personas mínimas se necesitan para asegurar que alguien tendrá un helado gratis? Podríamos pensar que se necesitan muchísimas, pero en realidad solo necesitamos 367, esto porque en el peor caso cada una de las primeras 366 personas va a cumplir un día diferente (incluyendo 29 de febrero) y cuando llegue la persona número 367 como todos los cumpleaños ya tienen una persona que cumple ese día, forzosamente compartirá cumpleaños con alguien. Con la misma lógica, en una escuela de 5000 estudiantes no basta con saber que *algún* par comparte cumpleaños: el principio del palomar también nos dice cuántas personas pueden caer, como mínimo, en el *mismo* día. Hay 366 “casillas” posibles (los días del año, contando 29 de febrero); si repartimos 5000 estudiantes lo más parejo posible entre esas casillas, a cada día le tocan al menos $\lfloor 5000/366 \rfloor = 13$ personas, pero sobran $5000 - 366 \cdot 13 = 242$ estudiantes que obligan a que al menos 242 días tengan una persona extra. En consecuencia, **algún** cumpleaños concentra al menos 14 personas ($\lceil 5000/366 \rceil = 14$): en el peor reparto posible habrá un día con catorce estudiantes que cumplen ese mismo día (y por tanto comparten cumpleaños entre sí). A esta idea se le llama **principio del palomar**.

7.2. Permutaciones

Supongamos que queremos escoger el nuevo CEO y VP de una lista de 5 candidatos y queremos saber de cuántas formas diferentes se puede hacer, a esto se le llama **permutación**: estamos *ordenando* una selección de 2 personas distintas entre 5, porque el cargo importa ($\text{CEO} \neq \text{VP}$).

Ejemplo (cómo agrupar las opciones). Si los candidatos son A, B, C, D, E , una forma natural de organizar las 20 parejas es **fijar primero al CEO** y listar quién puede ser VP:

- CEO A : VP puede ser $B, C, D, E \rightarrow (A, B), (A, C), (A, D), (A, E)$ (4 parejas);
- CEO B : VP puede ser $A, C, D, E \rightarrow$ 4 parejas;
- y así con CEO C, D o E .

Son 5 grupos de 4 parejas cada uno, o sea $5 \cdot 4 = 20$ en total. Otra agrupación equivalente es fijar al VP y contar cuántos CEOs quedan; el resultado es el mismo porque en una permutación *el orden de*

las etapas sí importa: escoger primero CEO y después VP no es lo mismo que contar combinaciones, donde el par $\{A, B\}$ no distingue quién va en cada puesto.

El número de permutaciones de tomar r elementos tomados de un conjunto de n distintos es:

$$P(n, r) = n \cdot (n - 1) \cdots (n - r + 1) = \frac{n!}{(n - r)!}$$

Esto es así porque pensemos que si tenemos n opciones para la primera elección, una vez que lo escogimos, nos queda $n - 1$ opciones para la segunda elección, y así hasta que hayamos hecho las r elecciones. Nótese que cuando $n = r$ estamos calculando de cuántas formas se puede reordenar un conjunto completo y usando la fórmula se vuelve simplemente $n!$, esto nos dice que en el ejemplo de arriba las formas de escoger al CEO y VP son $P(5, 2) = 20$.

Permutaciones con repetición

Hay casos donde los n elementos de donde elegimos no son todos distintos, veamos por ejemplo la cantidad de formas distintas en que se puede reordenar la palabra MISSISSIPPI,

Si los n elementos no son todos distintos, por ejemplo, hay n_1 copias del elemento 1, n_2 del elemento 2, etc., con $n_1 + n_2 + \cdots + n_k = n$, el número de permutaciones distintas es:

$$\frac{n!}{n_1! n_2! \cdots n_k!}$$

El denominador divide las repeticiones que no generan acomodos distintos, de forma que en el ejemplo de arriba notemos hay 11 letras, 1 M, 4 I, 4 S, 2 P, de donde las maneras de reordenarle son

$$\frac{11!}{1! 4! 4! 2!} = 34\,650$$

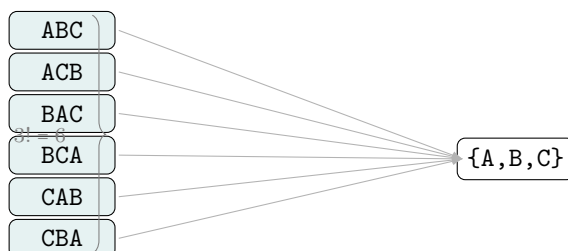
7.3. Combinaciones

Si ahora quisieramos de los 5 empleados agarrar 2 para que formen parte de una brigada no podemos aplicar directamente lo que aprendimos en permutaciones porque aquí el orden no importa, si primero elegimos al número 4 y luego al 2, es lo mismo que si elegieramos al 2 y después al 4, no hay distinción pues ambos formarían parte de la misma brigada. Lo que si aprendimos es que las formas de permutar un conjunto de r elementos es $r!$, entonces notemos que podemos calcular las permutaciones y luego dividir entre $r!$ que son los ordenamientos de cada selección que producen el mismo subconjunto, es decir si queremos hacer una selección de r elementos de un conjunto de n donde el orden no importa. El número de combinaciones es:

$$\binom{n}{r} = \frac{P(n, r)}{r!} = \frac{n!}{r! (n - r)!}$$

Permutaciones

Combinaciones



A las combinaciones también se les llama coeficientes binomiales porque son los coeficientes de la expresión $(a + b)^n$ de la siguiente forma:

$$(a + b)^n = \binom{n}{0}a^n + \binom{n}{1}a^{n-1}b + \binom{n}{2}a^{n-2}b^2 + \dots + \binom{n}{n}b^n$$

Algunas propiedades útiles para recordar son (se recomienda al lector intentar probar algunas de ellas):

- Simetría: $\binom{n}{r} = \binom{n}{n-r}$
- Factorización: $\binom{n}{r} = \frac{n}{k} \binom{n-1}{n-1}$
- Identidad de pascal: $\binom{n-1}{r-1} + \binom{n-1}{r} = \binom{n}{r}$
- Suma sobre k : $\sum_{k=0}^n \binom{n}{k} = 2^n$
- Suma sobre m : $\sum_{m=0}^n \binom{m}{k} = \binom{n+1}{k+1}$
- Suma alternada: $\sum_{r=0}^n (-1)^r \binom{n}{r} = 0$
- Suma de cuadrados: $\binom{n}{0}^2 + \binom{n}{1}^2 + \dots + \binom{n}{n}^2 = \binom{2n}{n}$

Si solo haremos calculos rápidos sin modulo y sin numeros grandes podemos implementar directamente

```

1 ll nCr(ll n, ll k){
2     if(k>n-k)k=n-k;
3     ll comb=1;
4     for(ll i=0;i<k;i++)comb=comb*(n-i)/(i+1);
5     return comb;
6 }
```

La implementación anterior es corta, pero conviene ser cuidadosos: $\binom{n}{k}$ crece muy rápido (por ejemplo $\binom{100}{50}$ ya no cabe en un entero de 64 bits) y es fácil obtener **overflow**. En programación competitiva, además, la respuesta casi siempre se pide **módulo** un primo M (típicamente $10^9 + 7$ o 998244353) justamente porque esos valores son demasiado grandes para imprimirlos tal cual. La estrategia habitual es **precalcular** factoriales e **inversos factoriales** módulo M una sola vez, y después responder cada consulta en $O(1)$ con

$$\binom{n}{k} \equiv \text{fac}[n] \cdot \text{invFac}[k] \cdot \text{invFac}[n-k] \pmod{M}.$$

En el precálculo, $\text{fac}[i]$ guarda $i!$ mód M . El arreglo $\text{inv}[i]$ guarda el **inverso modular** de i , es decir el entero tal que $i \cdot \text{inv}[i] \equiv 1 \pmod{M}$ (requiere que M sea primo y $\gcd(i, M) = 1$)¹. Con esos inversos se construyen los **inversos factoriales**: $\text{invFac}[0] = 1$ y, para $i \geq 1$,

$$\text{invFac}[i] = \text{invFac}[i-1] \cdot \text{inv}[i] \pmod{M},$$

de modo que $\text{invFac}[i] = (i!)^{-1} \pmod{M}$. *Ejemplo* con $M = 7$: como $3 \cdot 5 = 15 \equiv 1 \pmod{7}$, tenemos $\text{inv}[3] = 5$; entonces $\text{invFac}[3] = \text{inv}[1] \cdot \text{inv}[2] \cdot \text{inv}[3] = 1 \cdot 4 \cdot 5 = 20 \equiv 6 \pmod{7}$, y efectivamente $3! = 6 \equiv -1 \pmod{7}$, cuyo inverso es 6. Así, $\binom{5}{2} = 10 \equiv 3 \pmod{7}$ coincide con $\text{fac}[5] \cdot \text{invFac}[2] \cdot \text{invFac}[3] = 1 \cdot 4 \cdot 6 \equiv 3 \pmod{7}$.

¹La existencia del inverso y la recurrencia para calcular todos los $\text{inv}[i]$ en $O(M)$ se ven en la sección de inversos con módulo.

```

1 long long fac[MX], invFac[MX], inv[MX];
2 void precalcComb () {
3     fac[0] = inv[0] = 1;
4     for (int i = 1; i <= MX - 1; i++) {
5         fac[i] = fac[i - 1] * i % mod;
6     }
7
8     inv[0] = inv[1] = 1;
9     for (int i = 2; i <= MX - 1; i++) {
10        inv[i] = inv[mod % i] * (mod - mod / i) % mod;
11    }
12
13    inv_fac[0] = 1;
14    for (int i = 1; i <= MX - 1; i++) {
15        invFac[i] = invFac[i - 1] * inv[i] % mod;
16    }
17 }

```

Una vez hecho el precálculo, cada $\binom{n}{k}$ módulo M se obtiene con una sola línea. Como alternativa más lenta (sin arreglo invFac) se puede dividir módulo M multiplicando por el inverso de cada factorial con exponenciación binaria; en la práctica se usa casi siempre la versión precalculada.

```

1 long long nCr(long long n, long long r, long long mod){
2     if (r == 0) return 1;
3     long long num = fac[n];
4     long long den = (inverseMod(fac[r],mod) * inverseMod(fac[n-r],mod)) % mod;
5     return (num * den) % mod;
6 }
7
8 long long comb (int n, int k) {
9     if (n < 0 || k < 0 || n < k) return 0;
10    return fac[n] * inv_fac[k] % mod * inv_fac[n - k] % mod;
11 }

```

7.4. Principio de Inclusión-Exclusión

En ocasiones cuando tenemos conjuntos queremos saber cuántos elementos tiene su unión, podríamos pensar que si los conjuntos son A y B basta con hacer la suma $|A| + |B|$, sin embargo, es probable que los conjuntos no sean disjuntos, es decir, que haya elementos que pertenezcan tanto a A como B , en ese caso, les estaríamos contando doble. *Ejemplo.* Si $A = \{1, 2, 3, 4\}$ y $B = \{3, 4, 5, 6\}$, entonces $|A| + |B| = 8$, pero $A \cup B = \{1, 2, 3, 4, 5, 6\}$ solo tiene 6 elementos: el 3 y el 4 aparecieron una vez al contar A y otra al contar B . Es por esto que en realidad $|A \cup B| = |A| + |B| - |A \cap B|$, es decir, restamos una vez los que se están contando doble (aquí $|A \cap B| = |\{3, 4\}| = 2$ y $6 = 8 - 2$).

Similarmente si tuviésemos tres conjuntos A, B, C , notemos que al sumar $|A| + |B| + |C|$ estamos contando los elementos que están en dos de los conjuntos doble como en el caso de dos conjuntos, pero no solo eso, pues los elementos que están en los tres conjuntos, están siendo contados tres veces, y al restar todas las intersecciones de dos conjuntos se están restando tres veces, por lo que habrá que sumarlos, es decir

$$|A \cup B \cup C| = |A| + |B| + |C| - |A \cap B| - |A \cap C| - |B \cap C| + |A \cap B \cap C|$$

usando esta misma lógica para n conjuntos nos da el llamado **principio de inclusión-exclusión**, este nos dice que si tenemos $A_1, \dots, A_n$ conjuntos entonces tenemos que

$$|\bigcup_{i=1}^n A_i| = \sum_{i=1}^n |A_i| - \sum_{i < j} |A_i \cap A_j| + \dots + (-1)^{n+1} |A_1 \cap \dots \cap A_n|$$

Esta fórmula general puede parecer abstracta; la aplicamos en el ejercicio siguiente.

Ejercicio. Dados N , L y R , calcular cuántos enteros x del intervalo $[L, R]$ son coprimos con N (Co-prime, LCPC 2011). *Ejemplo.* Con $N = 6$, $L = 1$ y $R = 10$ se piden los $x \in \{1, 2, \dots, 10\}$ con $\gcd(x, 6) = 1$; como $6 = 2 \cdot 3$, basta que x no sea múltiplo de 2 ni de 3, y los que cumplen son 1, 5 y 7, así que la respuesta es 3.

Solución. Nótese que si hacemos $f(X)$ el número de x coprimos con N en el intervalo $[1, X]$, la respuesta será $f(R) - f(L - 1)$. Si bien podríamos intentar calcular la cantidad de números coprimos con N en el intervalo, es más fácil calcular lo opuesto, es decir, cuántos comparten un factor primo con N .

Con la criba podemos encontrar todos los factores primos $p_1, p_2, \dots, p_r$ de N . Ahora, sea D_i los números divisibles por p_i en el intervalo $[1, X]$, entonces $|D_i| = \lfloor \frac{X}{p_i} \rfloor$ por inclusión exclusión tenemos que:

$$|D_1 \cup D_2 \cup \dots \cup D_r| = \sum_i \lfloor \frac{X}{p_i} \rfloor - \sum_{i < j} \lfloor \frac{X}{p_i p_j} \rfloor + \sum_{i < j < k} \lfloor \frac{X}{p_i p_j p_k} \rfloor - \dots$$

y así tenemos que:

$$f(X) = X - |D_1 \cup \dots \cup D_r|$$

de forma que ya solo tenemos que implementarlo, para esto, iteraremos sobre los 2^r subconjuntos de factores primos de N . Para este tipo de problemas es importante analizar la complejidad, nótese que $2 \times 3 \times 5 \times 7 \times 11 \times 13 \times 17 \times 19 > 10^6$, de forma que si $N \leq 10^9$, este tendrá a lo más 9 factores primos, por lo que iterar sobre los $2^9 = 512$ subconjuntos sí es algo que se puede hacer rápidamente.

```

1 ll f(ll X, vector<ll>& primes) {
2     int k = primes.size();
3     ll res = 0;
4     for (int mask = 1; mask < (1 << k); mask++) {
5         ll prod = 1;
6         int bits = __builtin_popcount(mask);
7         for (int i = 0; i < k; i++)
8             if (mask >> i & 1) prod *= primes[i];
9         if (bits % 2 == 1) res += X / prod;
10        else res -= X / prod;
11    }
12    return X - res;
13 }
```

Complete el programa principal usando `f` para responder en el intervalo $[L, R]$.

7.5. Barras y estrellas

Supongamos tenemos r niños y n juguetes, ¿de cuántas formas podemos distribuir los juguetes entre los niños?, en este problema no hay restricciones, un niño puede tener 0 o todos los juguetes. Para resolver este problema usaremos "palitos" que separen los juguetes en secciones, por ejemplo, supongamos que tenemos 3 niños y 5 juguetes, entonces un acomodo donde el primer niño reciba dos juguetes, el

segundo dos y tercero uno se puede representar de la siguiente forma



De la misma forma si quisieramos r regiones, ocuparíamos $r - 1$ palitos, y notemos que podemos permutar los palitos por $(n + r - 1)!$ y después quitar las repeticiones dividiendo por $n!(r - 1)!$ así el número queda

$$\frac{(n + r - 1)!}{n!(r - 1)!}$$

esto también nos deja un resultado interesante y es que la ecuación $x_1 + x_2 + \dots + x_r = n$ donde $0 \leq x_i$ tiene una cantidad de soluciones $\frac{(n+r-1)!}{n!(r-1)!}$.

7.6. Lema de Burnside

Hay una clase de problemas de conteo donde la respuesta no es simplemente contar objetos, sino contar objetos **distintos bajo simetría**. Tomemos el ejemplo clásico: ¿cuántos collares distintos se pueden hacer con n cuentas de k colores, si dos collares son el mismo cuando uno se obtiene del otro por rotación? Contar directamente sobrecontea el mismo collar n veces (una por cada rotación). *Ejemplo.* Con $n = 3$ cuentas y $k = 2$ colores (digamos rojo R y verde G) hay $2^3 = 8$ coloraciones si numeramos las posiciones, pero varias son el mismo collar al rotar: $R-R-G$, $G-R-R$ y $R-G-R$ describen un solo collar; lo mismo $R-G-G$, $G-G-R$ y $G-R-G$. En total solo hay 4 collares distintos (RRR , GGG y esos dos tipos mixtos), no 8. El lema de Burnside resuelve exactamente esta clase de problemas.

Para formalizar “dos objetos son el mismo si uno se obtiene del otro por alguna simetría”, necesitamos el concepto de **acción de grupo**.

Tenemos dos ingredientes:

- Un conjunto X de todos los objetos que queremos contar (por ejemplo, todas las coloraciones posibles de n cuentas con k colores, hay k^n en total).
- Un conjunto G de **operaciones** (simetrías) que podemos aplicar. En el ejemplo de los collares, G son las n rotaciones posibles.

La acción de G sobre X es una función que a cada par (g, x) le asigna un nuevo objeto $g \cdot x \in X$ — el resultado de aplicar la operación g al objeto x . Para que sea una acción de grupo debe cumplir:

- La operación identidad no cambia nada: $e \cdot x = x$.
- Aplicar dos operaciones seguidas es lo mismo que aplicar su composición: $g_1 \cdot (g_2 \cdot x) = (g_1 \cdot g_2) \cdot x$.

Estas reglas básicamente dicen que la acción es compatible con la estructura del grupo y son fáciles de verificar en la práctica.

7.6.1. Órbitas y puntos fijos

La **órbita** de un objeto $x \in X$ es el conjunto de todos los objetos a los que puede llegar x al aplicarle alguna operación de G :

$$G \cdot x := \{g \cdot x \mid g \in G\}$$

Esta es la noción clave, dos objetos son *equivalentes* si están en la misma órbita. Nuestro objetivo es contar el número de órbitas $|X/G|$.

Para cada operación $g \in G$, sus **puntos fijos** son los objetos que no cambian al aplicarla:

$$X^g := \{x \in X \mid g \cdot x = x\}$$

Lema de Burnside:

El número de órbitas es igual al promedio del número de puntos fijos sobre todas las operaciones:

$$|X/G| = \frac{1}{|G|} \sum_{g \in G} |X^g|$$

La demostración completa puede consultarse en línea.²

Usemos este lema en el caso del problema de collares de n cuentas con k colores donde dos collares son iguales si uno se obtiene del otro por rotación. En este caso, X son todas las coloraciones: $|X| = k^n$.

Nótese que si tenemos un collar con n cuentas numeradas $1, 2, \dots, n$ en círculo con k colores diferentes. La rotación por r posiciones envía la cuenta i a la posición $i + r \pmod{n}$ por lo que hay n rotaciones distintas $r = 0, 1, \dots, n - 1$ donde $r = 0$ es la identidad, es decir, no rotar. Dos formas de colorear las cuentas son las misma si alguna se obtiene de la otra bajo alguna rotación. Estas rotaciones serán precisamente nuestro grupo G que actuara sobre X .

Ahora, notemos que nos interesa saber el número de orbitas porque nos interesa la cantidad de collares diferentes y cada orbita es un collar diferente. Sabiendo esto queremos entonces aplicar el lema de burnside y para ello nos interesa saber cuántos objetos fija la rotación por r posiciones. Notemos entonces que la rotación r fija una coloración si y solo si esta es periodica con periodo $\gcd(r, n)$ ya que la cuenta i debe tener el mismo color que la cuneta $i + r$ y esto agrupa las cuentas en exactamente $\gcd(r, n)$ clases donde todas las cuentas deben tener el mismo color. Y ya que cada una puede tener k colores diferentes, tenemos que hay $k^{\gcd(r, n)}$ coloraciones fijas.

Entonces por el lema de Burnside tenemos que:

$$|X/G| = \frac{1}{n} \sum_{r=0}^{n-1} k^{\gcd(r, n)}$$

La implementación entonces queda:

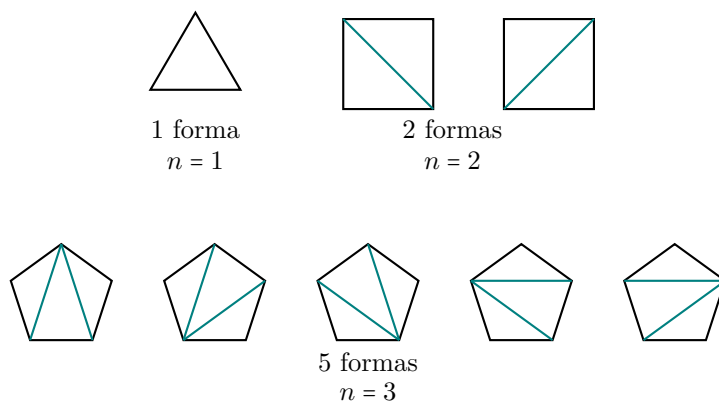
```
1 long long collares(int n, int k, long long mod) {
2     long long res = 0;
3     for (int r = 0; r < n; r++)
4         res = (res + power(k, __gcd(r, n), mod)) % mod;
5     return res % mod * power(n, mod-2, mod) % mod;
6 }
```

²https://artofproblemsolving.com/wiki/index.php/Burnside's_Lemma

7.7. Números de Catalan

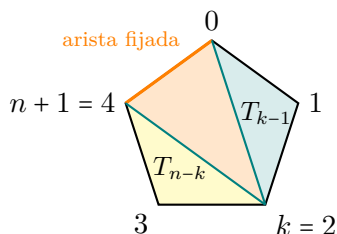
¿De cuántas formas se puede triangular un polígono convexo de $n + 2$ lados? Es decir, trazar $n - 1$ diagonales que dividan el polígono en exactamente n triángulos y que no se crucen entre sí.

Para los primeros casos podemos revisar la cantidad a mano:



Aunque pudimos hacer estos casos a mano, entre más lados tenga el polígono el número de triangulaciones se puede hacer muy grande, debido a esto es conveniente para nosotros buscar alguna otra forma de calcularlos.

Sea T_n el número de triangulaciones de un polígono de $n + 2$ lados. Fijamos la arista que va del vértice 0 al vértice $n + 1$. Esa arista pertenece a exactamente un triángulo de cualquier triangulación, llamemos k al tercer vértice de ese triángulo, con $k \in \{1, 2, \dots, n\}$. Por ejemplo para $n = 3$:



Al fijar el triángulo que contiene la arista fijada con tercer vértice $k = 2$, el polígono se divide en dos subpolígonos independientes: el subpolígono izquierdo (vértices $0, 1, \dots, k$) con T_{k-1} triangulaciones, y el derecho (vértices $k, k + 1, \dots, n + 1$) con T_{n-k} triangulaciones.

Notemos que los dos subpolígonos son independientes, es decir, la triangulación de uno no afecta al otro. Por lo tanto el número de triangulaciones con tercer vértice k es exactamente su producto $T_{k-1} \cdot T_{n-k}$. Sumando sobre todas las elecciones posibles de k , tenemos que:

$$T_n = \sum_{k=1}^n T_{k-1} \cdot T_{n-k}$$

En programación competitiva esta recurrencia casi siempre se resuelve con **programación dinámica** o **memoización**: guardamos $C_0, C_1, \dots, C_n$ en un arreglo (una "tabla" de una fila) y vamos llenándolo de izquierda a derecha, porque para calcular C_n solo hacen falta valores $C_0, \dots, C_{n-1}$ ya conocidos. La base es $C_0 = 1$ (el polígono degenerado con 0 triángulos internos).

Ejemplo (cómo se llena la tabla). Lo más cómodo es imaginar un arreglo de una fila que crece de izquierda a derecha; cada casilla nueva usa solo las anteriores:

$$\underbrace{[1]}_{C_0} \rightarrow \underbrace{[1, 1]}_{C_0, C_1} \rightarrow \underbrace{[1, 1, 2]}_{C_0, C_1, C_2} \rightarrow \underbrace{[1, 1, 2, 5]}_{C_0, C_1, C_2, C_3}.$$

Para obtener C_3 conviene ver la recurrencia como una **matriz de productos**. Con los valores ya calculados $C_0 = 1$, $C_1 = 1$ y $C_2 = 2$, formamos la matriz $M[i][j] = C_i \cdot C_j$:

	$j = 0$	$j = 1$	$j = 2$
$i = 0$	1	1	2
$i = 1$	1	1	2
$i = 2$	2	2	4

La fórmula $C_n = \sum_{k=1}^n C_{k-1}C_{n-k}$ equivale a sumar la **anti-diagonal** donde $i + j = n - 1$. Para $n = 3$ esa anti-diagonal son las celdas $(0, 2)$, $(1, 1)$ y $(2, 0)$:

	$j = 0$	$j = 1$	$j = 2$
$i = 0$	1	1	2
$i = 1$	1	1	2
$i = 2$	2	2	4

de modo que $C_3 = 2 + 1 + 2 = 5$. En el código, el bucle interno recorre esos pares (i, j) con $i = k - 1$ y $j = n - k$. La implementación recorre $i = 1, \dots, n$ y acumula la suma; cada C_i queda listo antes de calcular C_{i+1} :

```

1 vector<ll> catalan(int n) {
2     vector<ll> C(n + 1);
3     C[0] = 1;
4     for (int i = 1; i <= n; i++)
5         for (int k = 1; k <= i; k++)
6             C[i] += C[k - 1] * C[i - k];
7     return C;
8 }
```

Esta secuencia de números se llama **números de Catalan** y se denota $C_n = T_n$. Otra forma de expresar estos números es:

$$C_n = \frac{1}{n+1} \binom{2n}{n}$$

Nótese que estos números nos serán útiles cuando estemos resolviendo un problema en el que haya una estructura de tamaño n que se pueda dividir en dos partes independientes de tamaños $k - 1$ y $n - k$ en donde las contribuciones de cada parte se multipliquen, algunos ejemplos de problemas donde se usan son:

- Secuencias de n paréntesis balanceados.
- Árboles binarios completos con $n + 1$ hojas.
- Caminos reticulares de $(0, 0)$ a (n, n) que no cruzan la diagonal $y = x$.

7.8. Números de Stirling de segundo tipo

Los **números de Stirling de segundo tipo** $S(n, k)$ cuentan el número de formas de partir un conjunto de n elementos en exactamente k subconjuntos no vacíos.

Por ejemplo, $S(4, 2) = 7$, las 7 formas de partir $\{1, 2, 3, 4\}$ en 2 subconjuntos no vacíos son:

$$\begin{aligned} &\{1\}|\{2, 3, 4\}, \quad \{2\}|\{1, 3, 4\}, \quad \{3\}|\{1, 2, 4\}, \quad \{4\}|\{1, 2, 3\}, \\ &\{1, 2\}|\{3, 4\}, \quad \{1, 3\}|\{2, 4\}, \quad \{1, 4\}|\{2, 3\} \end{aligned}$$

Para calcularlos contamos con la recurrencia:

$$S(n, k) = k \cdot S(n-1, k) + S(n-1, k-1)$$

con $S(0, 0) = 1$ y $S(n, 0) = S(0, k) = 0$ para $n, k > 0$.

La intuición detrás de esta recurrencia consiste en observar que el elemento n tiene 2 opciones:

- n forma un subconjunto solo, en este caso partimos $\{1, \dots, n-1\}$ en $k-1$ subconjuntos de $S(n-1, k-1)$ formas, y $\{n\}$ es el k -ésimo subconjunto.
- n comparte subconjunto con otros elementos, en este caso partimos $\{1, \dots, n-1\}$ en k subconjuntos de $S(n-1, k)$ formas, y agregamos n a cualquiera de los k subconjuntos existentes.

7.9. Números de Bell

El n -ésimo **número de Bell** B_n cuenta el total de particiones de un conjunto de n elementos en subconjuntos no vacíos, sin importar cuántos subconjuntos haya:

$$B_n = \sum_{k=0}^n S(n, k)$$

Es decir, B_n es la suma de números de Stirling de segundo tipo. Sin embargo, calcular todos los números de Stirling de segundo tipo para calcular los números de Bell puede ser muy costoso computacionalmente, hay una forma más eficiente de calcularlos por medio del **triángulo de Bell**, el cual es un triángulo de números que se construye así:

- La primera entrada de cada fila es igual a la última entrada de la fila anterior: $T[i][0] = T[i-1][i-1]$.
- El resto de la fila sigue la regla: $T[i][j] = T[i][j-1] + T[i-1][j-1]$.
- El n -ésimo número de Bell es $T[n][0]$.

1				
1	2			
2	3	5		
5	7	10	15	
15	20	27	37	52

7.10. Ejercicios

- Triangle Coloring (Codeforces 1795D)
- Long Sandwich (Codechef)

7.11. Problemas

- Farm Game (Codeforces 1942E)
- Farm Game (Codeforces 1942E)
- Magic Bracelet (Spoj 2888)
- Buildings (2017-2018 ACM-ICPC GCPC)
- Prime Multiples (CSES 2185)
- Bracket Sequences I (CSES 2064)
- Blue Lock gone wild ★

Capítulo 8

Geometría computacional

La geometría computacional es el área que estudia cómo resolver problemas geométricos con algoritmos eficientes (por ejemplo, intersecciones, áreas, orden angular o envolventes convexas).

En geometría computacional conviene representar los objetos en código de forma explícita. Un **punto** (o **vector**) en el plano se modela como un par (x, y) ; las operaciones vectoriales usuales se aplican *coordenada a coordenada*. Según el contexto, un punto y un vector son intercambiables (origen implícito).

Ejemplo. Si $u = (2, 4)$ y $v = (4, 8)$, entonces las operaciones se hacen coordenada a coordenada:

$$u + v = (2 + 4, 4 + 8) = (6, 12),$$

$$v - u = (4 - 2, 8 - 4) = (2, 4),$$

$$2u = (2 \cdot 2, 2 \cdot 4) = (4, 8), \quad \frac{v}{2} = \left(\frac{4}{2}, \frac{8}{2}\right) = (2, 4).$$

En el código, usamos directamente `double` para las coordenadas reales, lo cual es útil cuando aparecen divisiones.

```
1 struct point {
2     double x,y;
3     point operator+(point p) {
4         return {x + p.x, y + p.y};
5     }
6     point operator-(point p) {
7         return {x - p.x, y - p.y};
8     }
9     point operator*(double a) {
10        return {x*a, y*a};
11    }
12    point operator/(double a) {
13        return {x/a, y/a};
14    }
15 };
```

Igualdad y comparadores. Al definir una estructura propia conviene fijar cuándo dos puntos son iguales y, si hace falta, un criterio de orden total.

```

1     return a.x==b.x && a.y==b.y;
2 }
3 bool operator!=(point a, point b) {
4     return !(a==b);
5 }

```

Observación. Definir un “menor que” canónico entre puntos del plano no es único: en la práctica el orden depende del problema. Con frecuencia conviene ordenar alrededor de un centro por sentido horario o antihorario; para ello sirven las operaciones entre vectores que siguen.

8.1. Bases

Producto punto

Producto punto: Dados vectores $a = (a_x, a_y)$ y $b = (b_x, b_y)$ (desde el origen), el **producto punto** es

$$a \cdot b = a_x b_x + a_y b_y.$$

Mide qué tan alineados están: si $a \cdot b > 0$ apuntan a direcciones parecidas; si $a \cdot b = 0$ son perpendiculares; si $a \cdot b < 0$ forman un ángulo obtuso.

Ejemplo. Sean $a = (2, 1)$ y $b = (3, -1)$. Entonces

$$a \cdot b = 2 \cdot 3 + 1 \cdot (-1) = 5 > 0,$$

así que el ángulo entre a y b es agudo. Si $c = (-1, 2)$, entonces $a \cdot c = 2 \cdot (-1) + 1 \cdot 2 = 0$, luego a y c son perpendiculares. Si $d = (-3, 0)$, se tiene $a \cdot d = -6 < 0$ (ángulo obtuso).

```

1     return a.x*b.x + a.y*b.y;
2 }

```

Observación. Con la implementación anterior, $a \cdot a$ es el cuadrado de la norma euclídea de a (sin raíz).

Ejemplo. Si $a = (3, 4)$, entonces $a \cdot a = 3^2 + 4^2 = 25 = \|a\|^2$. La norma euclídea es $\|a\| = \sqrt{25} = 5$, pero en código suele bastar comparar distancias con $a \cdot a$ y evitar la raíz.

Producto cruz en el plano

Producto cruz 2D: Para $a = (a_x, a_y)$ y $b = (b_x, b_y)$,

$$a \times b := a_x b_y - a_y b_x,$$

el área *orientada* del paralelogramo generado por a y b (equivalentemente, el doble del área orientada del triángulo con vértices O , a y b). En particular, $a \times b$ cambia de signo al intercambiar a y b , lo que codifica “ b a la izquierda o a la derecha de a ”.

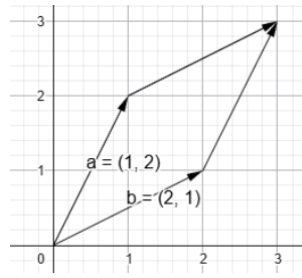
Ejemplo. Sean $a = (2, 0)$ y $b = (1, 2)$. Entonces

$$a \times b = 2 \cdot 2 - 0 \cdot 1 = 4 > 0,$$

así que, mirando desde el origen, b queda a la *izquierda* del rayo de a (giro antihorario de a hacia b). El paralelogramo de lados a y b tiene área $|a \times b| = 4$. Al intercambiar,

$$b \times a = 1 \cdot 0 - 2 \cdot 2 = -4 = -(a \times b).$$

Si $c = (3, 0)$ es colineal con a , entonces $a \times c = 2 \cdot 0 - 0 \cdot 3 = 0$ (tres puntos en una misma recta por el origen).



La implementación es corta y permite comprobar si tres puntos son colineales.

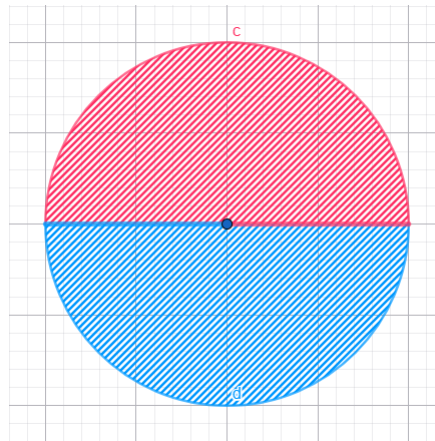
```
1  return a.x * b.y - a.y * b.x;
2  }
```

Orden angular alrededor de un punto

Con producto punto y cruz podemos ordenar puntos del plano en sentido antihorario (o horario) respecto a un origen.

Supongamos que queremos ordenar respecto al origen con dirección de referencia $v = (1, 0)$.

Si A es un conjunto finito de puntos, partimos el plano en dos regiones: N (puntos a la “izquierda” de v en el sentido del producto cruz, más los colineales con v en el mismo sentido) y S (el resto: a la derecha de v o colineales en sentido opuesto). En la figura, la zona roja corresponde a N y la azul a S .



Un punto $x \in A$ pertenece a N si y sólo si $v \times x > 0$, o bien $v \times x = 0$ y $v \cdot x > 0$ (usando las mismas rutinas de punto y cruz).

```
1  point v = {1,0};
2  return prodCruz(v,x) > 0 || ( prodCruz(v,x) == 0 && prodPunto(v,x) > 0 );
3  }
```

Para comparar $x, y \in A$, basta definir $x < y$ si $x \in N$ y $y \in S$, o si ambos caen en la misma región y $x \times y > 0$. La ordenación cuesta $O(n \log n)$ con un `sort` estándar.

```
1  return (N(x) == N(y) && prodCruz(x,y) > 0) || (N(x) && !N(y));
2  }
```

8.2. Intersecciones

Definición (vector perpendicular en el plano). Dado $v = (x, y)$, definimos

$$v^\perp := (-y, x)$$

(giro de 90° en sentido antihorario respecto al origen). Si $C = (x_C, y_C)$ y $D = (x_D, y_D)$, entonces $D - C = (x_D - x_C, y_D - y_C)$ y

$$(D - C)^\perp = (y_C - y_D, x_D - x_C).$$

Un problema recurrente es saber si dos segmentos AB y CD se intersectan. Hay muchos enfoques; aquí se evitan casos límite pensando primero en dos *rectas*: la primera $A + t(B - A)$ con $t \in \mathbb{R}$, y la segunda dada en forma implícita por la normal $(D - C)^\perp$:

$$(x - C) \cdot (D - C)^\perp = 0.$$

Si $(B - A) \times (D - C) \neq 0$, las rectas no son paralelas y se cortan en un único punto. Sustituyendo $x = A + t(B - A)$ y despejando t ,

$$t = \frac{(C - A) \cdot (D - C)^\perp}{(B - A) \cdot (D - C)^\perp}.$$

Ejemplo. Sean $A = (0, 0)$, $B = (1, 0)$, $C = (0, 1)$ y $D = (1, 2)$. Entonces $B - A = (1, 0)$, $D - C = (1, 1)$ y $(D - C)^\perp = (-1, 1)$; como $(B - A) \cdot (D - C)^\perp = -1 \neq 0$, las rectas no son paralelas. Además $C - A = (0, 1)$, luego

$$t = \frac{(0, 1) \cdot (-1, 1)}{(1, 0) \cdot (-1, 1)} = \frac{1}{-1} = -1,$$

y el punto común es $A + t(B - A) = (-1, 0)$ (compruébese que satisface $(x - C) \cdot (D - C)^\perp = 0$).

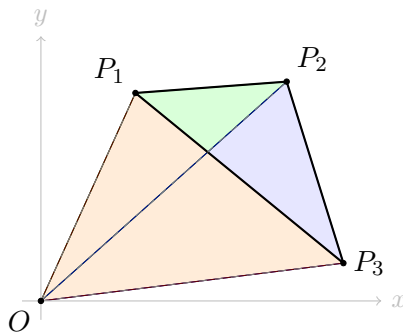
Sustituyendo t en $A + t(B - A)$ se obtiene el punto de intersección de las rectas; falta comprobar que cae sobre *ambos* segmentos (acotando t y el parámetro del otro lado si se desea).

Ejercicio. Implementar una función booleana que, dados cuatro puntos A, B, C, D , indique si los segmentos AB y CD se intersectan.

8.3. Polígonos

Cuando somos pequeños, el área del triángulo se enseña como “base por altura entre dos”; si solo tenemos coordenadas de los vértices, conviene una fórmula directa que generalice a polígonos.

Supongamos $P_1 = (x_1, y_1)$, $P_2 = (x_2, y_2)$ y $P_3 = (x_3, y_3)$. Tomando el origen $O = (0, 0)$, el área buscada se puede ver como suma y resta de áreas orientadas de los triángulos P_3OP_2 , P_2OP_1 y P_3OP_1 .



Estas áreas de los triángulos ya las sabemos calcular, pues recordemos que el producto cruz entre P_3 y P_2 nos da el área con signo del paralelogramo que forman y al dividirlo entre 2, el área del triángulo buscado es de donde deducimos que la fórmula del área de un triángulo es

$$A(P_1, P_2, P_3) = \frac{|P_1 \times P_2 + P_2 \times P_3 + P_3 \times P_1|}{2}$$

Nótese que no es necesario restar porque el producto cruz ya nos da el área con signo, entonces el signo de menos ya está incluido en la suma.

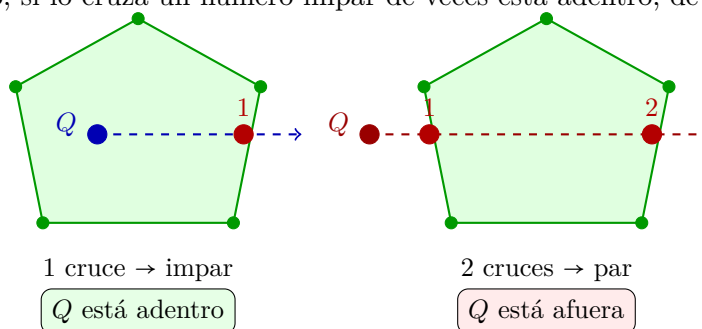
Resulta que esta misma lógica funciona para polígonos de n vértices donde se suman n triángulos haciendo que el exterior se cancele, la fórmula generalizada entonces quedaría

$$A(P_1, \dots, P_n) = \frac{|\sum_{i=1}^n P_i \times P_{i+1}|}{2}$$

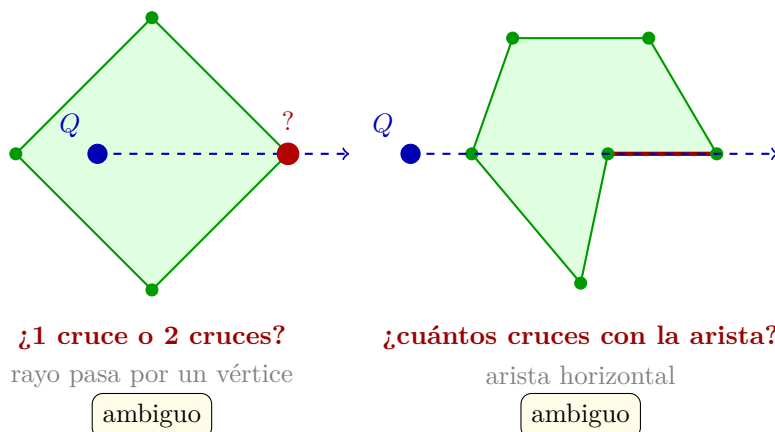
con $P_{n+1} = P_1$. Esta fórmula funciona también para polígonos no convexos.

Punto dentro o fuera de un polígono

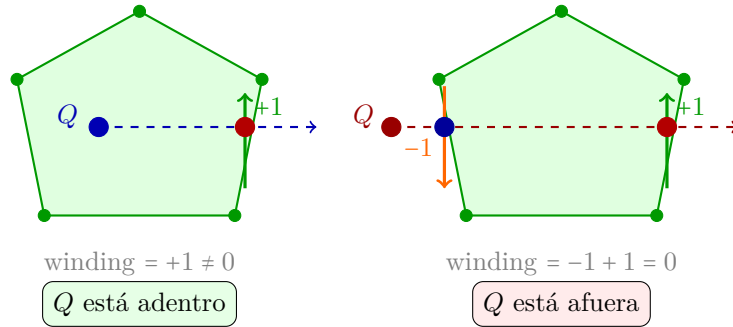
Otro problema que nos podemos encontrar al trabajar con polígonos es dado un punto Q determinar si está afuera o adentro de un polígono dado. El método más usado para resolver este problema se llama "ray casting", la idea es que se lanza un rayo horizontal desde Q hacia la derecha y contamos cuántas veces cruza el polígono, si lo cruza un número impar de veces está adentro, de lo contrario está afuera.



este es un método sencillo para problemas donde sabemos con certeza no pasarán casos extremos, ya que al lanzar el rayo podría suceder lo siguiente



Para resolver este problema, usamos un algoritmo que se llama "winding number"; consiste en la misma idea de lanzar un rayo y contar intersecciones, pero aquí cada cruce aporta $+1$ o -1 según la orientación con la que la arista atraviesa el rayo. En la figura siguiente, en el panel de la *izquierda* la arista que cruza el rayo va de abajo hacia arriba, así que se suma 1; en el panel de la *derecha*, la arista marcada con -1 va de arriba hacia abajo, así que se resta 1, mientras que el otro cruce (marcado con $+1$) vuelve a sumar 1. Si el resultado es diferente de 0, el punto está adentro.



- +1 arista cruza el rayo de abajo hacia arriba
- 1 arista cruza el rayo de arriba hacia abajo

Nótese que la dirección de cada arista que cruza el rayo se determina en dos pasos: intuitivamente, “subir” o “bajar” al cruzar el nivel $y = y_Q$ corresponde al sentido en que crece o decrece la ordenada y al recorrer la arista de P_i a P_{i+1} (para segmentos no verticales eso coincide con el signo de la pendiente $\Delta y / \Delta x$); formalmente, primero verificamos que los extremos de la arista estén en lados opuestos del nivel $y = y_Q$ usando las condiciones sobre las coordenadas y , y luego usamos $\text{prodCruz}(P_{i+1} - P_i, Q - P_i)$ para saber si Q queda a la izquierda o derecha de la arista, lo que determina si el cruce suma $+1$ o -1 (y evita depender de divisiones cuando la arista es casi vertical).

Ahora, el lector podrá preguntarse qué sucede con el caso donde el rayo intersecta exactamente en un vértice P_i . En este algoritmo las condiciones para que una arista cuente son asimétricas: para la arista $P_{i-1}P_i$ que *llega* a P_i se requiere

$$y_{P_{i-1}} \leq y_Q < y_{P_i}$$

como $y_{P_i} = y_Q$, la condición estricta $<$ no se satisface y la arista **no cuenta**. Para la arista P_iP_{i+1} que *sale* de P_i se requiere

$$y_{P_i} \leq y_Q < y_{P_{i+1}}$$

como $y_{P_i} = y_Q$, la condición $\leq$ sí se satisface, y la arista **cuenta** únicamente si $y_{P_{i+1}} > y_Q$, es decir, si la arista realmente cruza el rayo hacia arriba.

El mismo razonamiento elimina las aristas horizontales: si $y_{P_i} = y_{P_{i+1}} = y_Q$, ninguna de las dos condiciones se satisface y la arista simplemente no se cuenta, ya explicado el algoritmo, la implementación quedaría:

```

1  int wn = 0;
2  int n = poly.size();
3  for (int i = 0; i < n; i++) {
4      point a = poly[i], b = poly[(i+1) % n];
5      if (a.y <= q.y) {
6          // arista cruza hacia arriba: b esta estrictamente sobre el rayo
7          if (b.y > q.y && prodCruz(b-a, q-a) > 0)

```

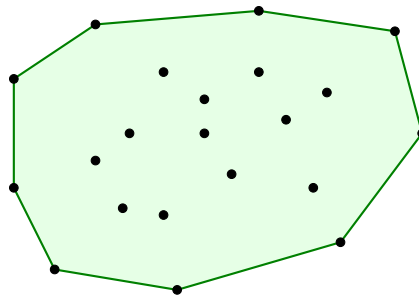
```

8         wn++;
9     } else {
10         // arista cruza hacia abajo: b esta sobre o bajo el rayo
11         if (b.y <= q.y && prodCruz(b-a,q-a) < 0)
12             wn--;
13     }
14 }
15 return wn;
16 }
17
18 bool dentro(vector<point> poly, point q) {
19     return winding_number(poly, q) != 0;
20 }

```

8.4. Convex Hull

Dado un conjunto de puntos $S \subset \mathbb{R}^2$, su **cápsula convexa** (*convex hull*) intuitivamente es la figura que se formaría si colocamos una liga estirada alrededor de todos los puntos, más formalmente es el polígono convexo más pequeño que los contiene a todos. En la imagen siguiente S son todos los puntos negros y el hull sería es el polígono dibujado de verde.



Existen varios algoritmos para calcularlo. El más conocido es **Graham Scan**; sin embargo, este ordena los puntos por ángulo por lo que hay que tener cuidado con los casos delicados cuando varios puntos tienen el mismo ángulo polar. Por ello preferimos **Andrew's Monotone Chain**, ambos algoritmos tienen la misma complejidad $O(n \log n)$, misma idea para construir el convex hull, pero ordena por coordenada x en lugar de ángulo, esto lo hace más fácil de implementar sin errores en un concurso.

Nótese que para saber qué polígono es el hull, sólo tenemos que saber qué puntos forman parte de él y en qué orden. El algoritmo de Andrew construye el convex hull en dos pasadas sobre los puntos ordenados por x , primero se construye la parte inferior y luego la superior.

Para la parte superior se van a recorrer los puntos de izquierda a derecha, y tendremos un vector *hull* con la característica que los últimos tres puntos siempre van a tener producto cruz es positivo, es decir dados A, B, C tenemos que $(B - A) \times (C - A) > 0$, inicialmente se agregan los dos primeros puntos a *hull* y después cada que lleguemos a un nuevo punto P si este cumple la condición se agrega, de lo contrario, se descarta el tope de la pila y después se agrega P a *hull*. Para la parte inferior es exactamente el mismo procedimiento con la única diferencia que se recorre de derecha a izquierda. La implementación quedaría:

```

1     int n = pts.size();
2     if (n < 2) return pts;
3     sort(pts.begin(), pts.end());

```

```

4     vector<point> hull;
5     /*construimos hull inferior
6     izquierda a derecha*/
7     for (int i = 0; i < n; i++) {
8         while (hull.size() >= 2 &&
9                /*producto cruz entre
10                B-A y C-A*/
11                prodCruz(hull.back()-hull[hull.size()-2], pts[i]-hull[hull.size()-2])
12                <= 0)
13             hull.pop_back();
14             hull.push_back(pts[i]);
15     }
16     /*construimos hull superior
17     derecha a izquierda*/
18     int lower_size = hull.size() + 1;
19     for (int i = n-2; i >= 0; i--) {
20         while (hull.size() >= lower_size &&
21                /*producto cruz entre
22                B-A y C-A*/
23                prodCruz(hull.back()-hull[hull.size()-2], pts[i]-hull[hull.size()-2])
24                <= 0)
25             hull.pop_back();
26             hull.push_back(pts[i]);
27     }
28     //al cerrarlo este punto se repite
29     hull.pop_back();
30     return hull;
31 }

```

La desigualdad ≤ 0 descarta también los puntos colineales, de modo que el hull resultante solo contiene los vértices extremos. Si el problema requiere incluir los puntos colineales sobre las aristas del hull, podemos hacer la desigualdad estricta.

Ejercicio Dado un conjunto de n puntos, encontrar el par de puntos más lejanos (diámetro).

8.5. Algoritmos de barrido

El barrido de línea no es por sí solo un algoritmo, si no una idea general detrás de varios algoritmos. La idea es mover una línea vertical de izquierda a derecha sobre el plano y procesar los objetos geométricos conforme se van encontrando en lugar de considerar todos a la vez, a esta acción de procesamiento se le llama evento. Es decir, que en cada momento solo nos importa lo que está tocando la línea, para esto es necesario mencionar que la línea no se mueve continuamente si no que salta de evento en evento.

Estos algoritmos usualmente tienen tres partes:

- **Cola de eventos** esta es una lista ordenada por x de los eventos.
- **Estructura de incidencia** es una estructura que nos dirá qué objetos intersectan la línea en este momento, la elección de la estructura a usar depende el problema, pueden usarse sets, multisets, segment trees, colas, entre otras.
- **Procesamiento de eventos** es la lógica con la que procesaremos cada que ocurra un evento, esta depende mucho de qué problema particular querramos resolver.

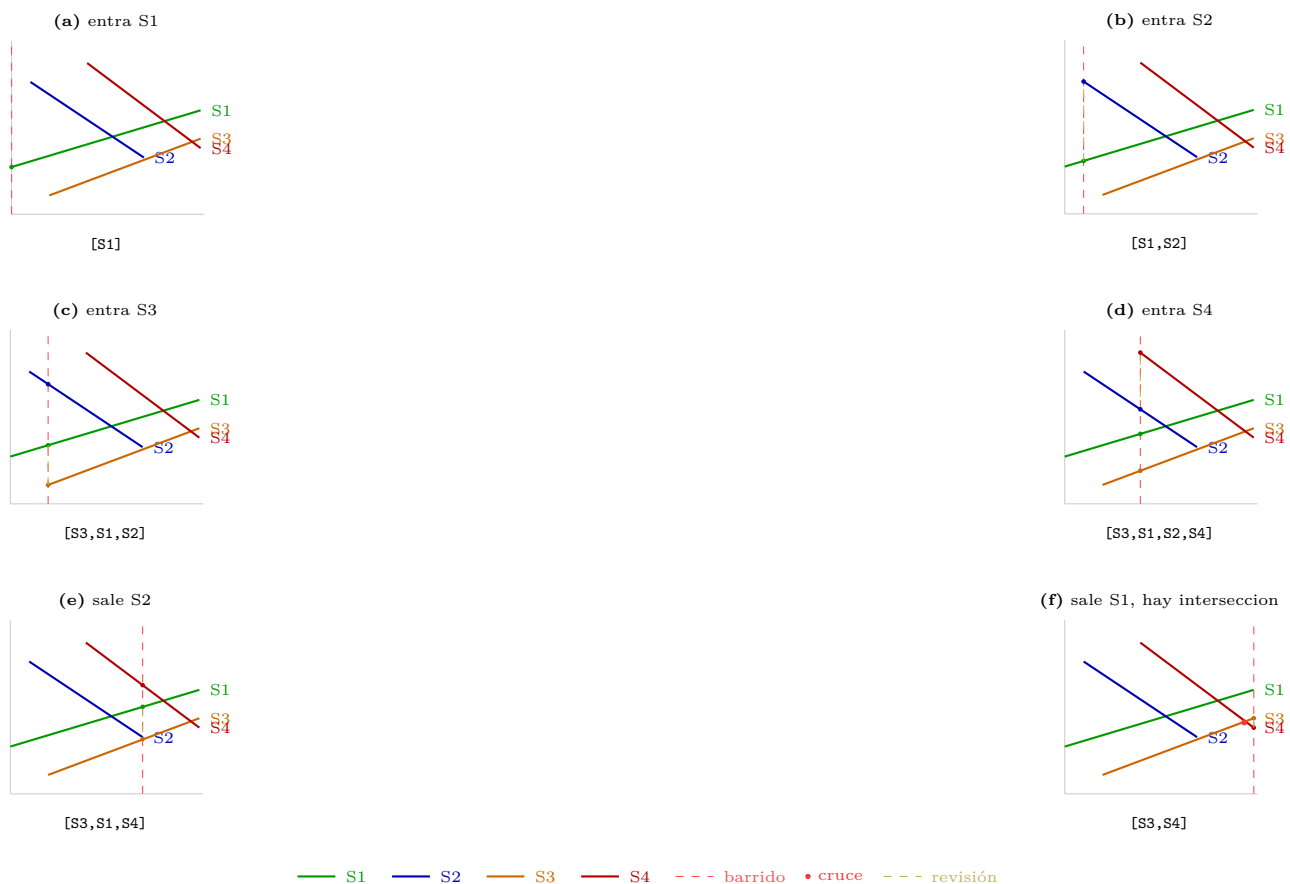
Propongamos el siguiente problema: Dados n segmentos en el plano, decir si al menos dos de ellos se intersectan.

Podríamos pensar ingenuamente que la única forma de resolver este problema es revisar todos los pares de segmentos y revisar intersección entre ellos es decir una complejidad de $O(n^2)$, sin embargo, usando barrido de línea podemos resolver este problema más rápido.

El algoritmo que veremos para resolver este problema se llama *Shamos – Hoey* y se basa en que si dos segmentos se llegan a intersectar existe un momento durante el barrido en el que son vecinos en la estructura ordenada por y . Esto hace que el problema se reduzca a que en lugar de revisar todos los pares, revisar los vecinos en la estructura cada que suceda un evento.

Los eventos que podemos tener en este problema son puntos que indican el inicio o final de un segmento, entonces tendremos dos tipos de eventos:

- **Inicio** Insertaremos el punto a la estructura que ya dijimos está ordenada por y y verificaremos si se intersecta con el vecino superior o inferior.
- **Final** Este evento indica el fin de un segmento, entonces lo quitaremos de la estructura, pues ya no nos interesa. Sin embargo, hay que tener cuidado es necesario revisar si los dos vecinos (que al quitar el punto se vuelven adyacentes) se intersectan.



Ejemplo (cómo leer la estructura). **Nota:** lo que sigue interpreta la misma figura que queda inmediatamente arriba (la secuencia de paneles (a)–(f) con los segmentos $S1$ – $S4$, la vertical roja del barrido

y la franja inferior con la leyenda **barrido / cruce / revisión**). En cada uno de esos paneles, entre corchetes se muestra el contenido del **set** en el instante en que la línea de barrido coincide con la vertical roja *de ese* panel. La lista enumera los segmentos que *cruzan* esa recta vertical, ordenados de *abajo hacia arriba* según la ordenada del punto de corte con $X = x$ (los puntos resaltados sobre la vertical, unidos por trazos amarillos). En **(a)** solo $S1$ es activo: $[S1]$. En **(b)**, en $x = 1$, el corte de $S1$ queda debajo del de $S2$, luego $[S1, S2]$. En **(c)** entra $S3$ por la izquierda y su intersección con la vertical es la más baja, después la de $S1$ y por último la de $S2$, de ahí $[S3, S1, S2]$. En **(d)** los cuatro segmentos ya cortan la vertical y se leen de abajo arriba como $[S3, S1, S2, S4]$. En **(e)** el evento de fin saca a $S2$; en $[S3, S1, S2, S4]$ los vecinos de $S2$ eran $S1$ por abajo y $S4$ por arriba, y al retirarlo $S1$ y $S4$ quedan *adyacentes* en $[S3, S1, S4]$, justo el par que el paso de borrado debe volver a examinar. En **(f)** $S1$ ya no es activo y solo quedan $S3$ y $S4$, adyacentes en $[S3, S4]$; al revisar ese par se encuentra la intersección.

Si en cualquier punto encontramos una intersección, entonces terminamos el algoritmo pues ya resolvimos el problema. Es importante mencionar, que si hay varios eventos en un mismo x , primero procesaremos los inicios y después los finales, así lograremos también comprobar intersecciones si los segmentos comparten vértices.

La estructura que nos convendrá para este problema es un `set<pair<point, point>>` de C++ ordenado por la coordenada y de cada segmento en la posición actual x de la línea. El comparador para ordenar evalúa para cada segmento

$$y(x) = y_1 + \frac{(y_2 - y_1)(x - x_1)}{x_2 - x_1}$$

al implementarlo hay que ser cuidadosos pues como estaremos trabajando con decimales al hacer comparaciones debemos agregar una tolerancia ε .

```

1 using seg = pair<point, point>;
2 //funcion que nos devuelve dado un segmento y un x, la y del segmento.
3 double getY(const seg& s, double x) {
4     if (abs(s.first.x - s.second.x) < 1e-9){
5         return s.first.y;
6     }
7     return s.first.y +
8         (s.second.y - s.first.y)
9         * (x - s.first.x)
10        / (s.second.x - s.first.x);
11 }
12
13 double sweepX;
14 //Comparador para ordenar los segmentos por y
15 struct cmp {
16     bool operator()(const seg& a, const seg& b) const {
17         double ya = getY(a, sweepX);
18         double yb = getY(b, sweepX);
19         if (abs(ya - yb) > 1e-9){
20             return ya < yb;
21         }
22         return a < b;
23     }
24 };
25
26 bool shamos_hoey(vector<seg> segs) {
27     int n = segs.size();
28     /*asegurarse que los segmentos
29     vayan de izquierda a derecha*/

```

```

30     for (auto& s : segs){
31         if (s.first.x > s.second.x){
32             swap(s.first, s.second);
33         }
34     }
35     /*
36     eventos guardara la x del punto,
37     1 o -1 que nos indicara si es de inicio
38     o fin respectivamente,
39     el indice en el vector original
40     */
41     vector<tuple<double,int,int>> eventos;
42     for (int i = 0; i < n; i++) {
43         eventos.push_back(
44             {segs[i].first.x, 1, i}
45         );
46         eventos.push_back(
47             {segs[i].second.x, -1, i}
48         );
49     }
50     sort(eventos.begin(), eventos.end(),
51     [](auto& a, auto& b){
52         if (abs(get<0>(a)
53             - get<0>(b)) > 1e-9){
54             return get<0>(a) < get<0>(b);
55         }
56         return get<1>(a) > get<1>(b);
57     });
58
59     set<seg,cmp> activos;
60     /*estructura que nos ayude
61     a borrar sin depender del
62     comparador que definimos*/
63     vector<set<seg,cmp>::iterator> pos(n, activos.end());
64     for (auto& [x, tipo, id] : eventos) {
65         sweepX = x;
66         //procesamos nodo de inicio
67         if (tipo == 1) {
68             auto it =
69                 activos.insert(segs[id]).first;
70             pos[id] = it;
71             auto sig = next(it);
72             auto ant = (it != activos.begin())
73                 ? prev(it)
74                 : activos.end();
75             /*buscar interseccion con
76             vecinos, se dejo como ejercicio
77             en intersecciones*/
78             if (sig != activos.end()
79                 && intersectan(it, sig)){
80                 return true;
81             }
82             if (ant != activos.end()
83                 && intersectan(it, ant)){
84                 return true;
85             }
86             //procesamos nodo de final
87         } else {
88             auto it = pos[id];
89             auto sig = next(it);
90             auto ant = (it != activos.begin())

```

```

91         ? prev(it)
92         : activos.end();
93     if (sig != activos.end()
94         && ant != activos.end()
95         && intersectan(*sig, *ant)){
96         return true;
97     }
98     activos.erase(it);
99 }
100 }
101 return false;
102 }

```

Nótese que hay $2n$ eventos y cada evento realiza a lo más dos chequeos de intersección que cada uno tiene complejidad de $O(1)$ y una adición o eliminación con complejidad $O(\log n)$. Además ordenar inicialmente los segmentos tiene complejidad $O(n \log n)$ entonces la complejidad del algoritmo es $O(n \log n)$ que en efecto es más rápido que $O(n^2)$.

En el problema que resolvimos bastaba con encontrar una intersección, podríamos preguntarnos qué sucede si queremos encontrar todas las intersecciones. El algoritmo que se usa para este caso se llama **Bentley-Ottmann** que encuentra todas las intersecciones en $O((n + k) \log n)$ donde k es el número de intersecciones. Podría parecer que la diferencia es sutil y sólo basta con seguir el algoritmo que estabamos usando antes sin detenerlo, pero recordemos que nuestro algoritmo dependía fuertemente en el orden de los segmentos para encontrar intersecciones, cuando sucede una intersección, el orden de dos segmentos se cambia esto provoca que cada que encontremos un cruce tendremos que cambiar el orden de la estructura y además revisar si los nuevos vecinos tienen cruces. Y no solo eso, si no que cada que agreguemos un segmento si encontramos una intersección tendremos que calcularla y agregarla a la cola de eventos.

Mayores complicaciones de Bentley-Ottmann

- Cuando dos segmentos cambian de orden, nuestro **set** ordenado por y ya no es correcto, pues el comparador de este cambia con el tiempo dependiendo de x , tendríamos que manualmente extraer e insertar los elementos necesarios.
- Si hay casos degenerados con tres segmentos o más que se intersectan en el mismo punto, hay que ser muy cuidadosos en que orden se procesan y cómo actualizamos la estructura usada.
- Si no sabemos con certeza una cota superior para la cantidad de intersecciones y se dé el peor caso (n^2), puede suceder que el algoritmo con complejidad $O(n^2 \log n)$ sea más lento que la fuerza bruta que era $O(n^2)$.

Usualmente no se necesita implementar Bentley Oattmann en concursos, ya que su implementación son alrededor de 200 líneas con casos delicados de manejar, sería complicado encontrar bugs bajo presión de tiempo, además que los problemas donde se requieren todas las intersecciones usualmente permiten fuerza bruta. Lo más usado es la idea en sí misma aplicado a problemas más sencillos como la unión de área de rectángulos como veremos en los ejemplos a continuación.

8.6. Ejercicios

8.7. Problemas

Capítulo 9

Teoría de gráficas

Una gráfica $G = (V, E)$ consiste en un conjunto de **vértices** V y un conjunto de **aristas** $E \subseteq \binom{V}{2}$. Asumimos familiaridad con las definiciones básicas: vecindad, grado, caminos, ciclos, conectividad. Esta sección se enfoca en resultados matemáticos que fundamentan el resto del capítulo y que raramente se ven en cursos de computación.

Recordemos que hay dos representaciones estándar de una gráfica, las cuáles tienen complejidades muy distintas por lo que sirven para diferentes problemas. En general, la lista de adyacencia es preferible cuando $E \ll V^2$; la matriz cuando se necesita consultar aristas individuales en $O(1)$ o cuando V es pequeño ($V \leq 500$) y se necesitan consultas en $O(1)$.

Operación	Lista de adyacencia	Matriz de adyacencia
Espacio	$O(V + E)$	$O(V^2)$
¿Existe arista (u, v) ?	$O(\deg u)$	$O(1)$
Vecinos de u	$O(\deg u)$	$O(V)$
Agregar arista	$O(1)$	$O(1)$

9.1. Lema de los apretones de mano

En todo grafo $G = (V, E)$:

$$\sum_{v \in V} \deg(v) = 2|E|$$

Demostración. Cada arista $\{u, v\}$ contribuye exactamente 1 al grado de u y 1 al grado de v , es decir, contribuye 2 a la suma total de grados. Sumando sobre todas las aristas: $\sum_v \deg(v) = 2|E|$.

En todo grafo, el número de vértices de grado impar es par.

Demostración. Sea I el conjunto de vértices de grado impar y P el de grado par. Entonces:

$$2|E| = \sum_{v \in I} \deg(v) + \sum_{v \in P} \deg(v)$$

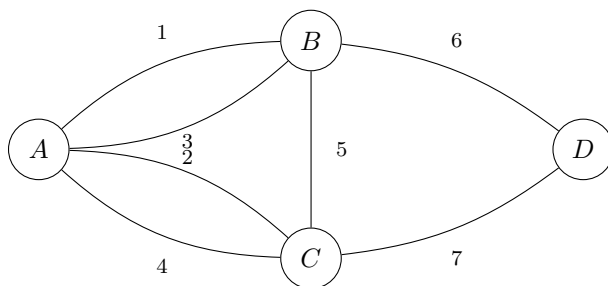
El segundo término es par (suma de números pares). Como $2|E|$ es par, el primer término también debe serlo.

Este corolario tiene consecuencias sorprendentes en el ICPC. Por ejemplo: en un torneo (grafo dirigido completo), la suma de grados de salida es $\binom{n}{2}$; argumentos de paridad sobre los grados pueden determinar si cierta configuración es posible sin necesidad de construirla.

9.2. Caminos y circuitos de Euler

Los puentes de Königsberg

En 1736, Leonhard Euler resolvió el siguiente problema planteado por los habitantes de la ciudad de Königsberg. La ciudad tenía 7 puentes que conectaban 4 zonas de tierra. ¿Es posible dar un paseo cruzando cada puente exactamente una vez y regresar al punto de partida?



Euler demostró que tal paseo es **imposible** y al hacerlo fundó la teoría de gráficas. Su argumento fue que cada vez que el paseo pasa por una zona (sin empezar ni terminar ahí), usa un puente para entrar y otro para salir. Por lo tanto esa zona debe tener un número par de puentes. Las zonas B y C tienen 3 puentes cada una, así que el paseo no puede existir.

Definición 9.2.1 (Camino y circuito euleriano). En una gráfica G :

- Un **camino euleriano** es un camino que recorre cada arista exactamente una vez.
- Un **circuito euleriano** es un camino euleriano que empieza y termina en el mismo vértice.
- G es **euleriano** si tiene un circuito euleriano.

Sea G conexa. Entonces:

- G tiene un **circuito euleriano** si y solo si todo vértice tiene grado par.
- G tiene un **camino euleriano** (no circuito) si y solo si exactamente dos vértices tienen grado impar, y el camino debe empezar en uno de ellos y terminar en el otro.

9.2.1. Algoritmo de Hierholzer

Verificar si existe un circuito euleriano es fácil. Encontrarlo es lo que requiere el algoritmo de Hierholzer.

Intuitivamente, el algoritmo parte de cualquier vértice y sigue aristas arbitrarias hasta regresar al punto de partida. Esto va a formar un circuito, pero puede no ser euleriano. Si algún vértice del circuito actual tiene aristas sin usar, se extiende el circuito desde ese vértice. Se repite esto hasta usar todas las aristas.

```

1 vector<int> hierholzer(int inicio) {
2     stack<int> st;
3     vector<int> circuito;
4     st.push(inicio);
5     while (!st.empty()) {
6         int v = st.top();
7         if (adj[v].empty()) {
8             circuito.push_back(v);
9             st.pop();
10        } else {
11            int u = adj[v].back();
12            adj[v].pop_back();
13            st.push(u);
14        }
15    }
16    reverse(circuito.begin(), circuito.end());
17    return circuito;
18 }

```

Muchos problemas se reducen a encontrar un camino o circuito euleriano de forma no obvia. El patrón general es construir un grafo donde lo que se quiere recorrer son las **aristas**, no los vértices. Imaginemos que nos dan un conjunto de palabras y nos preguntan, ¿es posible ordenarlas en una secuencia tal que la última letra de cada palabra sea la primera de la siguiente? Podemos modelar cada letra como un vértice, cada palabra como una arista dirigida de su primera letra a su última. La secuencia existe si y solo si el grafo tiene un camino euleriano.

9.3. Componentes fuertemente conexas

En un grafo **no dirigido**, la conectividad es simple, hay un camino entre dos nodos o no. Pero en un grafo **dirigido** la situación es más complicada, pues puede haber un camino de u a v sin que haya uno de v a u .

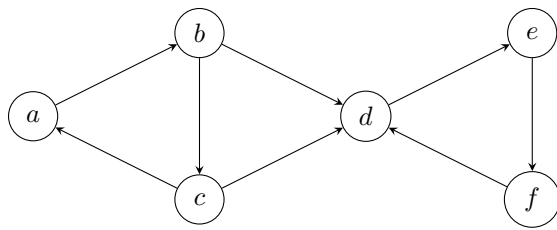
Definición 9.3.1. Una **componente fuertemente conexa** (SCC) de una gráfica dirigida es un subconjunto máximo de vértices C tal que para todo par $u, v \in C$ existe un camino de u a v y un camino de v a u .

9.3.1. Algoritmo de Kosaraju

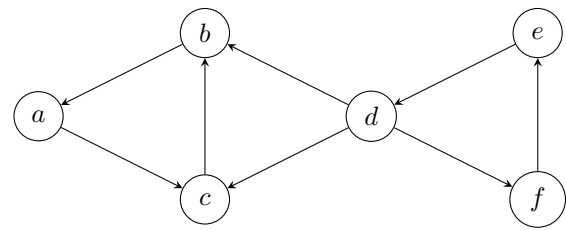
El algoritmo de Kosaraju encuentra todas las SCCs en $O(V + E)$ con dos DFS. El algoritmo se basa en una observación sobre el **grafo transpuesto** G^T (el mismo grafo pero con todas las aristas invertidas). Las SCCs de G y G^T son **exactamente las mismas**, invertir las aristas no cambia qué nodos se alcanzan mutuamente dentro de una SCC. Pero el orden en que las SCCs se “descubren” sí cambia.

El primer DFS lo hacemos en G y anotamos el orden en que terminan (orden de salida). El último nodo en terminar pertenece a la SCC que no tiene aristas entrantes desde otras SCCs.

El segundo DFS lo haremos en G^T procesando los nodos en orden inverso al de salida. Cada DFS que se pueda iniciar desde un nodo no visitado encontrará exactamente una SCC.



G



G^T

Este algoritmo funciona porque en el segundo DFS sobre G^T , empezamos por el nodo con mayor tiempo de salida. Desde ese nodo en G^T solo podemos alcanzar nodos de su propia SCC porque las aristas que salían de esa SCC en G ahora entran a ella en G^T , y las aristas que entraban ahora salen, por lo que los nodos alcanzables desde él en G^T forman su SCC.

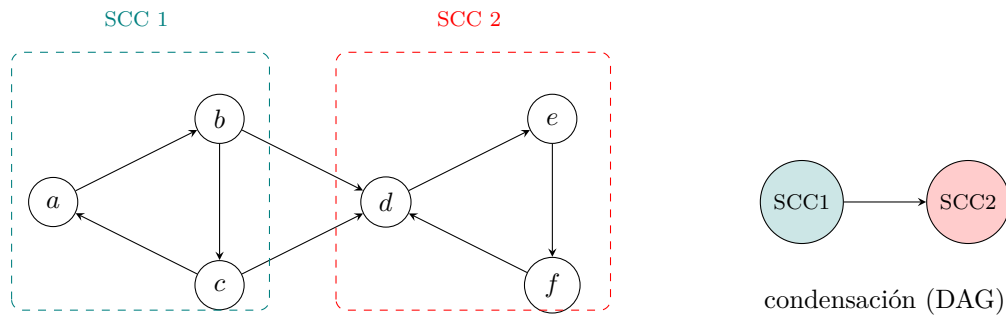
```

1 vector<int> adj[MAXN], radj[MAXN];
2 bool visited[MAXN];
3 vector<int> order; // orden de salida del primer DFS
4 ll comp[MAXN]; // SCC de cada nodo
5
6 void dfs1(int v) {
7     visited[v] = true;
8     for (int u : adj[v])
9         if (!visited[u]) dfs1(u);
10    order.push_back(v);
11 }
12
13 void dfs2(int v, int c) {
14     comp[v] = c;
15     for (int u : radj[v])
16         if (comp[u] == -1) dfs2(u, c);
17 }
18
19 int kosaraju(int n) {
20     // Primer DFS en G
21     fill(visited, visited+n, false);
22     for (int i = 0; i < n; i++)
23         if (!visited[i]) dfs1(i);
24
25     // Segundo DFS en G^T, en orden inverso de salida
26     fill(comp, comp+n, -1);
27     int num_scc = 0;
28     for (int i = n-1; i >= 0; i--) {
29         int v = order[i];
30         if (comp[v] == -1) dfs2(v, num_scc++);
31     }
32     return num_scc;
33 }

```

Gráfica de condensación

Una vez encontradas las SCCs, podemos construir la **gráfica de condensación**, una gráfica donde cada nodo representa una SCC y hay una arista de la SCC i a la SCC j si existe alguna arista en G de un nodo en i a un nodo en j . Esta gráfica siempre es un DAG (gráfica dirigida acíclica).



La gráfica de condensación es útil porque permite aplicar programación dinámica sobre el DAG para resolver preguntas sobre la gráfica original.

9.4. 2-SAT

Motivemos esta sección con este problema: The door problem. Se nos dice que hay n personas atrapadas en n cuartos. Cada cuarto tiene una puerta que puede estar bloqueada o desbloqueada. Hay m interruptores, activar o desactivar el interruptor cambia el estado de las puertas que controla. Cada puerta es controlada por dos interruptores. ¿Es posible operar los interruptores para que todas las puertas queden desbloqueadas al mismo tiempo?

Para resolver este problema, usaremos componentes fuertemente conexas, en este momento puede que el lector no vea la relación entre las *SCC* y el problema, pero esta sección buscará reducir el problema a *SCC*.

Supongamos tenemos n variables, cada una puede ser verdadera o falsa. Ahora, estas variables tienen restricciones, por ejemplo x_1 es verdadero o x_2 es verdadero. La pregunta es, ¿existe una asignación de verdadero/falso a todas las variables que satisfaga todas las restricciones?

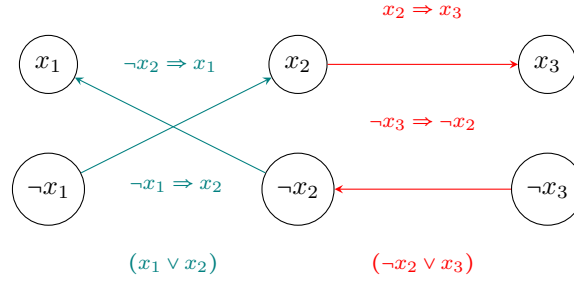
A diferencia del problema general de satisfacibilidad (que es NP-completo), cuando cada restricción involucra **exactamente dos variables** el problema se puede resolver en $O(V + E)$ usando *SCCs*. Este caso especial se llama **2-SAT**.

La observación fundamental que reduce este problema a *SCC* es que toda disyunción de 2 literales ($a \vee b$) es equivalente a dos implicaciones:

$$\neg a \Rightarrow b \quad \text{y} \quad \neg b \Rightarrow a$$

Si a es falso, entonces b debe ser verdadero (y viceversa).

Esto permite construir una gráfica dirigida con $2n$ nodos (x_i y $\neg x_i$ para cada variable) donde cada disyunción agrega dos aristas.



Una fórmula 2-SAT es satisfacible si y solo si en el grafo de implicaciones ninguna variable x_i está en la misma SCC que su negación $\neg x_i$. Si la fórmula es satisfacible, la asignación se obtiene directamente de los números de SCC que asigna Kosaraju:

- x_i = verdadero si $\text{comp}(x_i) > \text{comp}(\neg x_i)$
- x_i = falso si $\text{comp}(x_i) < \text{comp}(\neg x_i)$

Kosaraju numera las SCCs de forma que si existe una implicación de la SCC A hacia la SCC B , entonces A recibe un número menor que B . La SCC con número mayor es la “consecuencia” de la cadena de implicaciones. Asignar verdadero a la variable cuya SCC tiene número mayor garantiza que ninguna implicación se viola, siempre se va de falso hacia verdadero, nunca al revés.

En el problema de las puertas, cada puerta controlada por los interruptores a y b genera restricciones de exactamente esta forma.

Si la puerta está abierta, necesitamos que ambos interruptores que la controlan se activen (sean verdaderos) o que ninguno lo haga (sean falsos). Esto se expresa con dos restricciones:

- Si x_a es verdadero, x_b también debe serlo: $(\neg x_a \vee x_b)$
- Si x_b es verdadero, x_a también debe serlo: $(x_a \vee \neg x_b)$

Si está bloqueada, necesitamos que solamente uno de los interruptores se active.

- Si x_a es verdadero, x_b debe ser falso: $(\neg x_a \vee \neg x_b)$
- Si x_a es falso, x_b debe ser verdadero: $(x_a \vee x_b)$

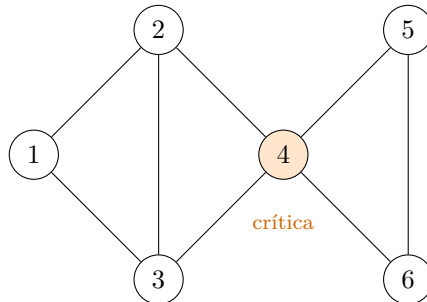
Para resolver el problema entonces basta con construir la gráfica, correr Kosaraju y verificar que ningún x_i esté en la misma SCC que $\neg x_i$.

Ejercicio. Implementar la solución del problema The door problem.

9.5. Puentes y puntos de articulación

Para esta sección, pensemos el siguiente problema llamado Network. En este problema, nos dicen que hay n centrales de telefonía conectadas por cables bidireccionales de forma que desde cualquier central se puede llegar a otra no necesariamente de forma directa. En ocasiones, puede haber un apagón en alguna central por lo que deja de operar. Esto puede causar que otras centrales queden incomunicadas entre sí, en tal caso se dice son centrales críticas. Nos piden encontrar cuántas centrales son críticas.

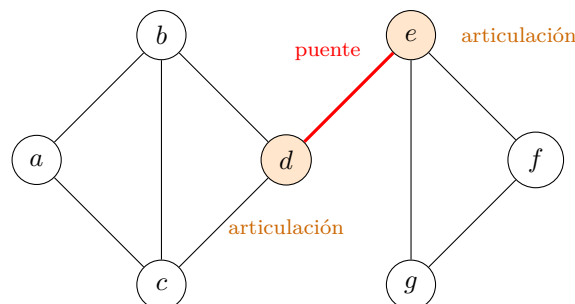
Nótese el siguiente ejemplo, si la central 4 falla, las centrales $\{5, 6\}$ quedan incomunicadas de $\{1, 2, 3\}$.



Definición 9.5.1 (Puente y punto de articulación). En un grafo conexo G :

- Una arista es un **puente** si al eliminarla el grafo se desconecta.
- Un vértice es un **punto de articulación** si al eliminarlo (junto con sus aristas incidentes) el grafo se desconecta.

En la siguiente gráfica, la arista roja $d-e$ es un puente. Los vértices d y e porque al eliminarlos desconectan la gráfica.



Regresando al problema de las centrales, notemos que el problema nos pide imprimir cuántos puntos de articulación tiene la gráfica dada. Podríamos intentar revisar cada vértice y ver si al quitarlo desconecta la gráfica con *BFS/DFS* pero esto costaría $O((V + E)V)$ lo que nos daría *TLE*, necesitamos algo más eficiente.

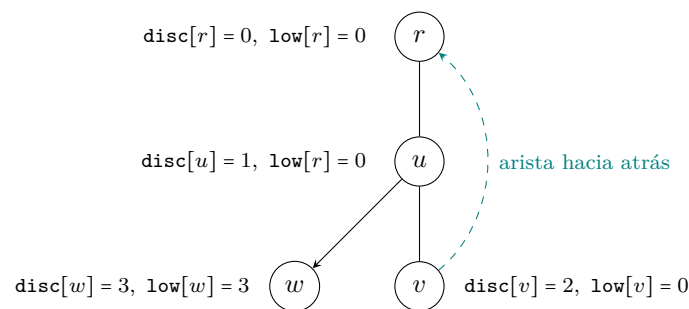
Al hacer DFS, el árbol de llamadas forma un árbol DFS. Las aristas que no forman parte de este árbol conectan un nodo con algún **ancestro** suyo les llamaremos aristas hacia atrás.

La idea para construir un algoritmo más eficiente es ver que un nodo U es punto de articulación si

tiene algún hijo V tal que **ningún nodo del subárbol de V puede llegar a un ancestro de U sin pasar por U** . Para revisar esto, asignamos a cada vértice un tiempo de descubrimiento $\text{disc}[v]$ que corresponde al orden en que el DFS lo visitó. Ahora definimos $\text{low}[v]$ como el menor disc alcanzable desde el subárbol de v :

$$\text{low}[v] = \min \begin{cases} \text{disc}[v] \\ \text{disc}[u] & \text{para toda arista hacia atrás } (v, u) \\ \text{low}[w] & \text{para todo hijo } w \text{ de } v \text{ en el árbol DFS} \end{cases}$$

Intuitivamente, $\text{low}[v]$ mide qué tan “arriba” puede escapar el subárbol de v sin usar la arista que lo conecta a su padre. En la siguiente gráfica v tiene $\text{low} = 0$ porque tiene una arista hacia atrás que le permite llegar a r que tiene $\text{low} = 0$ sin pasar por su padre. De forma similar, u hereda el $\text{low} = 0$ de su hijo v .



Para que una arista (uv) con v hijo de u en el DFS sea un puente, se debe cumplir que $\text{low}[v] > \text{disc}[u]$. Ya que eso significa que el subárbol de v no puede alcanzar a u sin pasar por la arista (u, v) . Dicho esto, es lógico que la condición para que un vértice u sea punto de articulación sea:

- Si u es la raíz del DFS, es punto de articulación si tiene ≥ 2 hijos en el árbol DFS.
- Si u no es raíz, es punto de articulación si tiene algún hijo v tal con $\text{low}[v] \geq \text{disc}[u]$.

Finalmente, la implementación nos quedaría:

```

1 vector<int> adj[MAXN];
2 int disc[MAXN], low[MAXN], cont = 0;
3 bool visited[MAXN], art[MAXN];
4 vector<pair<int, int>> bridges;
5
6 void dfs(int u, int parent) {
7     visited[u] = true;
8     disc[u] = low[u] = cont++;
9     int children = 0;
10
11     for (int v : adj[u]) {
12         if (!visited[v]) {
13             children++;
14             dfs(v, u);

```



```

15         low[u] = min(low[u], low[v]);
16
17         // Arista u-v es puente
18         if (low[v] > disc[u])
19             bridges.push_back({u, v});
20
21         // u es articulacion (caso no raiz)
22         if (parent != -1 && low[v] >= disc[u])
23             art[u] = true;
24
25     } else if (v != parent) {
26         // arista hacia atras
27         low[u] = min(low[u], disc[v]);
28     }
29 }
30 // u es articulacion (caso raiz)
31 if (parent == -1 && children >= 2)
32     art[u] = true;
33 }

```

Ejercicio. Implementar la solución completa de Network usando este algoritmo.

9.6. Árboles

Un **árbol** es un grafo conexo, es decir, todos sus vértices están conectados por un camino, sin ciclos. Esta definición es solo una de varias caracterizaciones equivalentes, todas igualmente válidas como definición y útiles en distintos contextos.

Para un grafo G con n vértices, las siguientes afirmaciones son equivalentes:

1. G es conexo y acíclico.
2. G es conexo y tiene exactamente $n - 1$ aristas.
3. Entre todo par de vértices existe exactamente un camino simple.

La caracterización (3), camino único entre todo par, será importante porque convierte preguntas sobre caminos en preguntas sobre estructura del árbol. Es la base del LCA y de la mayoría de técnicas sobre árboles.

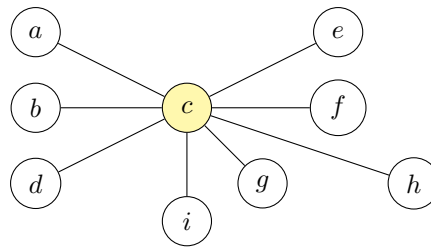
Para motivar este capítulo de árboles, veamos el problema Xenia and Tree, en este nos dan un árbol con n nodos. El nodo 1 está pintado de rojo y los demás de azul. Nos darán queries de dos tipos:

1. Nos dan un numero a que corresponde a un nodo azul y tenemos que pintarlo rojo.
2. Nos dan un numero b que corresponde a un nodo del árbol y tenemos que imprimir la distancia entre b y el nodo rojo más cercano.

Nótese que calcular la distancia a cada uno de los nodos rojos cada que nos dan una query no es eficiente y en este problema que nos pueden dar 10^5 queries en un árbol de 10^5 vértices, esta solución ingenua resultaría en *TLE*. Veremos algunas herramientas que nos ayudarían a resolver este problema.

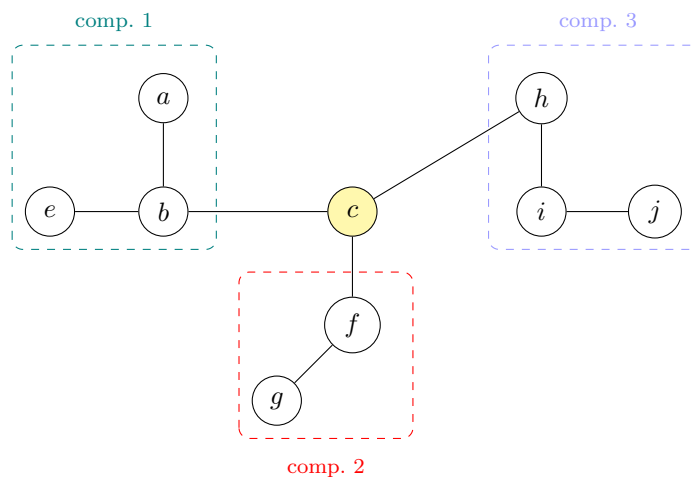
9.7. Centroide

El **centroide** de un árbol es un vértice c tal que al eliminarlo, ninguna componente conexa resultante tiene más de $\lfloor n/2 \rfloor$ vértices. Todo árbol tiene uno o dos centroides. Si tiene dos, son adyacentes. En la siguiente figura el vértice amarillo es el centroide, nótese que entonces si se elimina las componentes conexas se quedan con un vértice.



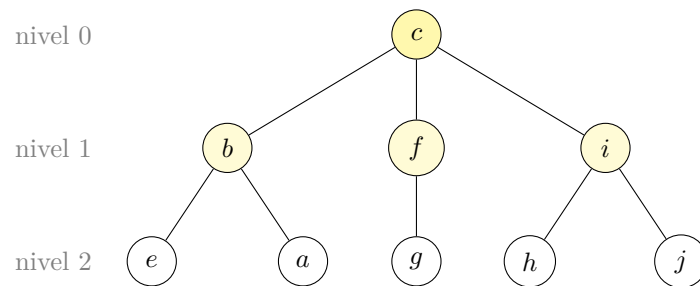
Descomposición por centroide

Tomemos un árbol más grande que el anterior y tomemos su centroide c . Al eliminarlo, el árbol se divide en varias componentes, cada una de tamaño $\leq \lfloor n/2 \rfloor$. Nótese que entonces cualquier camino de un nodo v en una componente a otro u en otra componente, este pasa por c , entonces si quisieramos la distancia entre u y v podemos sumar la distancia entre c y v y la distancia entre c y u .



La observación anterior solo resuelve caminos entre componentes distintas. Pero los nodos rojos también pueden estar en la misma componente que v . La solución es aplicar el mismo argumento recursivamente, cada componente tiene su propio centroide, y todo camino dentro de ella que no pasa por ese centroide, pasará por el centroide de alguna subcomponente más pequeña.

Esto define el **árbol de centroide**: tomamos el centroide c del árbol completo como raíz, recursamos en cada componente que queda al eliminarlo, y los centroides de esas componentes se vuelven hijos de c en el árbol de centroide.



Notemos entonces que para cualquier par de nodos u y v , existe un ancestro común en el árbol de centroide tal que el camino entre los nodos en el árbol original pasa por ese ancestro, el cual es el ancestro común de u y v , por ejemplo, para g y e en el árbol anterior, este es c , es decir el primer ancestro que comparten. En caso de e y a , es b .

Una propiedad importante es que el árbol de centroide tiene profundidad $O(\log n)$, ya que cada vez que bajamos un nivel, el subárbol tiene tamaño $\leq \lfloor n/2 \rfloor$ (por la propiedad del centroide). Tras k niveles el tamaño es $\leq n/2^k$. La recursión termina cuando el tamaño llega a 1, es decir a profundidad $\lceil \log_2 n \rceil$, esto es relevante para calcular la complejidad del algoritmo.

Volviendo al problema, con el árbol de centroide construido, mantenemos para cada nodo c :

$$\text{minDist}[c] = \min_{u \text{ rojo}} \text{dist}(c, u)$$

la distancia mínima desde c a cualquier nodo rojo en el subárbol *original* sobre el que c fue centroide. Inicialmente $\text{minDist}[c] = \infty$ para todo c . Ahora, cada que recibamos una query del tipo 1, en la que nos pidan pintar un vértice v de rojo, para todo nodo c ancestro de v en el árbol de centroide, debemos actualizar:

$$\text{minDist}[c] \leftarrow \min(\text{mindist}[c], \text{dist}(v, c))$$

Como hay $O(\log n)$ ancestros, esta operación cuesta $O(\log n)$ llamadas a **dist**.

Para el segundo tipo de query notemos que para cualquier nodo rojo u , existe un ancestro c de v en el árbol de centroide tal que el camino de v a u pasa por c . Entonces la respuesta es:

$$\min_{c \text{ ancestro de } v \text{ en árbol de centroide}} (\text{dist}(v, c) + \text{minDist}[c])$$

De nuevo $O(\log n)$ llamadas a **dist**.

Podría parecer que ya logramos optimizar la solución, sin embargo, estamos olvidando un detalle ¿cuánto cuesta calcular $\text{dist}(v, c)$?

La forma más directa es hacer un BFS o DFS desde c cada vez, pero eso cuesta $O(n)$ por llamada. Con $O(\log n)$ ancestros por query y m queries, el costo total sería $O(mn \log n)$ que para $n = m = 10^5$ da $\sim 10^{11}$ operaciones. Demasiado lento.

Necesitamos calcular $\text{dist}(u, v)$ en $O(\log n)$. Esto se logra con el **ancestro común más bajo** (LCA), que veremos en la siguiente sección.

La implementación para construir el árbol de centroide queda:

```

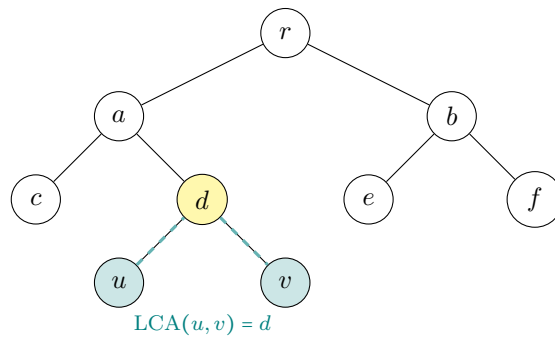
1  const int MAXN = 100005;
2  const int INF = 1e9;
3
4  vector<int> g[MAXN];    // grafica original
5  int sz[MAXN];         // tamano del subarbol en la componente actual
6  bool removed[MAXN];   // true si el nodo ya fue usado como centroide
7  int cpar[MAXN];        // padre en el arbol de centroide (-1 para la raiz)
8
9  // Calcula el tamano del subarbol de u en la componente actual
10 int calcSz(int u, int p) {
11     sz[u] = 1;
12     for (int v : g[u])
13         if (v != p && !removed[v])
14             sz[u] += calcSz(v, u);
15     return sz[u];
16 }
17
18 // Encuentra el centroide de la componente
19 int centroid(int u, int p, int treeSz) {
20     for (int v : g[u])
21         if (v != p && !removed[v] && sz[v] > treeSz/2)
22             return centroid(v, u, treeSz);
23     return u;
24 }
25
26 // Construye el arbol de centroide recursivamente
27 // u es cualquier nodo en la componente actual
28 // p es el padre en el arbol de centroide (-1 para la raiz)
29 void build(int u, int p) {
30     int total = calcSz(u, -1);    // tamano de esta componente
31     int c = centroid(u, -1, total); // encontrar su centroide
32     cpar[c] = p;                  // c es hijo de p en el arbol de centroide
33     removed[c] = true;            // marcar c como procesado
34     // Recursar en cada componente que queda al eliminar c
35     for (int v : g[c])
36         if (!removed[v])
37             build(v, c);
38 }

```

El patrón estándar para cualquier problema que use descomposición por centroide es este problema, subir por los $O(\log n)$ ancestros en el árbol de centroide y agregar o consultar información en cada uno. Lo que varía entre problemas es qué información se guarda y cómo se combina.

9.8. Ancestro común más bajo (LCA)

Dado un árbol con raíz, el **ancestro común más bajo** (LCA) de dos vértices u y v es el vértice más abajo que es ancestro de ambos.



La solución más directa que podríamos pensar es ver qué nodo está más bajo o profundo y llevarlo al mismo nivel que el otro subiendo de a un paso, luego subir ambos juntos hasta que coincidan. Por ejemplo para c y u en la figura de arriba, primero subiríamos u a su ancestro d y ya que ambos d y c están en el mismo nivel, los subiríamos a su ancestro a que como coincide, diríamos que este es su LCA .

Imaginemos que tenemos un árbol en forma de cadena $(1-2-3-\dots-n)$, calcular $LCA(1, n)$ cuesta $O(n)$ pasos. Con m queries, el costo total es $O(mn)$, lo que nos deja con el mismo problema que teníamos antes, necesitamos algo más eficiente. En lugar de subir de 1 en 1, ¿podríamos subir de a 2^k niveles de una vez?

Para subir d niveles desde un nodo v , podemos descomponer d en potencias de 2 (como en la representación binaria) y dar saltos de tamaño 1, 2, 4, 8, ... según corresponda. Si $d = 13 = 8 + 4 + 1$, damos un salto de 8, luego uno de 4, luego uno de 1, tres saltos en lugar de 13.

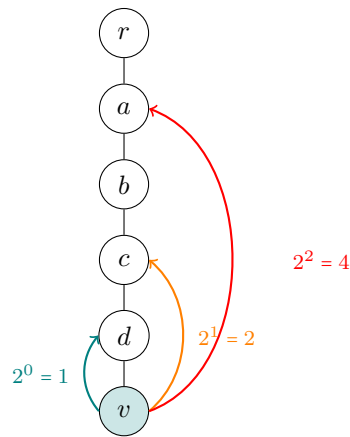
Entonces necesitamos precalcular, para cada nodo v y cada k :

$$\text{up}[v][k] = \text{ancestro de } v \text{ a distancia } 2^k$$

La forma de calcular esto será con una recurrencia, el ancestro a distancia 2^k es el ancestro a distancia 2^{k-1} del ancestro a distancia 2^{k-1} :

$$\text{up}[v][0] = \text{padre}(v), \quad \text{up}[v][k] = \text{up}[\text{up}[v][k-1]][k-1]$$

Como estamos subiendo en potencias de 2, $\lfloor \log_2 n \rfloor + 1$ niveles basta para cubrir cualquier distancia hasta n . En la siguiente imagen, vemos como para el árbol que es una cadena que mencionamos antes, esta forma de encontrar el ancestro sí resulta eficiente. Para el nodo v , los ancestros a distancia $2^0 = 1$, $2^1 = 2$ y $2^2 = 4$ se precálculan. Para subir 5 niveles, basta un salto de 4 y un salto de 1, solo 2 operaciones.



Lo que haremos entonces para calcular el *LCA* de dos nodos u y v será primero igualar sus profundidades. Si $\text{prof}(u) > \text{prof}(v)$, necesitamos subir u exactamente $d = \text{prof}(u) - \text{prof}(v)$ niveles. Descomponemos d en binario y usamos los saltos correspondientes.

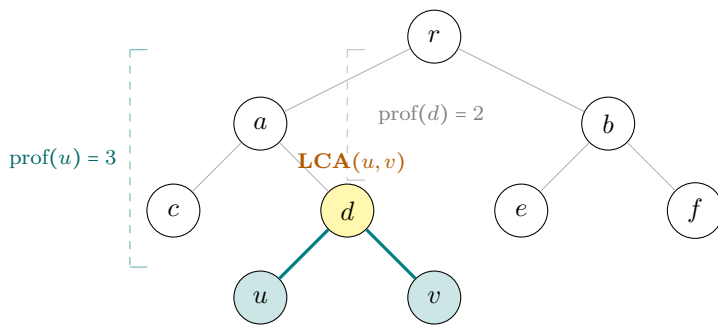
Una vez que coinciden, revisamos si $u = v$, así se cumple, ese es el LCA. De lo contrario, subimos ambos nodos simultáneamente, usando el salto más grande posible tal que $\text{up}[u][k] \neq \text{up}[v][k]$ (es decir, que no hayan convergido todavía). Después de este proceso, u y v son hijos del LCA.

```

1  const int LOG = 17;
2  int up[MAXN][LOG];
3  int prof[MAXN];
4
5  // Precalcular ancestros con DFS desde la raíz
6  void dfs(int v, int p, int d) {
7      up[v][0] = p;
8      prof[v] = d;
9      for (int k = 1; k < LOG; k++)
10         up[v][k] = up[up[v][k-1]][k-1];
11     for (int u : g[v])
12         if (u != p) dfs(u, v, d+1);
13 }
14
15 int lca(int u, int v) {
16     // Igualar profundidades
17     if (prof[u] < prof[v]) swap(u, v);
18     int diff = prof[u] - prof[v];
19     for (int k = 0; k < LOG; k++)
20         if ((diff >> k) & 1) u = up[u][k];
21
22     // revisar si ya encontramos el LCA
23     if (u == v) return u;
24
25     // Subirlos juntos
26     for (int k = LOG-1; k >= 0; k--)
27         if (up[u][k] != up[v][k]) {
28             u = up[u][k];
29             v = up[v][k];
30         }
31
32     // regresar el LCA
33     return up[u][0];
34 }

```

Notemos que para calcular la distancia entre dos nodos u y v , el camino de u a v sube desde u hasta el LCA d y luego baja hasta v . La distancia total es la suma de los dos tramos, que se simplifica a $\text{prof}(u) + \text{prof}(v) - 2 \cdot \text{prof}(\text{LCA})$.



$$\begin{aligned} \text{dist}(u, v) &= \underbrace{\text{prof}(u) - \text{prof}(d)}_{\text{subida}} + \underbrace{\text{prof}(v) - \text{prof}(d)}_{\text{bajada}} \\ &= \text{prof}(u) + \text{prof}(v) - 2 \cdot \text{prof}(d) \\ &= 3 + 3 - 2 \cdot 2 = 2 \end{aligned}$$

Finalmente, volviendo al problema, con $\text{dist}(u, v)$ implementado en $O(\log n)$, la solución completa consiste en:

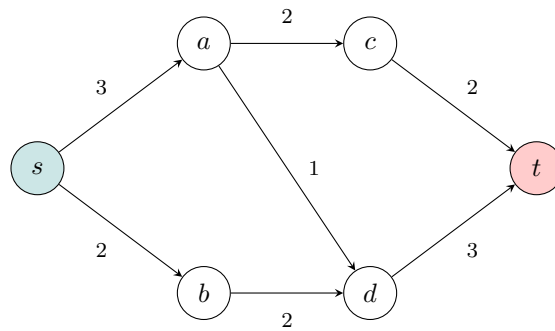
1. Preprocesar LCA con DFS. $O(n \log n)$.
2. Construir el árbol de centroide. $O(n \log n)$.
3. Para cada query de tipo 1 (pintar v), subir $O(\log n)$ ancestros en el árbol de centroide, calcular dist en $O(\log n)$ en cada uno. $O(\log^2 n)$.
4. Para cada query de tipo 2 (distancia mínima desde v), igual que el tipo 1, subir $O(\log n)$ ancestros en el árbol de centroide calculando dist en $O(\log n)$ en cada uno. $O(\log^2 n)$.

Nótese que la complejidad es $O(n \log n + m \log^2 n)$, que para $n = m = 10^5$ da $\sim 10^7$ operaciones, lo que sí es eficiente.

Ejercicio. Implementar la solución completa del problema Xenia and Tree (Codeforces 342E) usando lo visto en esta sección.

9.9. Flujo máximo

Imagina que tienes una red de servidores conectados por cables. Cada cable tiene un ancho de banda máximo (en MB/s). Quieres descargar un archivo de un servidor s a un servidor t lo más rápido posible. Puedes dividir la descarga en varias rutas que pasen información simultáneamente, pero ningún cable puede superar su capacidad. ¿Cuántos MB/s puedes transferir en total?



Lo que buscamos se llama el **flujo máximo** de s a t ; sin embargo antes del algoritmo, veremos algunas definiciones que nos serán de utilidad.

Definición 9.9.1 (Red de flujo). Una **red de flujo** es una gráfica dirigida $G = (V, E)$ con:

- Una función de **capacidad** $c : E \rightarrow \mathbb{R}_{\geq 0}$ (el ancho de banda de cada cable).
- Una **fuentes** s (servidor de origen) y un **sumidero** t (servidor de destino).

Un **flujo factible** es una función $f : E \rightarrow \mathbb{R}_{\geq 0}$ que satisface:

- **Capacidad:** $0 \leq f(u, v) \leq c(u, v)$ para toda arista, no se puede superar el ancho de banda del cable.
- **Conservación:** para todo $v \neq s, t$:

$$\sum_u f(u, v) = \sum_w f(v, w)$$

lo que llega a un servidor intermedio también sale de él.

El **valor del flujo** es $|f| = \sum_v f(s, v)$: cuántos MB/s salen de s en total.

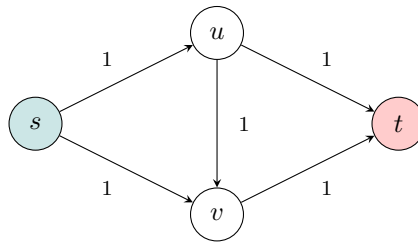
Definición 9.9.2 (Gráfica residual). Dado un flujo f , la **gráfica residual** G_f indica cuánta capacidad queda disponible para ajustar el flujo. Por cada cable (u, v) se construyen las siguientes aristas dirigidas si la capacidad residual correspondiente es mayor a 0:

- (u, v) con capacidad residual $c(u, v) - f(u, v)$, cuánto más puedo enviar por este cable.
- (v, u) con capacidad residual $f(u, v)$, cuánto flujo puedo “cancelar”, es decir redirigir a otra ruta.

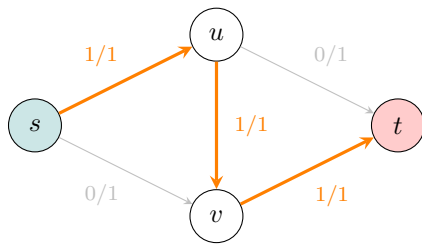
La arista hacia atrás es la idea clave que permite corregir decisiones anteriores. Si envié 2 MB/s por el cable (u, v) y encuentro una ruta mejor que pasa por (v, u) , puedo “devolver” hasta 2 MB/s de ese cable y redirigirlos.

9.9.1. Ford-Fulkerson

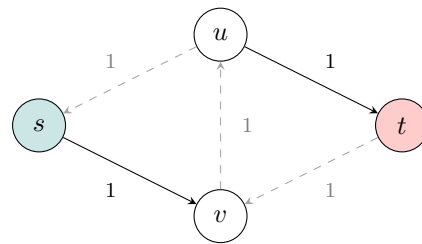
La idea de este algoritmo es tomar la gráfica residual y mientras exista una ruta de s a t con capacidad disponible, enviar flujo por ella. En la siguiente red, todas las capacidades son 1 y el flujo máximo es 2 (ruta $s \rightarrow u \rightarrow t$ y ruta $s \rightarrow v \rightarrow t$ simultáneamente).



Lo primero que haríamos sería buscar cualquier ruta de s a t con capacidad disponible. Encontramos $s \rightarrow u \rightarrow v \rightarrow t$ (capacidad 1 en cada cable). Enviamos 1 MB/s por esta ruta.



flujo actual = 1 MB/s

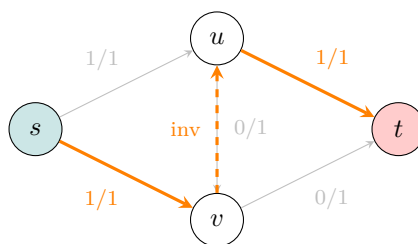


gráfica residual

(punteado = arista inversa)

Mirando el grafo residual, buscamos otra ruta de s a t . Vemos que $s \rightarrow v \rightarrow t$ no funciona porque $v \rightarrow t$ está saturado (capacidad residual 0).

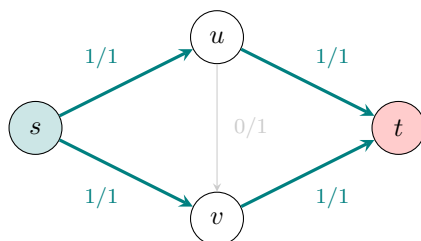
Pero notemos que la ruta $s \rightarrow v \rightarrow u \rightarrow t$ usa la **arista inversa** $v \rightarrow u$ y así “cancela” 1 MB/s que íbamos enviando de u a v , y lo redirige directamente de u a t .



Enviamos 1 MB/s por esta ruta y actualizamos:

Cable	Flujo antes	Cambio	Flujo después
$s \rightarrow v$	0	+1	1
$v \rightarrow u$ (inversa)	—	cancela $u \rightarrow v$	—
$u \rightarrow v$	1	-1	0
$u \rightarrow t$	0	+1	1
$s \rightarrow u$	1	sin cambio	1
$v \rightarrow t$	1	sin cambio	1

Nótese que entonces nos deja con el siguiente flujo en la red original:



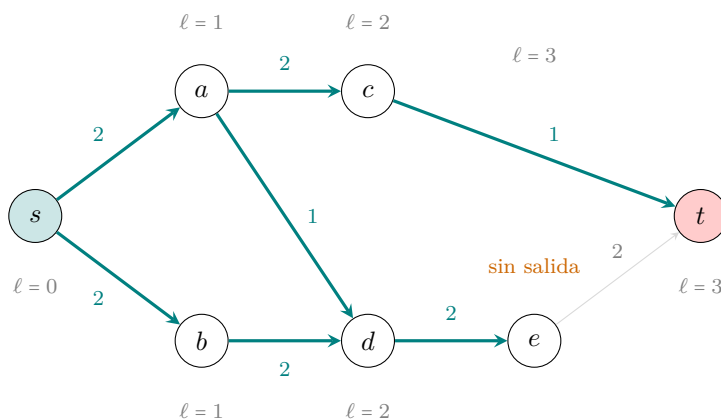
No hay más rutas disponibles en la gráfica residual, es aquí cuándo el algoritmo termina con flujo máximo = **2 MB/s**.

Notemos que con capacidades enteras, cada iteración aumenta el flujo en al menos 1, dando $O(|f^*| \cdot E)$ iteraciones donde $|f^*|$ es el flujo máximo. Si las capacidades son grandes esto es bastante lento, es por esto que veremos otro algoritmo que lo resuelve más rápido.

9.9.2. Dinic

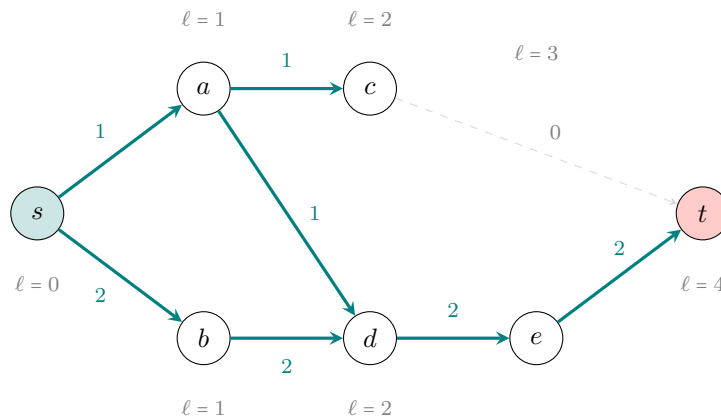
Dinic resuelve el problema de la cantidad de iteraciones eligiendo sistemáticamente los caminos **más cortos** primero y saturando **todos** los de esa longitud antes de pasar a los más largos. Veámos con el siguiente ejemplo cómo funciona:

Lo primero que haremos será correr un *BFS* desde s para calcular las distancias de s a los nodos y asignar niveles que representaremos con ℓ . Véase que t es alcanzable a distancia 3 (por $c \rightarrow t$) y también a distancia 4 (por $e \rightarrow t$), entonces $\ell(t) = 3$. Para que una arista (u, v) entre en la gráfica por niveles debe cumplir una regla: $\ell(v) = \ell(u) + 1$. La arista $e \rightarrow t$ no cumple, pues $(\ell(t) = 3 \neq \ell(e) + 1 = 4)$ por lo que se ignora.



Notemos que el único camino disponible es $s \rightarrow a \rightarrow c \rightarrow t$, el flujo que salga de este será entonces $\min(2, 2, 1) = 1$. El cable $c \rightarrow t$ queda saturado.

Ahora, el BFS en la gráfica residual ya no puede llegar a t por $c \rightarrow t$ (saturado). Ahora la ruta más corta a t pasa por e y tiene longitud 4:

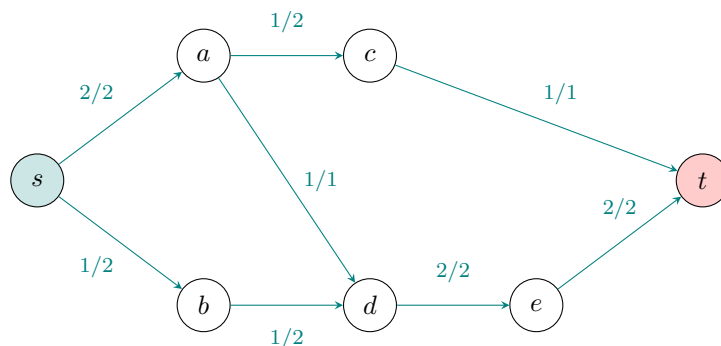


Lo que se hace a continuación es por medio de DFS, encontrar y saturar todos los caminos de longitud 4:

Para el camino $s \rightarrow a \rightarrow d \rightarrow e \rightarrow t$, revisamos el mínimo de las capacidades en el camino, $\min(1, 1, 2, 2) = 1$ y así agregamos 1 de flujo a cada arista en él, saturando $s \rightarrow a$ y $a \rightarrow d$.

Para el camino $s \rightarrow b \rightarrow d \rightarrow e \rightarrow t$, haremos lo mismo, recordemos que las capacidades de $d \rightarrow e$ y $e \rightarrow t$ tienen 1 restante porque usamos parte de su capacidad en el otro camino, entonces vemos el mínimo de las capacidades, $\min(2, 2, 1, 1) = 1$ y sumamos 1 al flujo de las aristas del camino.

Entonces la red se vería cómo:



Nótese que si quisiésemos hacer otra fase, el BFS ya no encontraría a t , por lo que el algoritmo concluye aquí. Para esta red el flujo máximo es 3. Con esto ya podemos resolver el problema inicial de servidores.

Nótese que como la longitud máxima es $|V| - 1$, hay a lo más $|V| - 1$ fases. Cada fase cuesta $O(VE)$, dando complejidad total $O(V^2E)$.

```

1 #define all(v) v.begin(),v.end()
2 const ll inf = 1e17;
3
4 struct flowEdge {
5     int u,v;
6     ll cap,flow = 0;
7     flowEdge(int u, int v, ll cap) : u(u), v(v), cap(cap) {};
8 };
9

```

```

10 struct Dinic{
11     vector<flowEdge> edges;
12     vector<vector<int>> adj;
13     int n,s,t;
14     int id = 0;
15     vector<int> level, next;
16
17     queue<int> q;
18     Dinic(int n, int s, int t) : n(n), s(s), t(t) {
19         adj.resize(n);
20         level.resize(n);
21         next.resize(n);
22         fill(all(level),-1);
23         level[s] = 0;
24         q.push(s);
25     }
26
27     void add(int u, int v, ll cap){
28         edges.emplace_back(u,v,cap);
29         edges.emplace_back(v,u,0);
30         adj[u].push_back(id++);
31         adj[v].push_back(id++);
32     }
33
34     bool bfs(){
35         while (!q.empty()){
36             int curr = q.front();
37             q.pop();
38             for (auto e : adj[curr]){
39                 if (edges[e].cap - edges[e].flow < 1) continue;
40                 if (level[edges[e].v] != -1) continue;
41                 level[edges[e].v] = level[edges[e].u] + 1;
42                 q.push(edges[e].v);
43             }
44         }
45         return level[t] != -1;
46     }
47
48     ll dfs(int u, ll flow){
49         if (flow == 0) return 0;
50         if (u == t) return flow;
51         for (auto& cid = next[u]; cid < adj[u].size(); cid++){
52             int e = adj[u][cid];
53             int v = edges[e].v;
54
55             if (level[edges[e].u] + 1 != level[v] || edges[e].cap - edges[e].flow <
1) continue;
56             ll f = dfs(v,min(flow,edges[e].cap - edges[e].flow));
57             if (f == 0) continue;
58             edges[e].flow += f;
59             edges[e ^ 1].flow -= f;
60             return f;
61         }
62         return 0;
63     }
64
65     ll maxFlow(){
66         ll flow = 0;
67         while (bfs()){
68             fill(all(next),0);
69             for (ll f = dfs(s,inf); f != 0ll; f = dfs(s,inf)) flow += f;

```

```

70         fill(all(level), -1);
71         level[s] = 0;
72         q.push(s);
73     }
74     return flow;
75 }
76
77 };

```

9.9.3. Teorema max-flow min-cut

Definición 9.9.3. Un **corte** s - t es una división de los vértices de V en dos partes (S, T) con $s \in S$ y $t \in T$. La **capacidad del corte** es la suma de las capacidades de los cables que van de S a T :

$$\text{cap}(S, T) = \sum_{\substack{u \in S, v \in T \\ (u, v) \in E}} c(u, v)$$

El **corte mínimo** es el de menor capacidad.

Un teorema muy importante es el **Teorema de Flujo Máximo y Corte Mínimo** que nos dice que el valor del flujo máximo de s a t es igual a la capacidad del corte mínimo s - t ¹.

Después de correr Dinic, si hacemos BFS en el grafo residual desde s . Los nodos alcanzables forman S y los no alcanzables T . Los cables saturados de S a T forman el corte mínimo.

Este teorema tiene consecuencias directas que usaremos en la sección siguiente de matching.

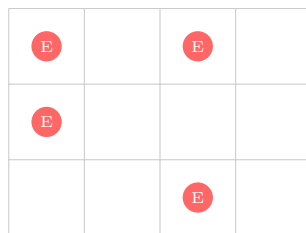
Una aplicación bonita que podemos darle es encontrar **caminos arista-disjuntos**, es decir, el número máximo de caminos de s a t que no comparten aristas. Este problema se traduce en buscar el mínimo número de aristas cuyo corte desconecta s de t . Se calcula dando capacidad 1 a cada arista y corriendo flujo máximo.

9.9.4. Matching máximo

Para motivar esta sección, veamos el siguiente problema, SAM I AM, en este problema, hay una cuadrícula $R \times C$ con N enemigos en ciertas celdas. Un cañón dispara en línea recta y elimina a todos los enemigos de una fila o columna. ¿Cuál es la mínima cantidad de disparos que se necesitan para eliminar a todos los enemigos y qué filas o columnas hay que disparar?

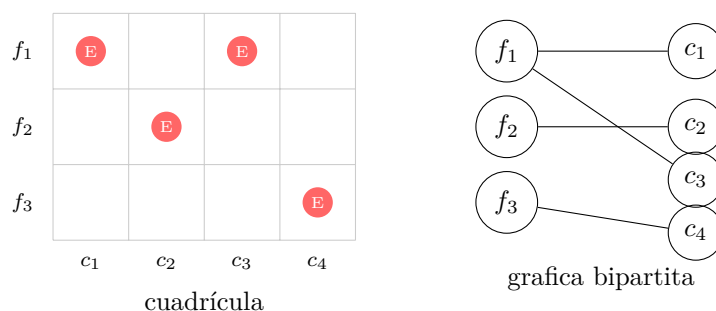
Podríamos intentar una estrategia greedy, es decir, intentar disparar las filas o columnas que tengan más enemigos, pero veámos en el siguiente ejemplo que pueden existir situaciones en las que alguna fila y columna tengan el mismo número de enemigos y no dé lo mismo cuál quitar. La solución optima es disparar la columna 1 y 3. Sin embargo, nuestro algoritmo greedy no tiene forma de distinguir entre la fila 1 y las columnas 1 y 3 entonces podría iniciar disparando la fila 1 y después, las filas 1 y 3, gastando 3 disparos.

¹Ver demostración



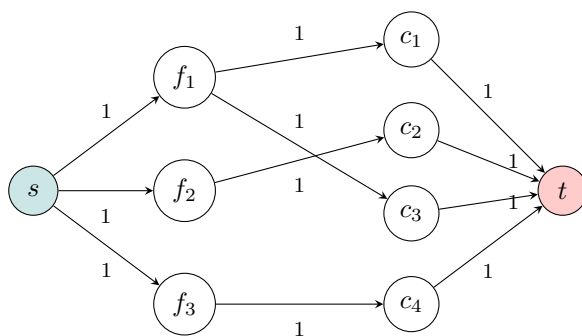
Necesitamos otra forma de pensar el problema. Podemos intentar modelarlo como una gráfica bipartita, este pensamiento surge de la idea que cada enemigo es cubierto si disparamos la fila r o la columna c .

- Lado izquierdo, un vértice por cada fila (R vértices)
- Lado derecho, un vértice por cada columna (C vértices)
- Por cada enemigo en (r, c) , una arista entre la fila r y la columna c



Nótese entonces que queremos seleccionar un conjunto de vértices S de forma que cualquier arista tenga al menos un extremo en S , a esto se le llama **cobertura de vértices**. Queremos entonces la cobertura de menor tamaño, en gráficas generales este problema es NP , pero en gráficas bipartitas como esta resulta que la clave para resolver el problema es usar flujo máximo.

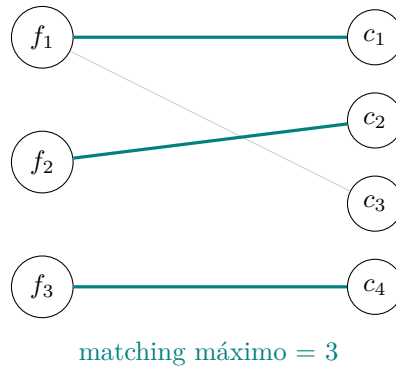
Esto puede parecer raro porque la gráfica que construimos no parece indicar por ningún lado un flujo, pero podemos construir la red agregando un nodo s conectado a cada fila y un nodo t conectado a cada columna. A todas las aristas les pondremos capacidad 1.



Si corremos Dinic en esta red, el flujo máximo resulta ser 3, que es exactamente el número mínimo de disparos que necesitamos. ¿Casualidad? Como veremos en esta sección, no lo es.

Definición 9.9.4. Un **matching** es un subconjunto de aristas tal que ningún vértice es extremo de más de una arista. El **matching máximo** es el de mayor cardinalidad.

Traducido al problema que estamos resolviendo, un matching en nuestra gráfica es un conjunto de enemigos tal que no hay dos en la misma fila ni en la misma columna. El matching máximo es el mayor conjunto así.



Hopcroft-Karp: Dinic en bipartitos

El matching máximo en una gráfica bipartita se puede calcular directamente con la red de flujo anterior usando Dinic. Pero como esta gráfica tiene estructura especial (bipartito, todas las capacidades 1), se puede optimizar, el algoritmo **Hopcroft-Karp** es exactamente Dinic aplicado a gráficas bipartitos, con complejidad $O(E\sqrt{V})$ en lugar de $O(V^2E)$.

```

1  const int INF = 1e9;
2  int n, m;
3  vector<int> adj[MAXN]; // adj[u] = vecinos de u (lado izquierdo)
4  int matchL[MAXN], matchR[MAXN]; // emparejamiento actual
5  int dist[MAXN];
6
7  bool bfs() {
8      queue<int> q;
9      for (int u = 1; u <= n; u++) {
10         if (!matchL[u]) { dist[u] = 0; q.push(u); }
11         else dist[u] = INF;
12     }
13     bool found = false;
14     while (!q.empty()) {
15         int u = q.front(); q.pop();
16         for (int v : adj[u]) {
17             int w = matchR[v];
18             if (!w) found = true;
19             else if (dist[w] == INF) { dist[w] = dist[u]+1; q.push(w); }
20         }
21     }
22     return found;
23 }
24
25 bool dfs(int u) {
26     for (int v : adj[u]) {

```

```

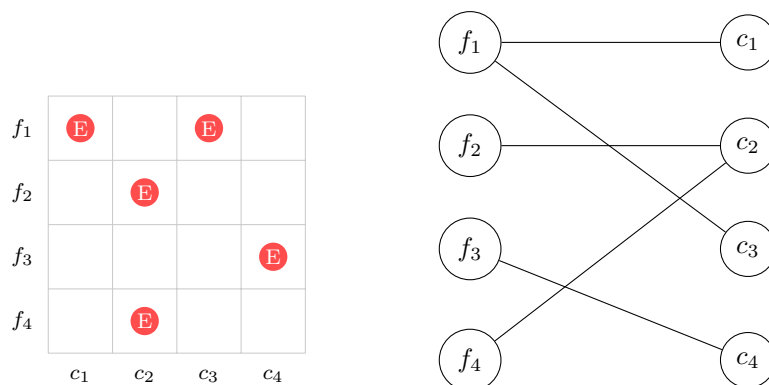
27     int w = matchR[v];
28     if (!w || (dist[w] == dist[u]+1 && dfs(w))) {
29         matchL[u] = v; matchR[v] = u;
30         return true;
31     }
32 }
33 dist[u] = INF;
34 return false;
35 }
36
37 int hopcroft_karp() {
38     fill(matchL+1, matchL+n+1, 0);
39     fill(matchR+1, matchR+m+1, 0);
40     int matching = 0;
41     while (bfs())
42         for (int u = 1; u <= n; u++)
43             if (!matchL[u] && dfs(u)) matching++;
44     return matching;
45 }

```

9.9.5. Teorema de König

El **teorema de König** nos dice que en una gráfica bipartita el matching máximo es igual a la cobertura mínima de vértices².

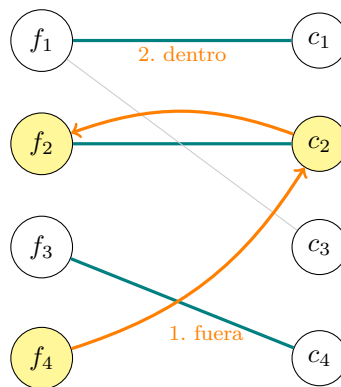
Veámos la red del problema de los cañones. El flujo máximo = matching máximo (cada unidad de flujo es un par emparejado). El corte mínimo = cubierta mínima (los vértices del corte son exactamente las filas/columnas que hay que disparar). Por el teorema anterior, son lo mismo, entonces calculado el flujo máximo, ¿cómo construir la cubierta mínima? Supongamos que tenemos esta cuadrícula con los enemigos:



Podemos usar Hopcroft-Karp o Dinic para encontrar el matching máximo de tamaño 3 con las aristas: f_1-c_1 , f_2-c_2 , f_3-c_4 .

Lo siguiente que haremos, será partir desde cada vértice libre del lado izquierdo, en este caso solo tenemos f_4 y alternar aristas fuera/dentro del matching. En la figura de abajo, usamos $f_4 \rightarrow c_2$ que está fuera del matching, luego $c_2 \rightarrow f_2$ que está dentro del matching y ya no tenemos a dónde movernos pues $c_2 \rightarrow f_2$ ya fue visitada.

²Ver demostración



Ahora, para construir la cubierta usaremos estos vértices que marcamos (los que quedaron en amarillo), la cubierta mínima se forma con:

- Los vértices del lado **izquierdo no marcados**
- Los vértices del lado **derecho marcados**

Vértice	Marcado	¿En la cubierta?
f_1	No	Sí (izq no marcada)
f_2	Sí	No
f_3	No	Sí (izq no marcada)
f_4	Sí	No
c_1	No	No
c_2	Sí	Sí (drch marcada)
c_3	No	No
c_4	No	No

De forma que la cubierta mínima es de tamaño 3 y consiste en $\{f_1, f_3, c_2\}$.

f_1	E		E	
f_2		E		
f_3				E
f_4		E		
	c_1	c_2	c_3	c_4

Ejercicio. Completar la solución del problema SAM I AM.

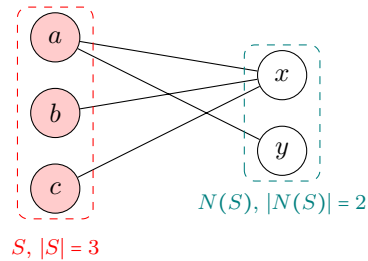
9.9.6. Teorema de Hall

Una pregunta natural que podría surgir es ¿cuándo existe un matching que empareja **todos** los vértices de un lado?

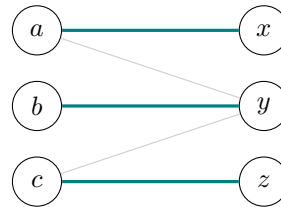
En un grafo bipartito con lados L y R , existe un matching que empareja todos los vértices de L si y solo si para todo $S \subseteq L$:

$$|N(S)| \geq |S|$$

donde $N(S)$ es el conjunto de vecinos de S en R .



No existe
matching perfecto



Matching perfecto existe

Matching de peso mínimo

Cuando las aristas tienen pesos y queremos el matching perfecto de costo mínimo (o máximo), se usa el **algoritmo Húngaro** (Kuhn-Munkres), con complejidad $O(n^3)$. Aparece en problemas de asignación óptima: dados n trabajadores, n tareas, y costos c_{ij} de asignar el trabajador i a la tarea j , minimizar el costo total. La mejor referencia que he encontrado para este algoritmo es este blog.

Bibliografía

- [1] cp-algorithms. *Sieve of Eratosthenes*.
<https://cp-algorithms.com/algebra/sieve-of-eratosthenes.html>
- [2] cp-algorithms. *Discrete Logarithm: when a and m are not coprime*.
<https://cp-algorithms.com/algebra/discrete-log.html#when-a-and-m-are-not-coprime>